

1

Repository

ひとことという

プロジェクトの全ての変更履歴をまとめて記録・管理する「保管庫」。

1

なぜ生まれた？

誰が・いつ・何を変更したか分からなくなり、「final_v2」「final_v2_修正後」のようなファイルが増殖していた。



2

それまでどうしていた？

フォルダごとコピーして、日付やバージョン名をファイル名に付けて管理していた。

	ファイルの手動保存	Repository
変更履歴の管理	ファイル名やコメントで手動管理	コミットとして自動記録
誰が変更したか	分からない・本人に聞くしかない	常に記録され追跡できる
過去に戻す	バックアップを探して復元	いつでも過去の状態を復元
共有方法	メールやチャットで送付	クローンしてそのまま共有

3

Repository で何ができるようになった？



変更履歴を丸ごと記録



いつでも過去の状態に戻せる



チームで一元管理・共有

関連キーワード

関連キーワード



Clone

リポジトリを丸ごと手元にコピーすること。

P.2



Commit

変更を記録として保存する単位。

P.5

一緒に覚えたい言葉



GitHub

リモートリポジトリを置いておけるWebサービス。



Working Tree

今実際に作業しているファイル群のこと。

2 Clone

ひとことというと

リモートリポジトリの中身を、変更履歴つきで手元に来るごとコピーしてくること。

1 なぜ生まれた？

他のPCやメンバーの環境に、同じプロジェクトを履歴ごと持ってくる方法が必要だった。



2 それまでどうしていた？

リポジトリの中身をZIPでダウンロードして展開していた（履歴は付いてこない）。

	ZIPダウンロード	Clone
手に入るもの	最新ファイルのみ	全履歴付きのコピー
更新の追従	そのつどダウンロードし直す	pullで差分だけ取得
変更の反映	できない・別送するしかない	push・pullで同期できる
初期セットアップ	展開してまた一からやり直し	git clone のひと言で完了

3 使用例

GitHub上のリポジトリを手元へ複製する例。

```
git clone https://github.com/example/repo.git
```

関連キーワード

関連キーワード



Repository
変更履歴をまとめて記録する保管庫。
P.1



Pull
クローン後、最新の変更を取り込む操作。
P.7

一緒に覚えたい言葉



Origin
cloneした際に自動で付くリモートの標準名。



SSH
安全に通信するための鍵認証の仕組み。

3

Fork

ひとことという

他人のリポジトリを、自分のGitHubアカウント配下に複製して自由に改造できるようにする機能。

1

なぜ生まれた？

編集権限のないリポジトリに直接手を加えることはできないが、自分なりに改造したり、OSSに貢献したい場面が増えていた。



2

それまでどうしていた？

コードをダウンロードして、別プロジェクトとしてゼロから作り直していた。

	コードの複製	Fork
元との関係	切れる（ただのコピー）	履歴・つながりを保ったまま複製
貢献のしやすさ	差分を伝える手段がない	Pull Requestで提案できる
編集権限	不要（そもそも別物になる）	不要（フォーク先は自分のもの）
元の更新の追従	手動で見比べるしかない	元リポジトリの変更を取り込める

3

Fork で何ができるようになった？



OSSへの貢献ができる



自分の実験場として自由に改造



元に迷惑をかけずに試せる

関連キーワード

関連キーワード



OSS

ソースコードが公開されたソフトウェア。

P.15



コントリビュート

プロジェクトへ改善を還元すること。

P.16

一緒に覚えない言葉



Pull Request

変更を元のリポジトリに取り込んでもらう提案。



Upstream

フォーク元の本家リポジトリを指す言葉。

4 Branch

ひとことについて

本流（main）から分かれて、他に影響を与えずに作業できる「並行世界」を作る仕組み。

1 なぜ生まれた？

新機能の開発中は本番のコードに触れなくなり、複数人が同時に違う機能を作ると変更同士がぶつかっていた。



2 それまでどうしていた？

フォルダをコピーして「app_new」のような名前で別々に作業していた。

	フォルダコピー	Branch
並行作業	フォルダを増やして管理	ブランチを分けるだけ
本流への影響	コピー作業中は本流を触れない	mainは常に安全なまま
統合方法	手作業で見比べてコピー	mergeで自動的に統合
切り替え	フォルダを開き直す	checkoutで一瞬

3 Branch で何ができるようになった？



複数人が同時並行で開発できる



実験して失敗しても main は無傷



完成したら merge で合流できる

関連キーワード

関連キーワード



Commit

ブランチ上に積み重ねる変更の記録。

P.5



Merge

分かれたブランチをひとつに統合する操作。

P.8

一緒に覚えたい言葉



main / master

プロジェクトの本流となるブランチの名前。



Checkout

作業対象のブランチを切り替える操作。

5

Commit

ひとことという

変更内容を、メッセージ付きで記録の1コマとして保存すること。

1 なぜ生まれた？

変更のまとまりを、理由つきで履歴に記録する単位が必要だった。



2 それまでどうしていた？

作業のキリのいいところで、ファイルをまるごと別名保存していた。

	別名保存	Commit
記録の単位	ファイル全体をまるごと	変更した差分だけ
変更理由	ファイル名や記憶に頼る	コミットメッセージに残す
巻き戻し	前のファイルを探し出す	特定のコミットにすぐ戻れる
差分確認	目視で見比べる	diffで一目瞭然

3 使用例

変更を記録する最小例。

```
git commit -m "fix: update login flow"
```

関連キーワード

関連キーワード



Branch

コミットを積み重ねる並行世界。

P.4



Push

コミットをリモートに送る操作。

P.6

一緒に覚えたい言葉



Staging Area

コミットする変更を仮に置いておく場所。



Diff

変更前後の差分を表示する仕組み。

6 Push

ひとことという

ローカルで積み上げたコミットを、リモートリポジトリに送って反映させること。

1 なぜ生まれた？

手元で進めた作業の内容を、チームやサーバーに反映する手段が必要だった。



2 それまでどうしていた？

修正したファイルをメールやチャットで送ったり、FTPでアップロードしていた。

	ファイル送付	Push
共有方法	メール・チャットで送付	コマンド一つで送信
反映されるもの	ファイルの最新版だけ	変更履歴ごと
衝突	気づかず上書きされがち	衝突があれば警告される
手間	そのつどファイルを探して送る	git push の一言

3 使用例

現在のブランチのコミットをリモートへ送る例。

```
git push origin main
```

関連キーワード

関連キーワード



Commit

pushする変更の記録の単位。

P.5



Pull

逆にリモートの変更を取り込む操作。

P.7

一緒に覚えない言葉



CI/CD

pushをきっかけに自動テスト・デプロイする仕組み。



Force Push

履歴を強制的に上書きするpush。取り扱い注意。

Pull

ひとことについて

リモートリポジトリの最新の変更を、自分の手元に取り込むこと。

1

なぜ生まれた？

チームメンバーが加えた変更を、自分の環境にも反映させる必要があった。



2

それまでどうしていた？

最新ファイルを送ってもらい、自分の変更と手作業で見比べながら統合していた。

	手動マージ	Pull
取得方法	送ってもらう・都度依頼	git pull で自動取得
統合作業	目視で見比べて手作業	fetch + merge で自動統合
最新性	聞かないと分からない	いつでも最新を取得できる
衝突対応	気づきにくい	コンフリクトとして明示される

3

使用例

リモートの最新変更を取り込む例。

```
git pull origin main
```

関連キーワード

関連キーワード



Push

自分の変更をリモートに送る操作。

P.6



Merge

pull の中で行われている統合処理。

P.8

一緒に覚えたい言葉



Fetch

リモートの変更を取得だけしてmergeしない操作。



Pull Request

コードレビューを経て取り込む提案の仕組み。

8 Merge

ひとことというと

分かれていたブランチの変更を、ひとつに統合すること。

1 なぜ生まれた？

ブランチで分かれて進めた作業を、最終的に1つにまとめる必要があった。



2 それまでどうしていた？

2つのファイルを並べて見比べ、必要な部分を手でコピペして1つにまとめていた。

	手動統合	Merge
統合方法	目視 + 手作業でコピペ	コマンドで自動統合
衝突検出	気づかず上書きしがち	コンフリクトとして明示
履歴	統合の過程は残らない	マージコミットとして記録
対象範囲	ファイル単位	プロジェクト全体を一括統合

3 Merge で何ができるようになった？



ブランチの変更を安全に統合



衝突箇所だけ集中して確認できる



統合の記録が履歴に残る

関連キーワード

関連キーワード



Branch

mergeで統合される側の並行世界。

P.4



Rebase

mergeとは別の、もう一つの統合方法。

P.9

一緒に覚えたい言葉



Conflict

自動で統合できず人の判断が必要になる状態。

P.11



Fast-forward

分岐がない場合に、そのままつなげる統合方法。

9

Rebase

ひとことというと

ブランチの土台を最新の main に乗せ替えて、履歴を一直線に整えること。

1 なぜ生まれた？

merge を繰り返すと履歴が枝分かれだらけになり、変更の流れが後から追いにくなっていた。



2 それまでどうしていた？

merge でブランチを統合していた（統合の跡が履歴に残り、次第に複雑になる）。

	Merge	Rebase
履歴の見たい	枝分かれして複雑	一直線できれい
元のコミット	そのまま残る	新しいコミットとして積み直す
統合方法	最後にまとめて合流	土台から作り直して合流
向いている場面	チームの統合記録を残したい	自分のブランチをきれいに整えたい

3 使用例

作業ブランチを main の最新の上に載せ直す例。

```
git fetch origin
git rebase origin/main
```

関連キーワード

関連キーワード



Merge

rebase と並ぶ、もう一つの統合方法。

P.8



Branch

土台を乗せ替える対象そのもの。

P.4

一緒に覚えたい言葉



HEAD

今いるコミットの位置を指す目印。



Squash

複数のコミットを1つにまとめる操作。

Cherry-pick

ひとことという

他のブランチにある特定の1コミットだけを、狙って自分のブランチに取り込むこと。

1 なぜ生まれた？

別ブランチの修正のうち1つだけを今のブランチにも反映したいのに、mergeだと全部ついてきてしまっていた。



2 それまでどうしていた？

該当の修正をファイルを開いて目視で探し、コピーして手動で反映していた。

	手動コピー	Cherry-pick
取り込む範囲	ブランチ全体を merge	狙った1コミットだけ
探す手間	ファイルを見比べて探す	コミット履歴から指定するだけ
履歴	反映元が分からなくなる	新しいコミットとして記録される
用途	全機能をまとめて統合したい	緊急のバグ修正だけに反映したい

3 使用例

別ブランチの特定コミットだけ取り込む例。

```
git cherry-pick a1b2c3d
```

関連キーワード

関連キーワード

- 

Commit
cherry-pickで摘み取る対象の単位。
P.5
- 

Branch
摘み取った先の並行世界。
P.4

一緒に覚えたい言葉

- 

Hotfix
本番の緊急バグを直すための修正のこと。
- 

SHA
コミットを一意に識別するハッシュ値。

11

Conflict

ひとことというと

同じ箇所を複数人が別々に変更していて、Gitが自動で統合できなくなった状態。

1 なぜ生まれた？

複数人が同じファイルの同じ箇所を同時に編集すると、片方の変更が知らないうちに消えてしまうことがあった。



2 それまでどうしていた？

後から保存した人のファイルで上書きされ、先の変更は静かに消えていた。

	無自覚な上書き	Conflict
検出	気づかれない	明示的に警告される
消える変更	静かに消えてしまう	両方残った上で選べる
解決方法	誰かに聞いて復元する	該当箇所を見て手で選ぶ
発生タイミング	上書き保存の瞬間	merge・pull時に判明する

3 Conflict で何ができるようになった？



変更が知らずに消えなくなる



衝突箇所だけピンポイントで確認



どちらを残すか自分で選べる

関連キーワード

関連キーワード



Merge

コンフリクトが表面化しやすい統合操作。

P.8



Pull

コンフリクトが判明するもう一つの場面。

P.7

一緒に覚えない言葉



マージツール

衝突箇所を見比べながら解決を助けるツール。



Diff

変更前後の違いを比較する表示形式。

Revert

ひとことというと

過去のコミットの変更を打ち消す、新しいコミットを作って元に戻すこと。

1 なぜ生まれた？

間違った変更を公開ブランチに反映してしまった後、安全に取り消したい場面があった。



2 それまでどうしていた？

何を変えたか思い出しながら、手で逆の修正を書いて元に戻していた。

	手動巻き戻し	Revert
元に戻す方法	記憶を頼りに手で書き直す	対象コミットを指定するだけ
履歴	間違いも修正も跡が残らない	取り消した記録として残る
安全性	共有後の履歴改変が必要になりがち	履歴を書き換えずに安全に戻せる
対象範囲	ファイル単位で手動対応	コミット単位で正確に指定

3 使用例

過去コミットを打ち消す新しいコミットを作る例。

```
git revert HEAD
```

関連キーワード

関連キーワード

- **Reset**
revertとよく比較される巻き戻し操作。
P.13
- **Commit**
revertが打ち消す対象の単位。
P.5

一緒に覚えたい言葉

- **ロールバック**
システムを以前の状態に戻す一般的な言葉。

13

Reset

ひとことというと

コミットの位置を過去に戻し、それ以降の変更をなかったことにする（または手元に戻す）こと。

1 なぜ生まれた？

誤ったコミットや手元の変更を、履歴の単位で安全に戻す操作が必要だった。



2 それまでどうしていた？

バックアップフォルダから昔のバージョンのファイルを探し出して上書きしていた。

	旧ファイル探し	Reset
戻し方	昔のファイルを探して上書き	コミットを指定して戻すだけ
戻る範囲	フォルダ単位でバラバラ	リポジトリ全体を正確に指定
変更の扱い	上書きすると消えてしまう	オプションで手元に残せる
共有後の利用	特に区別なく使ってしまう	共有済みの履歴では避けるべき

3 使用例

直前コミットだけ外し、変更は手元に残す例。

```
git reset --soft HEAD~1
```

関連キーワード

関連キーワード



Revert

resetより安全な、共有履歴向けの巻き戻し。

P.12



Commit

resetで戻す先を指定する単位。

P.5

一緒に覚えたい言葉



HEAD

今いるコミットの位置を指す目印。



Working Tree

今実際に作業しているファイル群のこと。

14

Tag

ひとことというと

特定のコミットに「v1.0.0」のような名前を付けて、目印として残すこと。

1 なぜ生まれた？

どのコミットが実際にリリースしたバージョンなのか、後から分からなくなっていた。



2 それまでどうしていた？

リリースのたびにプロジェクト全体をコピーして「release_v1.0」フォルダとして保管していた。

	フォルダ複製保管	Tag
目印の付け方	フォルダを丸ごとコピーして保管	該当コミットに名前を付けるだけ
容量	バージョンごとに増え続ける	履歴の中の目印だけなので軽い
探しやすさ	フォルダを漁って探す	タグ名で一発検索
用途	手元の目印だけ	リリースノートやCIの起点にも使える

3 使用例

リリース版の目印を付ける例。

```
git tag v1.0.0
git push origin v1.0.0
```

関連キーワード

関連キーワード



Commit

タグを貼り付ける対象そのもの。

P.5



Repository

タグごと履歴を保管する場所。

P.1

一緒に覚えたい言葉



セマンティックバージョンニング

v1.2.3のようにバージョンを意味づけて付けるルール。



リリースノート

そのバージョンで何が変わったかをまとめた文書。

ひとことという

ソースコードを公開し、誰でも自由に閲覧・改造・再配布してよいソフトウェアのこと。

1 なぜ生まれた？

ソフトウェアの中身を検証・改良できず、利用者が改善に参加できない状況を変える必要があった。



2 それまでどうしていた？

企業が作った非公開のソフトウェアを購入し、決められた機能をそのまま使っていた。

	非公開ソフトウェア	OSS
ソースコード	非公開	公開されていて誰でも読める
改造	できない	自由に改造・再配布できる
コスト	購入・ライセンス費用がかかる	多くが無料で使える
開発体制	1社が管理	世界中の開発者が共同で改善

3 誰がいつ作った？

作った人 Eric Raymond・Bruce Perens ら

時期 1998年

文脈 「Open Source Software」という呼び名が広まり、自由ソフトウェアの考え方を実務に伝えやすくなった。

関連キーワード

関連キーワード



Fork

OSSに手を加えるための最初の一步。

P.3



コントリビュート

OSSに改善を還元する行為そのもの。

P.16

一緒に覚えたい言葉



ライセンス

そのソフトウェアの利用・改変・再配布のルール。



メンテナー

OSSプロジェクトの管理・運営を担う人。

コントリビュート

ひとことでいうと

OSSなどのプロジェクトに対して、コードやドキュメントの改善を提案・還元すること。

1 なぜ生まれた？

OSSを使うだけで、見つけた改善をプロジェクトに返す流れが弱かった。



2 それまでどうしていた？

不具合を見つけても、手元だけで直して自分のプロジェクトの中で使っていた。

	受け取るだけ	コントリビュート
改善の反映先	自分の手元だけ	元のプロジェクト全体
方法	自己流にファイルを書き換える	ForkしてPull Requestを送る
恩恵	自分だけが得る	同じ問題で困る全員が得る
評価	特に残らない	GitHub上に活動履歴として残る

3 コントリビュートで何ができるようになった？



世界中のユーザーと一緒に育てられる



実務経験として実績が形に残る



レビューを通じて技術力が伸びる

関連キーワード

関連キーワード



Fork

コントリビュートの第一歩となる複製。

P.3



OSS

コントリビュートの対象になるプロジェクト。

P.15

一緒に覚えたい言葉



Pull Request

変更を提案してレビューしてもらう仕組み。



Issue

バグ報告や要望を管理する場所。

Client

ひとことというと

サーバーに要求（リクエスト）を送る側。ブラウザやスマホアプリなど、自分が直接触っている画面のこと。

1 なぜ生まれた？

アプリケーションを「要求する側」と「処理する側」に分けずに設計すると、機能が増えるほど役割の境界があいまいになり、どこで何を処理しているかを追いにくくなっていた。



2 それまでどうしていた？

システムを役割分担のない一枚岩として扱い、要求する部分と応答する部分を明確に区別していなかった。

	区別しない頃	クライアント/サーバーで理解
見え方	システム全体がひとかたまり	要求する側 = クライアントと分かる
実体	漠然としたイメージ	ブラウザ・アプリ・CLI等具体的な形
役割	切り分けられない	表示・入力・要求の送信を担当

3 Client で何ができるようになった？



画面の表示や入力を担当できる



サーバーに要求を送れる



サーバーとの役割分担を説明できる

関連キーワード

対になる技術



Server

クライアントからの要求を受け取り、処理して返す側。

P.22

やり取りの中身



Request

クライアントがサーバーに送る依頼のデータ。

P.23



Response

サーバーがリクエストに対して返す結果。

P.24

Server

ひとことというと

クライアントからの要求（リクエスト）を受け取り、処理して結果を返す側のプログラムや機械。

1 なぜ生まれた？

1台のマシンの中で要求を受け付ける役割と処理する役割が混在していると、負荷が集中した際にどこがボトルネックかを切り分けにくかった。



2 それまでどうしていた？

「重い処理をしてくれる場所」という程度の理解にとどまり、担うべき役割が明確に定義されていなかった。

	ふわっとした理解	サーバーの役割で理解
正体	ブラックボックス	要求を処理して返すプログラム
形	1台の機械だと思込む	クラウド上の仮想サーバーのことも多い
落ちる原因	わからず終い	プロセス停止・リソース枯渇等具体的に絞れる

3 Server で何ができるようになった？



リクエストを受け取って処理できる



複数のクライアントに同時対応できる



クラウド上でも同じ役割を果たせる

関連キーワード

対になる技術



Client

サーバーに要求を送る側。ブラウザやアプリなど。

P.21

関連キーワード



Request

クライアントがサーバーに送る依頼のデータ。

P.23

Request

ひとことというと

クライアントがサーバーに送る「これをやってほしい」という依頼のデータ。

1 なぜ生まれた？

通信の内容（宛先・方法・付随データ）を明示的に構造化しないと、送っている情報の全体像を後から追跡できなかった。



2 それまでどうしていた？

通信は「送れば何となく届くもの」として扱われ、中身の構造が特に定義されていなかった。

	ブラックボックス操作	リクエストとして理解
意識	画面の変化だけ見る	送っている中身（宛先・内容）を意識する
中身	わからない	メソッド・URL・ヘッダー・ボディで構成
デバッグ	勘に頼る	リクエスト内容を確認して原因を特定

3 Request で何ができるようになった？



通信の中身を確認できる



デバッグの手がかりにできる



API仕様書を読めるようになる

関連キーワード

対になる技術



Response

リクエストに対してサーバーが返す結果。

P.24

構成要素



Header

リクエストやレスポンスに添える付加情報。

P.34

Response

ひとことという

サーバーがリクエストに対して返す結果。ステータスコードと中身（ボディ）がセットになっている。

1 なぜ生まれた？

処理結果を単なるデータの塊として返すだけでは、成功したのか失敗したのかをクライアント側が機械的に判断できなかった。



2 それまでどうしていた？

結果の良し悪しを画面の見た目だけで判断しており、機械が読み取れる形式の情報は用意されていなかった。

	見た目だけで判断	レスポンスの中身で判断
判断材料	画面の見た目	ステータスコード＋ボディの中身
原因特定	勘	200 番台/400 番台/500 番台で切り分け
対応	とりあえず再読み込み	中身を見て原因を絞り込んでから対応

3 Response で何ができるようになった？



結果を正確に受け取れる



ステータスコードで切り分けられる



エラーの原因を特定しやすくなる

関連キーワード

対になる技術



Request

クライアントがサーバーに送る依頼のデータ。

P.23

関連キーワード



HTTP

リクエスト・レスポンスをやり取りする共通ルール。

P.25

HTTP

ひとことについて

クライアントとサーバーがリクエスト・レスポンスをやり取りするための共通ルール（プロトコル）

1 なぜ生まれた？

クライアントとサーバーが異なるメーカー・異なる実装で作られていても通信できるようにするには、共通のやり取りのルールが必要だった。



2 それまでどうしていた？

通信のたびに実装ごとの独自の手順でやり取りしており、互換性のない方式が乱立していた。

	ルールを意識しない	HTTPというルールで理解
認識	通信は自動でつながるもの	共通ルールに従ってやり取りしている
やり取り	なんとなく届く	メソッド・ヘッダーで構造化されている
トラブル対応	わからず終い	仕様に沿って原因を切り分けられる

3 誰がいつ作った？

作った人	Tim Berners-Lee ら
時期	1990年前後
文脈	World Wide Web の文書転送プロトコルとして設計された。

関連キーワード

発展形



HTTPS

HTTPに暗号化（TLS）を追加した安全な通信。

P.26

構成要素



Request

クライアントがサーバーに送る依頼のデータ。

P.23



Response

サーバーがリクエストに対して返す結果。

P.24

HTTPS

ひとことという

HTTPの通信をTLSで暗号化したもの。途中で盗み見・改ざんされにくくする仕組み。

1 なぜ生まれた？

通信内容が経路の途中で盗み見・改ざんされる恐れがあり、パスワードやカード番号を安全に送る手段が必要だった。



2 それまでどうしていた？

HTTPのまま平文でやり取りしており、通信経路上で内容を読み取られたり書き換えられたりする恐れがあった。

	HTTP（暗号化なし）	HTTPS（TLSで暗号化）
通信内容	素通し・覗き見できる	暗号化されて中身が読めない
改ざん	途中で書き換えられる恐れ	改ざんされると検知できる
信頼できるか	見た目では判断できない	証明書で相手が本物か確認できる

3 誰がいつ作った？

作った人	Netscape ら
時期	1994年ごろ
文脈	HTTPにTLS/SSLを組み合わせ、安全なWeb通信を行う形として普及した。

関連キーワード

元になった技術



HTTP

暗号化されていない通信プロトコル。

P.25

仕組みを支える技術



TLS

HTTPSの暗号化・認証を担う仕組み。

P.37



SSL証明書

通信相手が本物であることを証明する電子証明書。

P.38

DNS

ひとことというと

ドメイン名（www.example.comなど）をIPアドレスに変換する「インターネットの電話帳」。

1 なぜ生まれた？

サーバーの住所であるIPアドレスは数字の羅列で覚えにくく、人間が扱いやすい名前アクセスできる仕組みが必要だった。



2 それまでどうしていた？

接続先をIPアドレスの数字で直接指定するしかなく、覚えるにも共有するにも不便だった。

	名前解決を知らない頃	DNSとして理解
アクセス方法	ドメイン名を打つだけ	裏でIPアドレスに変換されている
障害時の見え方	なぜか繋がらない、で終わる	名前解決の失敗として切り分けられる
管理	意識しない	Aレコード・CNAME等設定を理解する

3 誰がいつ作った？

作った人	Paul Mockapetris
時期	1983年
文脈	IPアドレスと名前を対応づける分散的な仕組みとして設計された。

関連キーワード

関連キーワード



Domain

DNSで管理される、覚えやすい名前そのもの。

P.28



IP Address

DNSが変換した先にある、機械の住所。

P.29

Domain

ひとことという

インターネット上の住所（IPアドレス）に付けられた、人間が読める名前。

1 なぜ生まれた？

IPアドレスが変わってもアクセス先を一定に保ちたい、複数のサービスを覚えやすい名前で区別したいという要求があった。



2 それまでどうしていた？

サービスへのアクセスをIPアドレスの数字に依存しており、サーバーを移設するたびにアクセス方法が変わってしまっていた。

	名前を意識しない頃	ドメインとして理解
認識	リンクを辿るだけの存在	取得・更新して管理する資産
構造	ただの文字列	www.example.comのような階層構造
費用	無料だと思っていた	年単位でレジストラに更新料を払う

3 Domain で何ができるようになった？



覚えやすい名前で
発信できる



DNSに登録して
IPアドレスと紐付けられる



自分の名前として
管理・更新できる

関連キーワード

関連キーワード



DNS

ドメイン名をIPアドレスに変換する仕組み。

P.27



SSL証明書

ドメインが本物であることを証明する電子証明書。

P.38

IP Address

ひとことという

インターネットに繋がる機器1台1台に割り振られる、数字だけの住所。

1 なぜ生まれた？

インターネットに繋がる無数の機器の中から通信相手を一意に特定する、共通の住所の仕組みが必要だった。



2 それまでどうしていた？

機器同士の接続先を特定する統一的な方法がなく、ネットワークごとに独自の識別方法に頼っていた。

	住所を意識しない頃	IPアドレスとして理解
接続の仕組み	なんとなく繋がる	正確な住所を指定して接続している
種類	区別なし	グローバルIP / プライベートIPの違い
トラブル対応	原因不明	pingやIPで疎通確認ができる

3 IP Address で何ができるようになった？



機器を一意に特定できる



グローバル/プライベートを区別できる



pingなどで疎通確認ができる

関連キーワード

関連キーワード



DNS

ドメイン名をIPアドレスに変換する仕組み。

P.27



Port

同じIPアドレスの中で、アプリを区別する番号。

P.30



NAT

プライベートIPとグローバルIPを変換する仕組み。

P.41

Port

ひとことについて

同じIPアドレス（住所）の中で、どのアプリ・サービス宛かを区別する番号。

1 なぜ生まれた？

1台のサーバーでWebサーバーやメールサーバーなど複数のサービスを同時に動かすには、IPアドレスだけでは宛先を区別できなかった。



2 それまでどうしていた？

1台のマシンには1つのサービスしか動かせない前提で運用されており、複数サービスを同時提供する方法が確立されていなかった。

	ポートを意識しない頃	ポートとして理解
宛先の指定	IPアドレスだけで届くと思う	IPアドレス + ポート番号で届く
複数サービス	1台1サービスだと思う	同じ機械で複数サービスを同時待受
トラブル対応	原因不明	ポート番号の衝突・未開放に気づける

3 Port で何ができるようになった？



同じ機械で複数サービスを区別できる



アプリ宛にピンポイントで届けられる



衝突や未開放のトラブルに気づける

関連キーワード

関連キーワード



IP Address

機器そのものの住所。ポートはその中の区分け。

P.29



TCP

ポート番号を使って通信を確立するプロトコル。

P.35

Cookie

ひとことについて

ブラウザに保存される小さなデータ。サーバーがユーザーを識別するために使う。

1 なぜ生まれた？

HTTPは1回のやり取りごとに関係がリセットされる仕組みのため、ログイン状態などをリクエストをまたいで維持する方法が必要だった。



2 それまでどうしていた？

リクエストごとに関係がリセットされ、ページを移動するたびにログイン情報などの状態が失われていた。

	状態を覚える仕組みがない頃	Cookieで状態を覚える
ページ移動	毎回ログアウトされる	ログイン状態を維持できる
保存場所	なし	ブラウザ側に小さなデータとして保存
できること	都度ログインが必要	カート内容やログインを継続できる

3 Cookieで何ができるようになった？



ログイン状態を維持できる



一定期間データを保持できる



サーバーがユーザーを識別できる

関連キーワード

関連キーワード



Session

Cookieを使って実現される、ログイン状態などの管理。

P.32



JWT

署名付きトークンでログイン情報を持ち運ぶ方式。

P.33

Session

ひとことというと

ログインから終了までの、ユーザーとサーバーの一連のやり取りをひとまとまりとして管理する仕組み。

1 なぜ生まれた？

Cookie だけでは重要な情報をブラウザ側に平文で持たせることになり、改ざんや漏えいのリスクなくユーザーの状態を管理する方法が必要だった。



2 それまでどうしていた？

ユーザーの状態を示す情報をすべてブラウザ側のCookieに直接持たせており、内容を書き換えられるリスクがあった。

	Cookie だけで完結すると思う頃	セッションとして理解
情報の置き場所	ブラウザ (Cookie) だけ	サーバー側にも状態を保持
Cookie の役割	情報そのものを持つ	セッションIDという鍵だけを持つ
期限切れ	原因不明で困る	有効期限やログアウトで切れると理解

3 Session で何ができるようになった？



ログインから終了までを管理できる



サーバー側で状態を安全に保持できる



期限切れの挙動を理解できる

関連キーワード

関連キーワード



Cookie

セッションIDなどを保存する、ブラウザ側の小さなデータ。

P.31



JWT

サーバー側に状態を持たない、別の認証情報の持ち方。

P.33

ひとことについて

ユーザー情報を署名付きのトークンに詰め込んで持ち運ぶ、サーバー側に状態を持たない認証方式。

1 なぜ生まれた？

サーバーを複数台に増やして負荷分散すると、どのサーバーがリクエストを受けてもログイン状態を判定できる仕組みが必要になった。



2 それまでどうしていた？

ログイン状態をサーバー側のセッションストアで管理しており、複数サーバー間で情報を共有する仕組みが必要だった。

	セッション方式	JWT
状態の保持場所	サーバー側（セッションストア）	トークン自体に情報が入っている
複数サーバー対応	セッション共有の仕組みが必要	トークンさえあればどのサーバーでも検証可能
改ざん対策	サーバーが正本を持つので安心	署名で改ざんを検知する仕組みが必須

3 JWT で何ができるようになった？



サーバー側に状態を持たず認証できる



複数サーバー間で認証を共有しやすい



署名で改ざんを検知できる

関連キーワード

関連キーワード



Session

サーバー側で状態を管理する、もう一つの方式。

P.32



Cookie

JWTをブラウザに保存する際の入れ物としても使われる。

P.31



Header

JWTは多くの場合 Authorization ヘッダーに載せて送る。

P.34

Header

ひとことという

リクエストやレスポンスの本体（ボディ）とは別に付け加える、付加情報の集まり。

1 なぜ生まれた？

本文（データそのもの）とは別に、認証情報やデータ形式などの付加情報を通信のたびにやり取りする仕組みが必要だった。



2 それまでどうしていた？

やり取りされる情報は本文のみを想定しており、認証情報や形式指定などの付加情報を載せる場所が定義されていなかった。

	本文だけを意識する頃	ヘッダーも意識する
やり取りする情報	本文（ボディ）だけ	本文 + 付加情報（ヘッダー）
認証情報	どこに入れるか分からない	Authorizationヘッダーに載せる
トラブル対応	原因が本文しか見えない	Content-Type等ヘッダーからも原因を探れる

3 Header で何ができるようになった？



本文とは別に情報を付け加えられる



認証トークンなどを安全に運べる



通信のトラブルを切り分けやすくなる

関連キーワード

関連キーワード



Request

ヘッダーが付加される、依頼データそのもの。

P.23



JWT

Authorizationヘッダーに載せて送る認証トークン。

P.33

TCP

ひとことというと

データが確実に届いたかを確認しながら通信するプロトコル。信頼性を重視する。

1 なぜ生まれた？

ネットワーク上でデータが途中で失われたり順序が入れ替わったりする中で、送った内容が確実に、正しい順番で届くことを保証する仕組みが必要だった。



2 それまでどうしていた？

データを送りっぱなしにしており、途中で欠落したり順序が入れ替わったりしても検知・再送する手段がなかった。

	確認する仕組みを知らない頃	TCPとして理解
届いたかの確認	考えたことがない	受信確認（ACK）をしながら通信する
順序	気にしていない	送った順番通りに届くよう保証する
速度と引き換えに	特に意識しない	確認作業がある分、UDPより多少遅い

3 TCPで何ができるようになった？



データの到達を確認しながら送れる



順序を保証して届けられる



信頼性が必要な通信に向いている

関連キーワード

対になる技術



UDP

確認を取らず、とにかく速さを優先するプロトコル。

P.36

関連キーワード



Port

TCPが通信相手のアプリを特定するために使う番号。

P.30

UDP

ひとことという

データが届いたかの確認をせず、とにかく速く送ることを優先するプロトコル。

1 なぜ生まれた？

動画配信やリアルタイム通信では、確認応答による遅延よりも速度が優先され、多少のデータ欠落を許容してでも高速に送る仕組みが求められた。



2 それまでどうしていた？

TCPのように確認応答を取りながら通信しており、リアルタイム性が求められる用途では確認のオーバーヘッドが遅延として無視できなかった。

	TCPの発想しか知らない頃	UDPとして理解
届いたかの確認	必ず確認するものだと思う	確認せず送りっぱなしにできる
速度	確認作業の分、多少遅れると思う	確認がない分、圧倒的に速い
向いている場面	特に区別しない	動画配信・ゲーム等多少の欠落を許容できる場面

3 UDP で何ができるようになった？



確認を省いて高速に送れる



リアルタイム性が求められる通信に向く



多少のデータ欠落を許容できる

関連キーワード

対になる技術



TCP

確認を取りながら確実に届けるプロトコル。

P.35

ひとことについて

通信を暗号化し、相手が本物であることも確認する仕組み。HTTPSの安全性を支えている。

1

なぜ生まれた？

旧来のSSLに脆弱性が発見され、暗号方式を強化しつつ互換性を保った新しい暗号化の仕組みが必要になった。



2

それまでどうしていた？

SSLという名称の暗号化方式が使われていたが、既知の脆弱性が積み重なり安全性を維持できなくなっていた。

	SSL (旧世代)	TLS (後継)
名前	SSL 2.0/3.0	TLS 1.0〜1.3として進化
安全性	脆弱性が見つかり非推奨に	暗号方式を強化して安全性を維持
現状の呼び方	慣習で“SSL証明書”と呼ばれ続ける	実体はTLSで通信している

3

TLS で何ができるようになった？



通信内容を暗号化できる



通信相手が本物か確認できる



古いバージョンを無効化して安全性を保てる

関連キーワード

関連キーワード



HTTPS

TLSによって暗号化されたHTTP通信。

P.26



SSL証明書

TLS通信で相手の身元を証明する電子証明書。

P.38



Let's Encrypt

無料でSSL証明書を発行してくれる認証局。

P.39

SSL 証明書

ひとことという

サイトの運営者が本物であることを証明する電子証明書。TLS通信で使われる。

1 なぜ生まれた？

通信を暗号化するだけでは、接続先のサーバーが本物かどうかまでは保証できず、なりすましを見抜く手段が必要だった。



2 それまでどうしていた？

接続先が本物かどうかを、URLの見た目やドメイン名の印象だけで判断するしかなかった。

	見ただ目で判断する頃	証明書で判断
本人確認	URLの見た目だけで判断	認証局が発行した証明書で確認
期限	意識しない	有効期限があり、切れると警告が出る
取得方法	存在を知らない	認証局に申請、または自動発行サービスを使う

3 SSL 証明書 で何ができるようになった？



サイト運営者の身元を証明できる



なりすましサイトと区別できる



TLSによる暗号化通信の土台になる

関連キーワード

関連キーワード



TLS

この証明書を使って暗号化通信を行う仕組み。

P.37



Let's

Encrypt

この証明書を無料・自動で発行してくれるサービス。

P.39

39

Let's Encrypt

ひとことというと

SSL 証明書を無料・自動で発行してくれる認証局。個人開発のサイトも簡単に HTTPS 化できる。

1

なぜ生まれた？

SSL 証明書の取得に費用と手続きがかかるため、個人サイトや小規模プロジェクトでは常時 HTTPS 化が後回しにされがちだった。



2

それまでどうしていた？

SSL 証明書は有料の認証局に申請し、書類確認などの手続きを経て取得するものだった。

	有料・手続き重視の時代	Let's Encrypt
費用	有料が当たり前	無料で発行できる
発行手続き	書類・審査に時間がかかる	ドメイン所有の自動確認のみで発行
更新	手動で更新申請	自動更新の仕組みを組める（有効期限は短め）

3

Let's Encrypt で何ができるようになった？



無料で SSL 証明書を発行できる



発行から設定まで自動化できる



個人サイトでも気軽に HTTPS 化できる

関連キーワード

関連キーワード



SSL証明書

Let's Encryptが発行してくれる証明書そのもの。

P.38



HTTPS

この証明書によって実現される暗号化通信。

P.26

VPN

ひとことという

インターネット上に暗号化された専用の通信路（トンネル）を作り、離れた場所からでも安全に社内ネットワークなどに接続する仕組み。

1 なぜ生まれた？

拠点や自宅など、離れた場所から社内ネットワークへ安全に接続する手段が求められたが、専用線の敷設には多額の費用がかかった。



2 それまでどうしていた？

社外から社内ネットワークへアクセスするには、専用線を敷設するか、直接出社するしかなかった。

	専用線・出社前提	VPN
接続方法	専用線の敷設や出社が必要	インターネット経由で仮想的な専用線を作る
コスト	専用線は高額	ソフトウェアで低コストに実現
安全性	物理的に隔離されている	暗号化されたトンネルで安全性を確保

3 VPN で何ができるようになった？



外部から社内ネットワークに安全に接続できる



通信を暗号化してトンネルを作れる



リモートワークでも同じ環境で作業できる

関連キーワード

関連キーワード



NAT

プライベートIPとグローバルIPを交換する仕組み。

P.41



Proxy（フォワードプロキシ）

クライアントの代わりに通信を中継する仕組み。

P.43

NAT

ひとことというと

家庭や社内など、限られた場所だけで使うプライベートIPと、インターネット上のグローバルIPを変換する仕組み。

1 なぜ生まれた？

インターネットに接続する機器の数に対して、割り当て可能なグローバルIPアドレスの数が不足していた。



2 それまでどうしていた？

接続する機器1台ごとに個別のグローバルIPアドレスを割り当てる前提で運用されていた。

	1台1グローバルIPだ と思う頃	NATとして理解
IPの割り当て	機器ごとに別のグローバルIPが必要	家庭内はプライベートIP、外向きは共通のグローバルIP
IPアドレス不足	特に意識しない	限られたグローバルIPを節約できる
見え方	各機器が別々に見える と思う	外からは同じグローバルIPの中の1台に見える

3 NAT で何ができるようになった？



プライベートIP
とグローバルIP
を変換できる



限られたグローバルIPを節約できる



内部のネットワーク構成を外部から隠せる

関連キーワード

関連キーワード



IP Address

NATが変換する対象、プライベート/グローバルの住所。

P.29



サブネット (Subnet)

プライベートIPを整理して区切る仕組み。

P.42

サブネット (Subnet)

ひとことというと

1つのネットワークを、用途や部署などの単位で複数の小さなネットワークに分割する仕組み。

1 なぜ生まれた？

ネットワーク全体を1つの区画のまま運用すると、障害の影響範囲が広がりやすく、用途ごとのアクセス制御も難しかった。



2 それまでどうしていた？

ネットワークを1つの大きな区画としてまとめて管理しており、用途や部署ごとに細かく制御する手段がなかった。

	1つの大きなネットワーク	サブネットで分割
管理単位	全部まとめて1つ	用途ごと（公開用・非公開用等）に分割
障害の影響範囲	全体に波及しやすい	範囲を区切って影響を限定できる
セキュリティ	一律の設定しかできない	区画ごとに細かくアクセス制御できる

3 サブネット（Subnet）で何ができるようになった？



ネットワークを用途ごとに分割できる



区画ごとにアクセス制御できる



障害やトラブルの影響範囲を限定できる

関連キーワード

関連キーワード



IP Address

サブネットが分割の対象とする住所そのもの。

P.29



NAT

プライベートIPとグローバルIPを交換する仕組み。

P.41

Proxy（フォワードプロキシ）

ひとことという

クライアントの代わりにインターネットへアクセスし、結果を取り次いでくれる中継役のサーバー。

1 なぜ生まれた？

社内の全端末から個別に外部へ直接アクセスすると、アクセス制御やログ管理を端末ごとに行う必要があり、統制が取れなかった。



2 それまでどうしていた？

クライアントがそれぞれ直接インターネット上のサーバーに接続しており、アクセス制御やキャッシュを一括で行う仕組みがなかった。

	直接接続しか知らない頃	プロキシを経由すると理解
接続経路	クライアントから直接サーバーへ	クライアント→プロキシ→サーバーと中継
メリット	特に意識しない	アクセス制御・キャッシュ・匿名化ができる
見え方	個々の機器がそのまま外に見える	外からはプロキシだけが見える

3 Proxy（フォワードプロキシ）で何ができるようになった？



クライアントの代わりに通信を中継できる



アクセス制限やログ管理をまとめて行える



内部のクライアント構成を外部から隠せる

関連キーワード

関連キーワード



VPN

暗号化されたトンネルで社内ネットワークに接続する仕組み。

P.40



NAT

プライベートIPとグローバルIPを変換する仕組み。

P.41

イントラネット

ひとことでいうと

インターネットと同じ技術を使いながら、社内など限られた範囲だけで閉じたネットワークのこと。

1 なぜ生まれた？

社外に出せない情報を、社内だけで安全に共有するネットワークが必要になった。



2 それまでどうしていた？

紙や社内サーバの共有フォルダだけで情報を回しており、Webの便利さは社外のサイトだけのものだった。

	閉じた共有だけだった頃	イントラネットがある世界
情報の置き場	紙・共有フォルダ・口頭	社内Web・ポータルで参照
アクセス範囲	物理的に社内にいる人だけ	認証付きで社内網から閲覧
更新のしやすさ	配布や貼り替えが必要	ページを直せばすぐ反映
外との関係	社外ネットとは別物	同じWeb技術で内側だけ閉じる

3 イン트라ネット で何ができるようになった？



社内向けのWebサービスを置ける



部外者に見せずに情報共有できる



インターネットと同じ技術で学べる

関連キーワード

関連キーワード



VPN

外からイントラネットへ安全に入るトンネル。

P.40



Proxy

社内から外へ出るときの中継役。

P.43

JSON

ひとことについて

{ }と[]だけで表現する、JavaScriptのオブジェクトそのままの見た目のデータ形式。

1 なぜ生まれた？

APIやシステム間でのデータ交換にXMLを使うと、開始・終了タグの記述量が多くなり、パース処理も複雑になりがちだった。より簡潔に書けるデータ形式が求められていた。



2 それまでどうしていた？

2000年代まではXMLで構造化データをやりとりするのが主流だった。

	XML	JSON
書き方	開始・終了タグが必須	{ }と[]だけ
読みやすさ	ネストが深いと追いつらい	JSのオブジェクトと同じ感覚
パース	専用パーサーが必要	JSならそのまま扱える

3 誰がいつ作った？

作った人 Douglas Crockford

時期 2001年前後

文脈 JavaScript由来の軽量データ交換形式として整理・普及した。

関連キーワード

比較する技術



XML

タグで構造を表現する古参フォーマット。今も設定や仕様書で現役。

P.54



YAML

インデントで構造を表す、設定ファイル向けの書き方。

P.52

よく使う場面



API

JSONはAPIのレスポンス形式として最も広く使われている。

P.61

YAML

ひとことというと

波括弧を使わず、インデント（字下げ）だけで階層構造を表現するデータ形式。

1 なぜ生まれた？

設定ファイルをJSONで書くとコメントを残せず、括弧やカンマの対応関係も見落としやすかった。人が手で書きやすく、意図を書き添えられる設定形式が求められた。



2 それまでどうしていた？

JSONでも設定は書けるが、コメントが書けず、括弧やカンマの数え間違いも起きやすい。

	JSON	YAML
見た目	{ } と、が並ぶ	インデントのみ
コメント	書けない	# で書ける
向いている場面	API のレスポンス	手で書く設定ファイル

3 YAML で何ができるようになった？



手書きしやすい設定ファイルが書ける



コメントを残しながら設定できる



GitHub Actions や CI の定義に使われる

関連キーワード

比較する技術



JSON

括弧で構造を表すデータ形式。API レスポンスで定番。

P.51



TOML

YAML よりシンプルな階層表現を目指した設定形式。

P.53

TOML

ひとことという

key = value の並びで設定を書く、あいまいさの少ないシンプルな設定ファイル形式。

1 なぜ生まれた？

パッケージ管理ツールの設定ファイルには、複雑な入れ子構造よりも誰が読んでも一意に解釈できるシンプルな記法が向いていた。あいまいさの少ない設定形式が求められた。



2 それまでどうしていた？

YAML は表現力が高い分、インデントの深さやリストの書き方に複数の流儀があり、ツールによって解釈が割れることがあった。

	YAML	TOML
書き方の自由度	高い（複数の書き方がある）	低い（迷いにくい）
ネスト表現	インデントで表現	[section] で区切る
向いている場面	複雑な設定・CI 定義	ツールの設定ファイル 1 枚

3 TOML で何ができるようになった？



key = value で直感的に書ける



Cargo.toml などパッケージ設定の定番



書き方のブレが少なく読み間違いにくい

関連キーワード

比較する技術



YAML

インデントで階層構造を表す設定ファイル形式。

P.52



JSON

括弧で構造を表すデータ形式。プログラムからの生成向き。

P.51

ひとことというと

<tag>で囲んでデータの意味を表現する、開始・終了タグが特徴のマークアップ形式。

1 なぜ生まれた？

システム間でデータをやり取りする際、値だけでなく意味や階層構造まで一緒に表現できる、共通のマークアップ形式が求められていた。



2 それまでどうしていた？

JSON が普及する前は、Web API も設定ファイルも XML で書くのが当たり前だった。

	XML 登場前 (プレーンテキスト等)	XML
構造の表現	決まったルールがない	タグで階層と属性を表現
検証	独自ルールで確認	スキーマ (DTD/XSD) で検証できる
今の立ち位置	—	設定・帳票・SOAP 系 API で現役

3 XML で何ができるようになった？



タグ名だけで意味が伝わる可読性



スキーマで構造を厳密に検証できる



帳票やドキュメント形式として今も現役

関連キーワード

比較する技術



JSON

括弧で構造を表す軽量なデータ形式。今のAPIの主流。

P.51



スキーマ (Schema)

データの形を定義し、正しいかどうかを検証する仕組み。

P.56

CSV

ひとことについて

値をカンマ(,)で区切って1行1レコードを表す、最もシンプルな表形式のデータ形式。

1 なぜ生まれた？

表形式のデータをExcelやスプレッドシートでそのまま開けるようにするには、JSONやXMLのような入れ子構造を持たない、単純な区切り文字形式が必要だった。



2 それまでどうしていた？

JSONやXMLのような入れ子構造を表すデータ形式に対して、CSVは表形式のデータをそのまま渡したいときに使われる。

	JSON	CSV
構造	入れ子（ネスト）を表現できる	1行1レコードの平坦な表のみ
開きやすさ	エディタやコードで確認	Excelやスプレッドシートでそのまま開ける
向いている場面	APIのレスポンス	ダウンロードデータ・大量データの受け渡し

3 CSV で何ができるようになった？



Excelやスプレッドシートでそのまま開ける



大量データの入出力・ダウンロードに使う



仕組みが単純で他システムとも連携しやすい

関連キーワード

比較する技術



JSON

括弧で構造を表す軽量のデータ形式。ネスト構造も表現できる。

P.51

スキーマ (Schema)

ひとことでいうと

データがどんな項目を持ち、どんな型・制約であるべきかを定義した"データの設計図"。

1

なぜ生まれた？

サーバーに送られてくるデータの形式が保証されていないと、必須項目の欠落や型の不一致に気づけず、エラーが起きてから原因を調べることになっていた。データの形をあらかじめ定義しておく仕組みが必要だった。



2

それまでどうしていた？

スキーマがない状態では、データの形が正しいかどうかを目視やその場しのぎのif文で確認するしかなかった。

	スキーマなし	スキーマあり
形式チェック	目視やif文で都度確認	定義に沿って自動で検証
ドキュメント	コードを読まないといけない	スキーマ自体が仕様書になる
変更の影響	気づかず壊れることがある	違反がすぐエラーとして分かる

3

スキーマ (Schema) で何ができるようになった？



データの形を自動で検証できる



必須項目・型・制約を明文化できる



DBやAPI、設定ファイルなど幅広く使う

関連キーワード

関連する語



XML

タグで構造を表現するデータ形式。DTD/XSDというスキーマ機構を持つ。

P.54



OpenAPI

APIの入出力の形をスキーマとして定義する仕様。

P.65

API

ひとことという

アプリやサービス同士が、決められたルールに沿ってやり取りするための“窓口”。

1 なぜ生まれた？

外部サービスの機能やデータを自分のアプリから利用したいとき、画面をスクレイピングする以外に構造化された取得手段がなかった。



2 それまでどうしていた？

外部のサービスが持つ機能やデータを使いたいとき、以前は画面ごと取り込む（スクレイピング）しかなかった。

	画面のスクレイピング	API
取得する対象	見た目込みのHTML全体	必要なデータだけ
安定性	デザイン変更で簡単に壊れる	仕様が変わらない限り安定
やり取りの形	人が読む前提のページ	プログラムが読む前提のデータ

3 API で何ができるようになった？



他社のサービスの機能をそのまま使える



決まったルールでデータだけをやり取り



フロントとサーバーの間の窓口にもなる

関連キーワード

関連する語



Endpoint

APIの窓口の中の、個別の入り口となるURL。

P.62



REST

APIの設計スタイルの1つ。URLと動詞で操作を表す。

P.63



GraphQL

必要なデータだけを1回で取得できるAPIの仕組み。

P.64

Endpoint

ひとことというと

APIの中で、特定の機能やデータにアクセスするための個別のURL。

1 なぜ生まれた？

APIを1つの窓口としてまとめて捉えようと、機能ごとにどこへリクエストを送ればよいかが分かりにくく、個々の入り口を明確に区別する必要があった。



2 それまでどうしていた？

APIを「1つの大きな窓口」とだけ捉えていると、実際にどこにリクエストを送ればいいのか迷う。

	APIをひとまとめに捉える	Endpointという単位で捉える
粒度	サービス全体	機能ごとのURL (/users, /posts 等)
見つけ方	ドキュメントを一から読む	仕様書で機能とURLが対応している
変更の影響	APIが変わったと大雑把に捉える	どのエンドポイントが変わったか特定できる

3 Endpoint で何ができるようになった？



機能ごとにURLを分けて設計できる



/users や /posts のような単位で管理



ドキュメントやログでも扱いやすい単位

関連キーワード

関連する語



API

アプリ同士がやり取りするための窓口全体。

P.61



slug

URLの一部に使われる、人にも読める識別子。

P.67



REST

URLと動詞で操作を表現するAPI設計スタイル。

P.63

REST

ひとことという

URL（名詞）とHTTPメソッド（動詞）の組み合わせで、操作の意味を表現するAPI設計のスタイル。

1 なぜ生まれた？

APIの設計者ごとにURLの命名や動詞の使い方がバラバラだと、規模が大きくなるほど一貫性が失われ、他の開発者が理解しづらくなっていた。



2 それまでどうしていた？

RESTを意識しない設計では、操作ごとにURLをその都度考えて命名することになり、統一感がなくなりがちだった。

	RESTを意識しないAPI	REST
URL 設計	/getUser, /deleteUser 等動詞混じり	/users のように名詞のみ
操作の表現	URLの中に動詞を書く	GET/POST/PUT/DELETE 等HTTPメソッドで表現
一貫性	設計者ごとにルールが ブレる	リソース単位で統一し やすい

3 誰がいつ作った？

作った人	Roy Fielding
時期	2000 年
文脈	博士論文で提唱された Web 向けのアーキテクチャスタイル。

関連キーワード

比較する技術



GraphQL

必要なデータだけを1つのエンドポイントで取得できる設計。

P.64

関連する語



Endpoint

RESTのURL設計の対象になる、個々の入り口。

P.62



Status Code

REST APIのレスポンスで結果を伝える3桁の数字。

P.66

GraphQL

ひとことという

欲しい項目だけを指定して、1回のリクエストでまとめて取得できるAPIの問い合わせ言語。

1 なぜ生まれた？

画面に必要なデータを揃えるためにREST APIを複数回呼び出す実装では、リクエスト回数が増え、過不足のあるデータが返ってくることも多かった。



2 それまでどうしていた？

RESTでは、1つのエンドポイントが返すデータの形はあらかじめ固定されている。

	REST	GraphQL
取得できるデータ	エンドポイントごとに固定	欲しい項目だけ自分で指定
リクエスト回数	画面に必要な分だけ複数回	1回のリクエストでまとめて取得
過不足	使わない項目まで返ってくることもある	過不足なく必要な分だけ返る

3 誰がいつ作った？

作った人 Facebook

時期 2012年開発・2015年公開

文脈 複雑な画面に必要なデータだけ取得するために作られた。

関連キーワード

比較する技術



REST

URLと動詞で操作を表現するAPI設計スタイル。

P.63

関連する語



スキーマ (Schema)

GraphQLでもデータの形を定義するために使われる。

P.56

OpenAPI

ひとことについて

APIのエンドポイントや入出力の形を、YAMLやJSONで記述する仕様のフォーマット。

1 なぜ生まれた？

API仕様が文書化されていない現場では、フロントエンドとバックエンドの間でレスポンス形式のすり合わせを、その都度コードを読んで行う必要があった。



2 それまでどうしていた？

OpenAPIがない現場では、API仕様はドキュメントに手書きするか、コードを読んで確認するしかなかった。

	手書きの仕様書・口頭確認	OpenAPI
仕様の形	Wordやスプレッドシートに個別記述	決まったフォーマット (YAML/JSON)
実装との整合	更新を忘れて仕様と実装がズれる	ツールで検証・モックまで生成できる
共有のしやすさ	ファイルを都度共有	Swagger UI等でそのまま閲覧できる

3 OpenAPI で何ができるようになった？



API仕様を1つのファイルにまとめられる



Swagger UIなどでブラウザから確認できる



モックサーバーやクライアントコードを自動生成

関連キーワード

関連する語



API

OpenAPIはこのAPIの仕様を記述するためのフォーマット。

P.61



スキーマ (Schema)

OpenAPIの中でリクエスト・レスポンスの形を定義する仕組み。

P.56

Status Code

ひとことというと

リクエストの結果が成功か失敗か、失敗ならどんな理由かを伝える3桁の数字。

関連キーワード

関連する語



REST

Status CodeはREST APIのレスポンスで結果を伝える手段。

P.63

1 なぜ生まれた？

エラーの内容をレスポンス本文の文章だけで判断していると、成功・失敗の区別や原因の特定に毎回時間がかかっていた。



2 それまでどうしていた？

ステータスコードを見ない・意識しない実装では、エラーの原因を毎回レスポンスの中身の文章だけから推測するしかない。

	本文のメッセージだけで判断	Status Code で判断
成功 / 失敗の判定	文章を読んで解釈	200 番台か400/500 番台かで機械的に判定
原因の切り分け	都度サーバーログを確認	404=見つからない、401=未認証等型で分かる
実装のしやすさ	文字列を都度パースして分岐	ステータスコードで分岐処理を書ける

3 Status Code で何ができるようになった？



成功 / 失敗を数字だけで機械的に判定



404 や500 など原因の見当がつく



フロント側で分岐処理を書きやすい

slug

ひとことという

記事やページを識別するために使う、URL に埋め込める短い文字列（例: /posts/how-to-use-docker ）

1 なぜ生まれた？

記事やページの URL を数字の ID だけで構成すると、人間にもクローラーにも内容が伝わらず、検索結果での見え方にも不利だった。



2 それまでどうしていた？

ID だけを URL に使っていると、人間にもクローラーにもそのページの中身が伝わらない。

	ID だけの URL	slug を使った URL
見た目	/posts/1024	/posts/how-to-use-docker
人への伝わりやすさ	中身が推測できない	タイトルの内容が一目で分かる
SEO への影響	キーワードが含まれない	URL にもキーワードが含まれる

3 slug で何ができるようになった？



URL に人が読める文字列を埋め込む



Endpoint の一部として識別子に使える



SEO やシェア時の見た目が改善する

関連キーワード

関連する語



Endpoint

slug はエンドポイントの URL の一部として使われることが多い。

P.62

gRPC

ひとことという

Googleが開発した、サーバー同士の内部通信を高速かつ型安全に行うための通信の仕組み。

1 なぜ生まれた？

マイクロサービス間の通信でJSON形式のREST APIを多用すると、サービス数が増えるほど通信量・処理コストが積み重なり、パフォーマンスが課題になっていた。



2 それまでどうしていた？

これまでのHTTPリクエスト/レスポンス型の通信（Ch2で扱ったもの）は、人が読めるテキスト（JSON等）でやり取りするため分かりやすい反面、データ量や速度の面で不利になることがあった。

	HTTP + JSON	gRPC
データ形式	人が読めるテキスト（JSON）	バイナリ形式（Protocol Buffers）
速度・データ量	比較的大きく遅い	軽量で高速
向いている場面	ブラウザとのやり取り	サーバー同士の内部通信

3 gRPC で何ができるようになった？



バイナリ形式で高速に通信できる



マイクロサービス間の通信に向いている



スキーマ（型定義）から複数言語のコードを生成

関連キーワード

関連する語



REST

URLと動詞で操作を表現する、人にも読みやすいAPI設計。

P.63



スキーマ（Schema）

gRPCも通信するデータの形をスキーマとして定義する。

P.56

WebSocket

ひとことというと

一度接続を確立したら、サーバーとクライアントが双方向にいつでもデータを送り合える通信の仕組み。

1

なぜ生まれた？

チャットや通知のようにリアルタイム性が求められる機能を通常のHTTPリクエストで実現しようとする、一定間隔でAPIを呼び続ける（ポーリング）必要があり、サーバー負荷が増大していた。



2

それまでどうしていた？

通常のHTTPリクエスト/レスポンス（Ch2）は、クライアントが聞かない限りサーバーからは何も送られてこない。

	HTTPのポーリング	WebSocket
通信の方向	クライアントから聞きに行く一方通行	接続後は双方向にいつでも送れる
リアルタイム性	一定間隔でしか更新されない	変化があった瞬間に届く
向いている場面	通常のページ表示	チャット・通知・対戦ゲーム等

3

誰がいつ作った？

作った人 IETF

時期 2011年

文脈 RFC 6455 として標準化され、双方向通信の定番になった。

関連キーワード

関連する語



Pub/Sub

変化があった相手に自動で知らせる、より広い仕組み。

P.74



Webhook

こちらもイベント発生時に通知する仕組みだが、通信の形が異なる。

P.73

Webhook

ひとことという

サービス側で何かが起きたときに、登録しておいたURLへ自動でリクエストを送ってくれる仕組み。

1 なぜ生まれた？

外部サービスの状態変化（決済完了など）を検知するために定期的にAPIを呼び続けると、確認間隔ぶんタイムラグが生じ、サーバー負荷も無駄にかかっていた。



2 それまでどうしていた？

Webhookがない場合、外部サービスの状態変化を知るには、こちらから定期的にAPIを叩いて確認（ポーリング）するしかない。

	定期的なポーリング	Webhook
確認の主体	自分から定期的に聞きに行く	相手が変わった瞬間に知らせてくれる
タイムラグ	ポーリングの間隔ぶん遅れる	ほぼリアルタイムに届く
実装の準備	APIを叩くコードだけでよい	通知を受け取るURL（受け口）を用意する

3 Webhook で何ができるようになった？



外部サービスからの通知を受け取れる



ポーリングよりリアルタイムに近い



決済・デプロイ通知など幅広く使われる

関連キーワード

関連する語



Pub/Sub

Webhookと同じく通知の仕組みだが、間に仲介役を挟む点異なる。

P.74



Endpoint

Webhookの通知を受け取るための窓口として自分のAPIに用意する。

P.62

Pub/Sub

ひとことについて

発行者（Publisher）がメッセージを送り、購読者（Subscriber）が仲介役を通じて受け取る非同期の通信モデル。

1 なぜ生まれた？

1つのイベント（注文など）を複数の送信先（在庫システム・通知システムなど）に連携する際、送信元のコードに宛先を直接書き込んでいくと、宛先が増えるたびに送信元の修正が必要になっていた。



2 それまでどうしていた？

Webhookのように送信元が受信先を直接知っている構成では、通知先が増えるたびに送信元の実装を修正する必要がある。

	Webhook で直接送る	Pub/Sub
送信先の把握	送信元が受信先の URL を知っている	送信元は仲介役（トピック）に送るだけ
受信先の追加	送信元のコードを修正して追加	購読するだけで新しい受信先を追加できる
関係性	送信元と受信先が密結合	間にトピックを挟んで疎結合

3 Pub/Sub で何ができるようになった？



送信元と受信先を直接つながずに済む



受信先を後から自由に追加できる



複数のサービスへの同時通知に向いている

関連キーワード

関連する語



Webhook

こちらは送信元が受信先を直接知っている、1対1に近い通知方式。

P.73



WebSocket

こちらは接続を保ったまま双方向にやり取りする通信方式。

P.72

Database

ひとことというと

データを一箇所にまとめて、複数人で安全に読み書きでき、検索しやすく保存する仕組み。

1 なぜ生まれた？

顧客データやアプリの状態をファイルで直接管理すると、複数人が同時に編集した際にデータが上書きされて消失する危険があった。



2 それまでどうしていた？

ファイルに直接データを書き込んで、そのまま保存・共有していた。

	ファイル保存	データベース
同時アクセス	上書きされて壊れやすい	複数人が同時に安全に読み書きできる
検索	全部読んで目で探す	条件を指定して一瞬で検索できる
整合性	壊れても気づきにくい	ルールで壊れたデータを弾ける

3 Database で何ができるようになった？



大量データも高速に検索できる



複数人で同時に読み書きできる



ルールでデータの整合性を守れる

関連キーワード

データベースの種類



RDB

表同士を関係で結びつけて管理する方式。

P.82



NoSQL

表の形にこだわらない柔軟なデータベース。

P.84

構成する要素



Table

データを行と列で整理する表そのもの。

P.85

ひとことについて

行と列を持つ「表（テーブル）」同士を関連付けてデータを管理する方式。

1 なぜ生まれた？

1つの巨大な表にあらゆる情報を詰め込んで管理すると、同じ値を繰り返し入力することになり、更新漏れやデータの不整合が起きやすかった。



2 それまでどうしていた？

1つの巨大な表に、あらゆる情報を詰め込んで管理していた。

	1つの表	RDB
データの重複	同じ情報を何度も書く	表を分けて重複を減らせる
更新	複数箇所を直す必要あり	1箇所直せば済む
整合性	矛盾したデータが混在する	関係のルールで矛盾を防げる

3 RDB で何ができるようになった？



表を分けて重複を減らせる



関係のルールでデータを守れる



複雑な業務データも整理できる

関連キーワード

実装ソフトと対になる考え方



RDBMS

RDBの考え方を実装したソフトウェア。

P.83



NoSQL

表の形にこだわらない別のアプローチ。

P.84

関連する要素



Table

行と列でデータを整理する表。

P.85



Foreign Key

表同士の関係を表すキー。

P.88

RDBMS

ひとことというと

RDB（関係データベース）の考え方を実装し、実際にデータを保存・検索・操作できるようにするソフトウェア。

1 なぜ生まれた？

RDBという設計上の考え方だけでは、実際にデータを保存・検索・更新する具体的な処理を実行できなかった。



2 それまでどうしていた？

RDBの仕組みを、必要になるたびに自分でゼロから実装しようとしていた。

	自前で実装	RDBMS
信頼性	自分でバグと戦うことになる	実績あるソフトウェアが保証してくれる
機能	検索やトランザクションを自作	SQLや排他制御が最初から使える
選択肢	ゼロから設計するしかない	MySQLやPostgreSQL等から選べる

3 RDBMS で何ができるようになった？



RDBの仕組みを
すぐ使い始められ
る



トランザクション
や制約を任せられ
る



クラウドサービス
としても使える

関連キーワード

関連する考え方



RDB

表同士を関係で結びつけるデータの考え方。

P.82

操作する言語



SQL

RDBMSに指示を出すための共通言語。

P.90

NoSQL

ひとことについて

テーブルの形に縛られず、柔軟な形でデータを保存できるデータベースの総称。

1 なぜ生まれた？

サービスの急成長で、ユーザーごとに項目が異なるようなデータをRDBの固定された表構造に収めようとする、と、カラムが増え続けて設計が破綻しかなかった。



2 それまでどうしていた？

RDB（表構造）だけを使って、あらゆる種類のデータを管理しようとしていた。

	RDBのみ	NoSQL
データの形	事前に列を固定する必要がある	形が違うデータも自由に保存できる
スケール	1台のサーバーで頑張りがち	複数台に分散しやすい
向いている場面	整合性重視の業務データ	ログ・キャッシュ等柔軟なデータ

3 NoSQL で何ができるようになった？



形の違うデータも柔軟に保存できる



大量データを分散して扱える



読み書きを高速化しやすい

関連キーワード

対になる考え方



RDB

表同士を関係で結びつけるデータの考え方。

P.82



RDBMS

RDBを実装したソフトウェア。

P.83

代表的な製品



Redis

高速なキー・バリュ型データベース。



MongoDB

柔軟なドキュメント型データベース。

Table

ひとことという

データを行（レコード）と列（カラム）に整理して格納する、データベースの基本単位。

1 なぜ生まれた？

データを決まった形式のないテキストや配列にそのまま書き込むと、後から検索や更新を正確に行うことができなかった。



2 それまでどうしていた？

テキストや配列に、決まった形もなくデータを書き並べていた。

	形が不定	Table
構造	毎回バラバラな形式になる	列でルールが決まっている
検索	どこに何があるか探す	列名で欲しいデータを取り出せる
追加	書式がブレやすい	同じ形式のレコードとして追加できる

3 Table で何ができるようになった？



同じ形式でデータを並べられる



列名で欲しいデータをすぐ取り出せる



型やルールでデータの形を保てる

関連キーワード

構成する要素



Record

テーブルの1行分のデータ。

P.86



Primary Key

1件を一意に特定する値。

P.87



Foreign Key

他の表とつながりを持たせるキー。

P.88

Record

ひとことについて

テーブルの中の1行分、つまり「1件分」としてまとめたデータ。

1 なぜ生まれた？

データを項目ごとにバラバラに扱っていると、どこからどこまでが「1件分」なのかが曖昧になり、一部の項目だけを更新して不整合を起こしやすかった。



2 それまでどうしていた？

データの単位を意識せず、項目ごとにバラバラに扱っていた。

	項目バラバラ	Record
単位	項目だけを見て操作する	1件分をまとめて扱う
追加・削除	一部の項目だけ更新しがち	1行まるごと追加・削除できる
識別	「誰のデータか」が曖昧になる	主キーで1件を確実に特定できる

3 Record で何ができるようになった？



1件分のデータをまとめて扱う



行単位で追加・更新・削除できる



主キーで1件を確実に特定できる

関連キーワード

関連する要素



Table

レコードが積み重ねられる表そのもの。

P.85



Primary Key

レコードを一意的に特定する値。

P.87

Primary Key

ひとことという

テーブルの中で1件のレコードを一意に特定するための、重複しない値。

関連キーワード

関連する要素



Record

主キーで特定される1件分のデータ。

P.86



Foreign Key

主キーを他の表から参照するキー。

P.88

1 なぜ生まれた？

名前やメールアドレスなど意味を持つ項目でレコードを特定すると、同じ値を持つ別のレコードまで誤って操作してしまう恐れがあった。



2 それまでどうしていた？

名前やメールアドレスなど、意味のある項目を頼りにレコードを特定していた。

	名前で特定	Primary Key
一意性	同じ値の人が現れうる	絶対に重複しない値を保証する
変更	名前変更で特定方法が崩れる	値が変わらないIDで安定して参照
事故	別人のデータを誤更新しうる	常に狙った1件だけを操作できる

3 Primary Key で何ができるようになった？



常に1件だけを正確に特定できる



重複や誤更新を防げる



他の表からも安全に参照できる

Foreign Key

ひとことという

別のテーブルの主キーを参照して、テーブル同士の関係を表す値。

1 なぜ生まれた？

テーブル同士のつながりをアプリ側のコードだけで管理すると、親データを削除した際に、存在しない親を指す壊れた子データが残ってしまう恐れがあった。



2 それまでどうしていた？

テーブル同士のつながりを、アプリ側のコードだけで気をつけて管理していた。

	アプリ側で担保	Foreign Key
親の削除	子データが孤児として残る	DB が削除を止める / 連動して消せる
不正なデータ	存在しない親を指すデータが作れる	存在する親しか指せない
依存関係	コードを読まないと分からない	テーブル定義を見れば分かる

3 Foreign Key で何ができるようになった？



表同士のつながりをDB が保証する



親を消すときの挙動を制御できる



存在しないデータへの参照を防げる

関連キーワード

関連する要素



Table

キーで関係づけられる表同士。

P.85



Primary Key

外部キーが参照する側のキー。

P.87

Index

ひとことという

特定の列の検索を高速化するために作る、データの「索引」。

1 なぜ生まれた？

索引がない状態でテーブルを検索すると、データ件数が増えるほど全件を読み比べる処理コストが増大し、検索が遅くなっていった。



2 それまでどうしていた？

索引がない状態で、テーブルを頭から全部読んで探していた。

	インデックスなし	Index
検索方法	全件を1行ずつ確認する	索引から一気に絞り込む
速度	データが増えるほど遅くなる	大量データでも高速なまま
コスト	検索は速いが書き込みは軽い	書き込みや保存容量は少し増える

3 Index で何ができるようになった？



大量データでも検索が高速なまま



待たされるページが減る



速さと更新コストのバランスを取れる

関連キーワード

関連する要素



Table

インデックスが張られる表。

P.85



Primary Key

通常は自動でインデックスが張られる。

P.87

SQL

ひとことについて

データベースに対して、検索・追加・更新・削除を指示するための言語。

1 なぜ生まれた？

データベース製品ごとに操作方法が異なると、製品を変更するたびに操作方法を一から覚え直す必要があった。



2 それまでどうしていた？

各データベース製品が、それぞれ独自の操作方法を持っていた。

	製品ごとに独自	SQL
学習コスト	製品ごとに覚え直す	一度覚えれば多くの製品で通用
書き方	製品依存でバラバラになる	SELECT / INSERT など共通構文
移行	製品を変えたと書き直しになる	基本構文はそのまま使い回せる

3 SQL で何ができるようになった？



共通の書き方でデータを操作できる



条件を指定して正確に絞り込める



多くのDB製品で同じ知識が使える

関連キーワード

関連する要素



RDBMS

SQLで操作する対象のソフトウェア。

P.83



ORM

SQLを直接書かずに操作する仕組み。

P.91

ORM

ひとことという

プログラムのオブジェクトとテーブルを対応させ、SQLを直接書かずに操作できるようにする仕組み。

1 なぜ生まれた？

SQL文を文字列として自分で組み立てると、値の埋め込み方を誤った際にSQLインジェクションの脆弱性を作り込んでしまう恐れがあった。



2 それまでどうしていた？

SQL文を自分で組み立てて、文字列としてそのまま実行していた。

	SQL 直書き	ORM
書き方	SQL文を文字列で組み立てる	コードのオブジェクトとして操作
安全性	書き方次第で脆弱性を作りうる	安全な形に自動で変換してくれる
可搬性	DB製品ごとにSQLを書き分ける	同じコードで複数のDBに対応しやすい

3 ORM で何ができるようになった？



SQLを意識せずデータ操作できる



安全な形にSQLを組み立ててくれる



コードの記述量を減らせる

関連キーワード

関連する要素



SQL

ORMが裏側で組み立てている言語。

P.90



RDBMS

ORMが最終的に操作するソフトウェア。

P.83

Migration

ひとことというと

テーブルの追加・変更などのDB設計変更を、コードの形で記録・管理する仕組み。

1 なぜ生まれた？

本番環境に手作業でALTER TABLEなどのSQLを実行すると、環境ごとにカラムの型や状態がずれ、リリース後にエラーが発生しやすかった。



2 それまでどうしていた？

テーブル変更のSQLを、環境ごとに手作業で実行していた。

	手作業でSQL	Migration
再現性	環境ごとに結果がズレうる	同じ変更を全環境に確実に反映
履歴	いつ何を変えたか分からない	変更履歴がコードとして残る
共有	口頭やメモで伝達しがち	コードをチームで共有・レビューできる

3 Migration で何ができるようになった？



変更履歴を時系列で管理できる



全環境に同じ変更を確実に反映できる



変更をコードとしてレビューできる

関連キーワード

一緒に使う仕組み



Seeder

確認用データを自動投入する仕組み。

P.93

関連する要素



RDBMS

マイグレーションで変更する対象。

P.83

Seeder

ひとことについて

開発・テスト用のダミーデータを、コードで自動的に投入する仕組み。

1 なぜ生まれた？

動作確認用のデータを管理画面やSQLで毎回手動で登録すると、環境を作り直すたびに同じ作業を繰り返す必要があった。



2 それまでどうしていた？

確認用データを、管理画面やSQLで毎回手動で入力していた。

	手動入力	Seeder
再現性	毎回微妙に違うデータになる	同じデータをいつでも再現できる
手間	環境ごとに手で入力する	コマンド1つで一括投入できる
共有	テストデータが人によって違う	チームで同じテストデータを使える

3 Seeder で何ができるようになった？



確認用データを一括で投入できる



環境構築の時間を短縮できる



チームで同じデータを共有できる

関連キーワード

一緒に使う仕組み



Migration

テーブルの構造そのものを変更する仕組み。

P.92

Transaction

ひとことという

複数の処理を「全部成功」か「全部やり直し」のどちらかにまとめる仕組み。

1 なぜ生まれた？

複数の処理を1つずつ順番に実行すると、途中でサーバーが落ちた際に、一部の処理だけが反映された中途半端な状態が残ってしまう恐れがあった。



2 それまでどうしていた？

複数の処理を、1つずつ順番に実行して結果を信じていた。

	個別に実行	Transaction
途中失敗	中途半端な状態が残る	全部なかったことにできる
一貫性	一部だけ成功した状態になりうる	全部成功するまでは反映されない
対象	1件ずつ処理する	複数の処理をひとまとまりで扱う

3 Transaction で何ができるようになった？



全部成功か全部失敗かにできる



処理の途中で落ちても安全



複数処理を1つの単位で扱える

関連キーワード

関連する仕組み



Lock

同時アクセスの衝突を防ぐ仕組み。

P.95

関連する要素



RDBMS

トランザクションを保証するソフトウェア。

P.83

Lock

ひとことという

複数の処理が同じデータに同時にアクセスしたときの衝突を防ぐ仕組み。

1 なぜ生まれた？

同時アクセスを考慮せずに処理すると、複数の処理が同じデータへほぼ同時にアクセスした際、在庫チェックのようなチェックをすり抜けて不整合が生じる恐れがあった。



2 それまでどうしていた？

同時アクセスを考えず、処理は順番に実行されるものだと思い込んでいた。

	ロックなし	Lock
同時アクセス	同じデータを同時に書き換えられる	一方が終わるまで待たせられる
整合性	在庫や残高の数が合わない	1件ずつ確実に処理される
代償	事故が起きるまで気づきにくい	待ち時間や順番待ちが発生する

3 Lock で何ができるようになった？



同時に書き換えられるのを防げる



在庫や残高の整合性を守れる



処理の順番を保証できる

関連キーワード

関連する仕組み



Transaction

複数の処理を1つの単位にまとめる仕組み。

P.94

ロックの種類



行ロック

更新中の1行だけをロックする方式。

P.96



テーブルロック

テーブル全体をロックする方式。

P.97



デッドロック

ロック同士が待ち合って止まる状態。

P.98

行ロック

ひとことという

更新中の1行(レコード)だけをロックし、他の行はそのまま操作できるようにする仕組み。

1 なぜ生まれた？

1件の更新のためにテーブル全体をロックすると、無関係な行への処理まで待たされ、システム全体の応答が遅くなっていった。



2 それまでどうしていた？

テーブル全体をロックして、1件の更新のために全部を止めていた。

	テーブルロック	行ロック
ロック範囲	テーブル全体	更新中の行だけ
他の処理	無関係の行も待たされる	別の行は同時に処理できる
向いている場面	一括で大量更新するとき	個別の注文・更新処理

3 行ロックで何ができるようになった？



該当の1行だけを貸し切れる



他の行の処理は止めずに進められる



必要最小限の範囲だけ制御できる

関連キーワード

関連する仕組み



Lock

同時アクセスの衝突を防ぐ仕組み全般。

P.95



テーブルロック

表全体を対象にする粗いロック。

P.97

テーブルロック

ひとことというと

テーブル全体をロックし、他の処理からの読み書きを丸ごと止める仕組み。

1 なぜ生まれた？

大量データを一括更新する際に行ロックだけで対応すると、更新対象の行が多いほどロックの管理コスト自体が処理を遅くしてしまっていた。



2 それまでどうしていた？

行ロックだけで、大量データの一括更新も乗り切ろうとしていた。

	行ロックのみ	テーブルロック
対象	更新する行の数だけロック管理	テーブル単位でまとめて制御
向いている場面	個別の更新処理	一括更新やメンテナンス作業
他の処理への影響	関係ない行は動ける	テーブル全体が使えなくなる

3 テーブルロックで何ができるようになった？



テーブル全体をまとめて制御できる



大量データの一括更新に強い



用途に応じてロックの粒度を選べる

関連キーワード

関連する仕組み



行ロック

該当する行だけを対象にする細かいロック。

P.96



Lock

同時アクセスの衝突を防ぐ仕組み全般。

P.95

デッドロック

ひとことというと

複数の処理が互いに相手のロック解放を待ち合って、どちらも先に進めなくなる状態。

1

なぜ生まれた？

複数の処理がそれぞれ異なる順序でロックを取得すると、互いに相手のロック解放を待ち合い、両方の処理が先に進めなくなる恐れがあった。



2

それまでどうしていた？

ロックの取得順序を意識せず、処理ごとに好きな順番でロックを取っていた。

	順序バラバラ	デッドロック対策
ロック順序	処理ごとにバラバラ	常に同じ順番で取得する
発生時	お互い待ち続けてフリーズ	DBが検知して片方を強制終了
対策	気づかず放置しがち	順序統一やタイムアウトで予防

3

デッドロックにどう対応できる？



循環待ちの状態だと理解できる



タイムアウトで強制的に解消できる



ロック順序を揃えて予防できる

関連キーワード

関連する仕組み



Lock

同時アクセスの衝突を防ぐ仕組み全般。

P.95



行ロック

デッドロックが起きうる細かいロック単位。

P.96



テーブルロック

デッドロックが起きうる粗いロック単位。

P.97

ひとことというと

一覧表示のループの中で関連データを都度取得してしまい、件数分だけSQLが増えてしまう問題。

1 なぜ生まれた？

一覧画面でループの中から関連データを都度呼び出す実装では、表示件数と同じ回数だけSQLが発行され、件数が増えるほど処理が重くなっていた。



2 それまでどうしていた？

気づかないうちは、ループの中で関連データを呼び出すコードを普通に書いていた。

	気づかず書いたコード	eager loading で対策
発行されるクエリ数	1 + N 回	1〜2 回
表示速度	件数に比例して遅くなる	件数が増えてもほぼ一定
気づき方	本番で遅くなって発覚しがち	gem やログで事前に検出できる

3 N+1 で何ができるようになった？



一覧の表示が速くなる



件数が増えても劣化しにくい



クエリ数を意識する癖がつく

関連キーワード

関連するキーワード



Cache

同じ問い合わせを何度もしないための一時保管。

P.100



コネクションプール

クエリが増えると接続数の上限にも響いてくる。

P.115

Cache

ひとことについて

一度取得・計算した結果を保存しておき、次回以降はそれを使い回して高速化する仕組み。

1 なぜ生まれた？

同じ集計や問い合わせをアクセスのたびにDBへ直接投げ直すと、内容が変わっていなくても毎回同じ処理コストがかかっていた。



2 それまでどうしていた？

キャッシュを知る前は、表示のたびに毎回DBへ直接問い合わせていた。

	毎回DBに問い合わせ	キャッシュを使う
応答時間	毎回の処理時間がかかる	2回目以降はほぼ即返る
DBへの負荷	常にフルで問い合わせる	負荷を大きく減らせる
データの鮮度	常に最新	古い値を返す期間が発生しうる

3 Cache で何ができるようになった？



応答が速くなる



DBへの負荷を減らせる



速さと鮮度のトレードオフを学べる

関連キーワード

関連するキーワード



Redis

メモリ上で動く、キャッシュの置き場所として定番。

P.108



N+1

ループ内クエリが多発も、キャッシュで緩和できることがある。

P.99

正規化

ひとことというと

データの重複を減らすために、テーブルを目的ごとに分割して整理する設計手法。

1 なぜ生まれた？

1つのテーブルに同じ情報を繰り返し書き込むと、一部だけ更新を反映し忘れて、同じ対象なのにデータが食い違ってしまふ恐れがあった。



2 それまでどうしていた？

正規化を知らないうちは、1つの大きなテーブルに情報を全部詰め込んでいた。

	正規化前(1テーブルに詰め込み)	正規化後(テーブルを分割)
データの重複	同じ情報を何度も書く	重複を減らせる
更新のしやすさ	直し漏れが起きやすい	1箇所直せば反映される
JOINの発生	ほぼ不要	テーブルをまたぐ分だけ増える

3 正規化 で何ができるようになった？



更新漏れを防げる



データの整合性を保ちやすい



無駄な容量を減らせる

関連キーワード

差分ペア



非正規化

あえて重複を持たせて読み取りを速くする、正規化の逆張り。

P.102

関連するキーワード



ER図

正規化した結果のテーブル関係を絵にしたもの。

P.103

非正規化

ひとことというと

正規化で分けたデータをあえて重複させて持たせ、JOINを減らして読み取りを速くする設計。

1 なぜ生まれた？

正規化を徹底したテーブル構成では、画面表示のたびに多数のテーブルをJOINする必要があり、表示速度が低下しやすかった。



2 それまでどうしていた？

正規化だけで設計していた頃は、JOINの数が増やすたびにどんどん増えていった。

	正規化のみ	非正規化を併用
読み取り速度	JOINが多く遅くなりがち	JOINを減らして速くできる
JOINの数	関連の数だけ増える	重複を持たせた分だけ減る
更新の手間	1箇所直せばよい	重複箇所すべてを同期する必要がある

3 非正規化 で何ができるようになった？



読み取りが速くなる



JOINを減らせる



集計値をキャッシュ的に持てる

関連キーワード

差分ペア



正規化

重複を減らし、更新漏れを防ぐための基本の設計手法。

P.101

関連するキーワード



Cache

非正規化と同じく、読み取り速度を優先する手段のひとつ。

P.100

ER図

ひとことという

テーブルの構造と、テーブル同士のつながり（1対多など）を図で表したもの。

1 なぜ生まれた？

テーブル数が多いプロジェクトでは、コードやSQLを読むだけではテーブル同士の関係性を把握しづらかった。



2 それまでどうしていた？

ER図がない、もしくは古いままの現場では、ドキュメントを読んでも関係が分からなかった。

	ER図なし・古いまま	ER図を整備・更新
新規参入者の理解速度	コードを読み解く必要がある	図を見れば全体像がつかめる
設計変更時の影響範囲	見落としやすい	関係を辿って確認しやすい
共有のしやすさ	口頭・文章で説明が必要	図1枚で共有できる

3 ER図で何ができるようになった？



全体構造が一目でわかる



設計レビューがしやすい



変更の影響範囲を追やすい

関連キーワード

関連するキーワード



正規化

ER図に描かれるテーブル分割の考え方そのもの。

P.101



非正規化

正規化した図をあえて崩す判断も、ER図の上で検討する。

P.102

UUID

ひとことというと

ランダム性の高い長い文字列でIDを作り、複数の場所で同時に発行してもぶつからないようにする仕組み。

1 なぜ生まれた？

auto incrementの連番IDをそのままURLなどで公開すると、IDから件数や作成順序といった情報が推測されてしまう恐れがあった。



2 それまでどうしていた？

それまではauto increment（連番）のPrimary Keyを、そのまま外部にも見せていた。

	auto increment（連番）	UUID
採番のタイミング	DBへの登録時に決まる	アプリ側で事前に決められる
複数サーバーでの衝突	採番がぶつかる可能性がある	ほぼ衝突しない
外部公開時の推測しやすさ	連番から件数が推測できる	推測されにくい

3 UUIDで何ができるようになった？



分散環境でも安全に採番できる



連番から件数を推測されにくい



登録前にIDを決めておける

関連キーワード

関連するキーワード



ER図

Primary KeyをUUIDにするかは設計段階で決めておきたい。

P.103

MySQL

ひとことという

世界中の現場で長く使われてきた、実績重視のオープンソースリレーショナルデータベース。

1 なぜ生まれた？

新規プロジェクトでDBを選定する際、社内外に知見や情報が豊富で、トラブル時にも解決策を見つけやすい製品が求められた。



2 それまでどうしていた？

DB選定を経験する前は、DBの違いを意識せず「なんとなく有名だから」で決めていた。

	なんとなく選ぶ	理由を言語化して選ぶ
情報量・実績	意識していなかった	困ったときの情報が見つかりやすい
ホスティングの選択肢	意識していなかった	クラウド・レンタルサーバーで広くサポート
機能の豊富さ	意識していなかった	PostgreSQLと都度比較する視点を持てる

3 MySQL で何ができるようになった？



困ったときの情報が見つかりやすい



幅広い環境でサポートされている



中〜大規模まで実績で安心できる

関連キーワード

比較する技術



PostgreSQL

機能の豊富さで選ばれることが多いRDB。

P.106



SQLite

サーバー不要で、小規模用途に向くRDB。

P.107

PostgreSQL

ひとことについて

JSON型や高度な検索機能など、機能の豊富さに定評があるオープンソースリレーショナルデータベース。

1 なぜ生まれた？

複雑な検索条件や位置情報検索、柔軟なデータ型など、標準的なRDBの機能だけでは対応しづらい要件が出てきた。



2 それまでどうしていた？

MySQL一択だと思っていたが、要件によっては機能面で足りないことに気づいた。

	MySQL	PostgreSQL
型の豊富さ	基本的な型が中心	JSON型・配列型等も扱える
拡張機能	限定的	全文検索・位置情報検索等が充実
標準SQLへの準拠度	独自拡張が多い	標準SQLに忠実で挙動が読みやすい

3 PostgreSQL で何ができるようになった？



複雑なデータ構造も保存できる



拡張機能で高度な検索ができる



SQL標準準拠で挙動を追いやすい

関連キーワード

比較する技術



MySQL

情報量と実績の多さで選ばれることが多いRDB。

P.105

SQLite

ひとことというと

サーバープロセスを立てず、1つのファイルとしてデータベースを扱える軽量なリレーショナルデータベース。

1 なぜ生まれた？

個人開発や簡単な検証のたびに本格的なDBサーバーを構築するのは、規模に見合わないコストがかかっていた。



2 それまでどうしていた？

SQLiteを知る前は、小さいアプリでも毎回サーバー型のRDBを立てて使っていた。

	サーバー型RDB	SQLite
セットアップの手軽さ	サーバーの起動・設定が必要	ファイル1つで即使える
同時書き込みへの強さ	複数接続の同時書き込みに強い	同時書き込みは苦手
向いている用途	本番の複数人利用	試作・テスト・小規模用途

3 SQLite で何ができるようになった？



インストール不要
ですぐ試せる



テスト・開発環境
で高速に動く



ファイル1つで持
ち運べる

関連キーワード

比較する技術



MySQL

本番の複数人利用
に向く定番RDB。

P.105



PostgreSQL

機能重視ならこちら
のRDB。

P.106

Redis

ひとことについて

データをメモリ上に持つことで、キャッシュやセッション管理などを高速に処理できるインメモリ型データストア。

1

なぜ生まれた？

セッション情報やランキングのような一時的で頻繁にアクセスされるデータを毎回RDBに問い合わせると、応答速度がボトルネックになっていた。



2

それまでどうしていた？

Redis導入前は、セッションやランキングのような一時的なデータもRDBに保存していた。

	RDBに保存	Redisに保存
読み書きの速度	ディスクI/Oが発生する	メモリ上なので非常に速い
データの永続性	基本的に永続化されている	設定しないと再起動で消える
向いている用途	正確さが重要なデータ	セッション・ランキング・キャッシュ

3

Redis で何ができるようになった？



読み書きが非常に速い



キャッシュ・セッション管理に最適



ランキングなど独自の使い方ができる

関連キーワード

関連するキーワード



Cache

Redisはキャッシュの保存先として使われることが多い。

P.100



MongoDB

同じNoSQLでも、こちらはドキュメント指向データベース。

P.109

MongoDB

ひとことという

スキーマ（テーブル構造）を事前に固定せず、JSONに近い形式でデータを柔軟に保存できる NoSQL データベース。

1 なぜ生まれた？

仕様が固まっていない段階で頻繁にカラムを変更すると、そのたびにマイグレーションを書く必要があり、開発の負担になっていた。



2 それまでどうしていた？

RDB 一本で作っていた頃は、スキーマ変更のたびにマイグレーションと向き合っていた。

	RDB（スキーマ固定）	MongoDB（スキーマレス）
データ構造の自由度	カラム追加にマイグレーションが必要	構造を決めずに保存できる
JOIN（結合）のしやすさ	テーブルをまたいで結合しやすい	結合は基本的に苦手
データの整合性	DB 側である程度保証される	アプリ側で保証する必要がある

3 MongoDB で何ができるようになった？



構造が変わりやすいデータを扱いやすい



JSON形式のデータをそのまま保存できる



水平方向にスケールさせやすい

関連キーワード

関連するキーワード



正規化

RDBの正規化とは対照的な、構造を固定しない考え方。

P.101



Redis

同じNoSQLでも、こちらはインメモリのキー・バリュ型。

P.108

View（ビュー）

ひとことというと

複雑な SELECT 文をあらかじめ定義しておき、あたかも 1 つのテーブルであるかのように参照できるようにする仕組み。

1

なぜ生まれた？

複数テーブルを JOIN する同じ集計ロジックの SQL をあちこちのコードに書くと、修正漏れによって画面ごとに結果が食い違う不整合が起きやすかった。



2

それまでどうしていた？

ビューを知らない頃は、同じ JOIN・集計ロジックの SQL をあちこちに書いていた。

	都度 SQL を書く	ビューを使う
SQL の重複	あちこちに同じ SQL が散らばる	ビューとして 1 箇所にまとまる
修正時に直す箇所	書いた分だけ全部直す必要がある	ビューの定義を直せばよい
実データの保存	該当しない	ビュー自体は実体を持たない

3

View（ビュー）で何ができるようになった？



複雑な SQL を使い回せる



ロジックを 1 箇所にまとめられる



テーブルのように扱える

関連キーワード

関連するキーワード



Stored Procedure

SELECT 文だけでなく、処理そのものを DB 側に置く仕組み。

P.111



Cache

重い集計を都度計算させたくない点はビューと共通。

P.100

111

Stored Procedure

ひとことという

一連のSQL処理をDBサーバー側に手続きとして保存しておき、呼び出すだけで実行できるようにする仕組み。

1 なぜ生まれた？

大量データの集計処理をアプリ側のコードで行うと、DBとアプリの間で何度もデータをやり取りする分だけ時間がかかっていた。



2 それまでどうしていた？

ストアドプロシージャを使う前は、アプリ側のコードで処理を全部書いていた。

	アプリ側で処理	ストアドプロシージャ
処理速度（通信回数）	DBとのやり取りが多く発生する	DB内で完結し通信を減らせる
バージョン管理のしやすさ	Gitで差分・レビューがしやすい	DB側にあり管理が難しくなりがち
複数アプリからの利用	アプリごとに実装しがち	同じ処理を共通で呼び出せる

3 Stored Procedure で何ができるようになった？



DBとアプリ間の通信を減らせる



複雑な処理をDB内で完結できる



複数アプリから同じ処理を呼び出せる

関連キーワード

関連するキーワード



View（ビュー）

処理ではなく、SELECT文をまとめて名前をつける仕組み。

P.110

ACID

ひとことというと

原子性・一貫性・独立性・永続性という、トランザクションが満たすべき4つの性質の頭文字。

1 なぜ生まれた？

複数のSQLを個別に実行すると、一部だけ成功して失敗した際に、データが中途半端な状態のまま残ってしまう恐れがあった。



2 それまでどうしていた？

トランザクションを意識する前は、1つ1つのSQLをただ個別に実行していた。

	個別にSQL実行	トランザクションでまとめる
途中で失敗したときの挙動	中途半端な状態が残る	全部なかったことできる
同時アクセス時の安全性	データが壊れる可能性がある	独立性が保たれる
障害後のデータ	消える可能性がある	永続性が保証される

3 ACID で何ができるようになった？



途中失敗しても中途半端にならない



同時アクセスでもデータが壊れない



障害後もデータが消えない

関連キーワード

関連するキーワード



MongoDB

ACIDの厳格さより、柔軟さ・速度を優先することが多い。

P.109



Sharding

DBを分割すると、ACIDを保つ難易度も上がる。

P.113

Sharding

ひとことというと

1つの大きなテーブル・DBを複数のサーバーに分割して、それぞれに一部のデータだけを持たせる仕組み。

1 なぜ生まれた？

サービスの成長でデータ量・アクセス数が増え
ると、1台のDBサーバーだけでは処理を捌き
きれなくなっていった。



2 それまでどうしていた？

シャーディング前は、1台のDBサーバーにスペックを盛って（スケールア
ップして）対応していた。

	スケールアップ（1台 を強化）	シャーディング（複数 台に分割）
対応できるデータ量	1台のスペックが上限 になる	台数を増やして拡張で きる
JOIN・トランザクシ ョン	同じDB内なら容易	シャードをまたぐと難 しくなる
構成の複雑さ	シンプル	分割ルール的设计・運 用が複雑になる

3 Sharding で何ができるようになった？



1台の限界を超え
て扱える



台数を増やして負
荷分散できる



成長に合わせて拡
張できる

関連キーワード

関連するキーワード



レプリケーシ ョン

分割ではなく複製
で負荷を分散す
る、もう1つの手
段。

P.114



ACID

DBを分割するほ
ど、この4性質を
保つのが難しくな
る。

P.112

レプリケーション

ひとことというと

1つのDBのデータを別のDBサーバーに複製し続け、読み取りアクセスなどを分散させる仕組み。

1

なぜ生まれた？

アクセスが増えるとDBへの読み取り（SELECT）が集中し、1台のサーバーだけでは負荷が偏ってCPU使用率が張り付いてしまっていた。



2

それまでどうしていた？

レプリケーション導入前は、1台のDBに全ての読み書きが集中していた。

	DB1台に集中	レプリケーション（複製して分散）
読み取りの負荷分散	1台に集中する	複数台に分散できる
障害時の切り替え	1台が落ちると止まる	他のレプリカで継続できる
データ反映のタイムラグ	発生しない	わずかな遅延が発生する

3

レプリケーション で何ができるようになった？



読み取りアクセスを分散できる



1台が落ちてても他で継続できる



バックアップとしても使える

関連キーワード

関連するキーワード



Sharding

複製ではなく分割で負荷を分散する、もう1つの手段。

P.113

コネクションプール

ひとことでいうと

DBへの接続をあらかじめ複数用意してプールしておき、リクエストごとに使い回すことで接続のオーバーヘッドを減らす仕組み。

1 なぜ生まれた？

リクエストのたびにDBへの接続を新規に張ると、接続処理自体のコストが積み重なり、同時接続数の上限に達しやすかった。



2 それまでどうしていた？

コネクションプールを意識する前は、リクエストのたびにDBへの接続を新しく張っていた。

	都度接続	コネクションプールで 使い回し
接続確立のコスト	毎回発生する	あらかじめ用意した接続を使い回す
同時接続数の管理	意識しづらい	上限を決めて管理できる
接続不足時の挙動	エラーが発生しやすい	待機させて捌くこともできる

3 コネクションプールで何ができるようになった？



接続確立の時間を
節約できる



同時接続数を制御
しやすい



高負荷時も安定して
捌ける

関連キーワード

関連するキーワード



N+1

クエリ数が増えるほど、接続の使い回し方も重要になる。

P.99



Cache

接続や問い合わせそのものを減らす、もう1つの手段。

P.100

ダンプ (Dump)

ひとことというと

テーブルの定義やデータの中身を、ファイルとして丸ごと書き出すこと（またはそのファイル）

1 なぜ生まれた？

本番のデータを別環境に移したり、障害時に戻したりするには、DBの中身をファイルとして持ち出せる必要があった。



2 それまでどうしていた？

ダンプという発想の前は、必要な行を手でSELECTしてCSVにしたり、サーバごとコピーしたりしていた。

	手作業での持ち出し	ダンプで一括書き出し
取得単位	必要な表を個別に抽出	スキーマごと・DBごと書き出せる
再現性	手順が人依存になりやすい	同じコマンドで何度でも取れる
復元	INSERTを手で組み立てる	ダンプファイルを流し込めば復元
用途	ちょっとした確認向き	バックアップ・移行・検証データに

3 ダンプ (Dump) で何ができるようになった？



DBのスナップショットをファイルにできる



障害時にバックアップから戻せる



本番相当のデータを開発環境に持ち込める

関連キーワード

関連キーワード



Migration

スキーマ変更をコードとして管理する仕組み。

P.92



Seeder

初期データやテスト用データを流し込む仕組み。

P.93

ひとことという

見出しや段落、画像などの「要素」を並べて、Webページの構造を作るための言語。

1 なぜ生まれた？

研究者同士で文書を共有するために、装飾がなくても構造がわかる書き方が必要だった。



2 それまでどうしていた？

それまでは、Word文書のような「見たまま」を編集するファイルでやり取りしていた。

	普通の文書ファイル	HTML
見た目の編集	見たまま(WYSIWYG)で編集	タグで構造だけを指定
共有方法	ファイルをやり取り	URLで誰でも閲覧できる
機械的な読み取り	レイアウト情報が中心	見出し・段落等意味が明確

3 HTML で何ができるようになった？



見出しや段落を意味づけできる



リンクで文書同士をつなげる



ブラウザさえあれば誰でも見られる

関連キーワード

セットで使う技術



CSS

HTMLの見た目を整える言語。

P.122



JavaScript

HTMLに動きを与える言語。

P.123



DOM

HTMLをJSから操作できる形にしたもの。

P.127

ひとことというと

色・余白・レイアウトなど「見た目」だけをHTMLから分離して指定する言語。

1 なぜ生まれた？

昔はHTMLタグそのものに色やフォントを直接埋め込んでいたため、デザインを変えるたびに全ページのタグを書き換える必要があった。



2 それまでどうしていた？

それまではHTMLタグに直接、見た目の指定を書き込んでいた。

	HTMLに直接指定	CSS
見た目の変更	全ページのタグを書き換え	1つのファイルを直せば全体に反映
保守性	HTMLとデザインが密結合	構造(HTML)と見た目(CSS)を分離
再利用	ページごとに書き直す	クラス指定で使い回せる

3 CSSで何ができるようになった？



色・余白・フォントを一括管理できる



画面サイズに応じて見た目を変えられる



アニメーションも表現できる

関連キーワード

関連する技術



HTML

CSSが見た目を整える対象の文書構造。

P.121



CSSフレームワーク

よく使うCSSをあらかじめ用意した工具箱。

P.141

JavaScript

ひとことという

Web ページ内で計算や画面操作を行い、ユーザー操作に応じて動的に変化させる言語。

1 なぜ生まれた？

HTML/CSS だけでは静止した文書しか作れず、入力チェックやアニメーションなど。



2 それまでどうしていた？

それまではHTML/CSS だけの、動きのない静的なページが中心だった。

	静的なHTML/CSSのみ	JavaScript追加後
ページの振る舞い	リンクをクリックしたら別ページに遷移するだけ	クリックした場で表示を変えられる
入力チェック	サーバーに送ってからエラー表示	送信前にブラウザ内でチェックできる
更新方法	ページ全体を再読み込み	一部だけ書き換え可能

3 JavaScript で何ができるようになった？



クリックや入力に反応できる



ページの一部だけ書き換えられる



サーバーと通信もできる

関連キーワード

関連する技術



HTML

JavaScriptが操作する対象の構造。

P.121



CSS

JavaScriptが書き換える対象の見た目。

P.122



TypeScript

JavaScriptに型を追加した言語。

P.124

TypeScript

ひとことというと

JavaScriptに型情報を追加し、実行前にバグの芽をエディタ上で検出できるようにした言語。

1 なぜ生まれた？

JavaScriptは型が自由な分、実行するまで型ミスに気づきにくかった。



2 それまでどうしていた？

それまではJavaScriptのまま、型を意識せずに開発していた。

	JavaScript	TypeScript
型のチェック	実行するまでわからない	書いている時点でエディタが警告
大規模開発	どこで何の型が渡るか読みづらい	型定義がドキュメント代わりになる
実行環境	そのままブラウザ/Node.jsで動く	コンパイルしてJSに変換してから動く

3 TypeScript で何ができるようになった？



変数の型を事前に定義できる



エディタの補完が効きやすくなる



コンパイル時にミスに気づける

関連キーワード

関連する技術



JavaScript

TypeScriptのベースになった言語。

P.123



バニラ

型を付けない素のJavaScriptで書くこと。

P.125

バニラ (Vanilla JS)

ひとことというと

ReactやVueなどのライブラリを使わず、標準機能だけで書くJavaScriptのこと。

1 なぜ生まれた？

昔はjQueryのようなライブラリを使うのが当たり前で、素のJSで書くことがむしろ珍しくなり、「何も足していない」定番フレーバーになぞらえて呼ばれ始めた。



2 それまでどうしていた？

それまではブラウザごとの挙動の違いを吸収するために、jQueryなどが重宝されていた。

	jQuery	バニラJS (標準API)
要素の取得	<code>\$('#id')</code> のように書く	<code>document.querySelector()</code> のように書く
依存	ライブラリの読み込みが必須	追加ライブラリ不要
ブラウザ対応差	jQueryが吸収してくれた	標準APIが揃った今はほぼ気にしなくてよい

3 バニラ (Vanilla JS) で何ができるようになった？



追加のライブラリなしで動く



標準APIだけで完結する



書き方の自由度は自分次第

関連キーワード

関連する技術



JavaScript

バニラJSの元になる標準言語。

P.123



DOM

バニラJSが標準APIで直接操作する対象。

P.127

ES Modules

ひとことという

import/exportを使って、JavaScriptのコードをファイル単位で分割・再利用できるようにする標準の仕組み。

1 なぜ生まれた？

昔は<script>タグを並べる順番でグローバル変数がぶつかったり、ファイル同士の依存関係がコードから追えなかったりした。



2 それまでどうしていた？

それまでは<script>タグを並べる順番に頼って、グローバル空間を共有していた。

	グローバルスクリプト	ES Modules
ファイル分割	<script>タグの数だけ読み込み順に注意	import/exportで依存関係を明示
変数の衝突	グローバル空間を全ファイルで共有	ファイルごとにスコープが分離
再利用	コピーで使い回すことも多かった	exportした関数やクラスをimportするだけ

3 ES Modules で何ができるようになった？



必要な部分だけimportできる



依存関係がコードから追える



ビルドツールとも相性がよい

関連キーワード

関連する技術



JavaScript

ES Modulesが分割対象とする言語。

P.123

DOM

ひとことというと

ブラウザが読み込んだHTMLを、JavaScriptから操作できるようツリー状のオブジェクトとして表現したもの。

1 なぜ生まれた？

HTMLはただの文字列（テキスト）なので、そのままではJavaScriptから。



2 それまでどうしていた？

それまではHTMLをただの文字列として扱うしかなく、JSから直接触れなかった。

	HTML 文字列そのまま	DOM (オブジェクトの木)
JSからの操作	文字列を書き換えても反映されない	getElementById等で直接操作できる
構造の把握	テキストを読んで判断するしかない	親子・兄弟関係をたどれる
更新	ページ全体を再解釈	該当ノードだけ差し替え可能

3 DOM で何ができるようになった？



要素を取得・変更できる



イベントを取り付けられる



一部のノードだけ書き換えられる

関連キーワード

関連する技術



HTML

DOMのもとになる文書構造。

P.121



Event

DOMの要素に取り付ける反応の仕組み。

P.128



Virtual DOM

本物のDOMの代わりに使う仮想の木。

P.137

Event

ひとことというと

クリックやキー入力など、ユーザーやブラウザの動作をきっかけに JavaScript の処理を実行させる仕組み。

1

なぜ生まれた？

JavaScriptは基本的に上から順番に実行されるだけなので、「ボタンが押されたら」のようにタイミングを待って処理を始めたい場面に対応できなかった。



2

それまでどうしていた？

それまでは、状態が変わっていないか定期的にチェックし続ける発想しかなかった。

	定期的な状態チェック	Event
処理の開始	定期的に状態をチェックし続ける	発生したその瞬間に呼ばれる
無駄な処理	何も起きてなくてもチェックし続ける	何も起きなければ何もしない
コードの書き方	if文で状態を監視	addEventListenerで登録するだけ

3

Event で何ができるようになった？



クリックや入力に
反応できる



複数の処理を同時
に待ち受けられる



非同期処理ともよ
く組み合わせる

関連キーワード

関連する技術



DOM

Eventを取り付ける対象の要素。

P.127



非同期

(Asynchronous)

Eventの発生を待つ考え方の土台。

P.138



Component

Eventへの反応をまとめて持つ部品。

P.129

Component

ひとことという

ボタンやカードなど、画面の一部をひとまとまりの部品として切り出し、組み合わせて画面を作る考え方。

1 なぜ生まれた？

ページ全体でHTML/CSS/JSを書いていると、同じような見た目の部品をあちこちにコピペし、1箇所直すたびに全部を探して修正する必要があった。



2 それまでどうしていた？

それまではページ単位でHTML/CSS/JSをその都度書いていた。

	ページ単位で記述	Component単位で部品化
再利用	似た見た目をコピペで量産	1つのComponentを呼び出すだけ
修正	使っている箇所を全部探して直す	Component本体を直せば全箇所に反映
状態と見た目	HTMLとJSが別ファイルで散らばりがち	StateやPropsとセットで1部品にまとまる

3 Componentで何ができるようになった？



見た目と振る舞いをひとまとめでできる



何度でも使い回せる



組み合わせて画面を作れる

関連キーワード

関連する技術



State

Componentが内部に持つ今の状態。

P.130



Props

親から子のComponentへ渡す値。

P.131



Virtual DOM

Componentの変化を効率よく反映する仕組み。

P.137

State

ひとことというと

開閉中かどうか、入力中の文字など、Componentが内部で持ち続ける「今の状態」のデータ。

1 なぜ生まれた？

画面はクリックや入力のたびに見た目が変わる。「今どういう状態か」をどこかに覚えておかないと、変化のたびに画面全体を作り直すことになってしまう。



2 それまでどうしていた？

それまではDOMを直接書き換えて、その場その場で見た目を管理していた。

	DOMを直接書き換え	Stateで管理
見た目の変更	都度DOMを探して書き換える	Stateを更新するだけで見た目が追従
現在の状態の把握	DOMを見て判断するしかない	変数（State）を見ればわかる
バグの発生源	見た目とデータがずれることがある	データ（State）が唯一の正解

3 State で何ができるようになった？



「今の値」を保持できる



更新すると画面に反映される



Propsとして子に渡すこともできる

関連キーワード

関連する技術



Component

Stateを内部に持つ部品。

P.129



Props

Stateを子に渡すときの形。

P.131



Virtual DOM

Stateの変化を効率よく反映する仕組み。

P.137

Props

ひとことという

親の Component から子の Component へ、表示内容や設定値を渡すためのデータ。

1 なぜ生まれた？

同じボタン Component でも「送信」と表示したい場所と「キャンセル」と表示したい場所がある。



2 それまでどうしていた？

それまでは Component ごとに文言を書き換えたコピーを作っていた。

	コピーして文言を変更	Props で外から渡す
表示の出し分け	ボタンごとに Component をコピーして文言を変更	1 つの Component に Props で文言を渡す
修正のしやすさ	コピーした数だけ修正が必要	Component 本体を直せば全箇所に反映
データの向き	決まっていない	親 → 子への一方向

3 Props で何ができるようになった？



呼び出し側から表示内容を指定できる



Component を使い回しやすくなる



State と組み合わせる

関連キーワード

関連する技術



Component

Props を受け取る部品。

P.129



State

Props のもとになることが多いデータ。

P.130

CSR

ひとことという

サーバーからは最小限のHTMLとJavaScriptだけを送り、実際の画面はブラウザ側のJavaScriptが組み立てる方式。

1 なぜ生まれた？

ReactやVueのようなComponent志向のライブラリが普及し、画面遷移のたびにサーバーへ問い合わせず、ブラウザ内で高速に画面を差し替えたいというニーズが強まった。



2 それまでどうしていた？

それまではアクセスのたびにサーバーが完成したHTMLを生成して返していた。

	旧来のサーバーレンダリング	CSR
初回表示までの流れ	サーバーが完成形のHTMLを返す	空に近いHTMLを返し、JSが画面を組み立てる
画面遷移	ページごとにサーバーへリクエスト	遷移後の差分だけJSが書き換え
サーバーの負荷	表示のたびにHTMLを生成	初回以降はデータだけ返せばよい

3 CSRで何ができるようになった？



ページ遷移がヌルサクになる



Component中心に作りやすい



初期表示にはJS読み込みの時間がかかる

関連キーワード

関連するレンダリング方式



Component

CSRで画面を組み立てる部品の単位。

P.129



SSR

サーバー側で先にHTMLを完成させる方式。

P.133



Hydration

届いたHTMLにあとから動きを与える処理。

P.136

ひとことという

ブラウザで組み立てる前に、サーバー側で完成したHTMLを生成してから返す方式。

1 なぜ生まれた？

CSRは初回表示までJSの読み込み・実行を待つ必要があり、表示が遅れたり検索エンジンが内容をうまく認識できないという課題があった。



2 それまでどうしていた？

それまではCSRで、空のHTMLとJSを送ってからブラウザ側が画面を組み立てていた。

	CSR	SSR
初回表示までの流れ	空のHTML+JSを送りブラウザが組み立てる	サーバーが完成したHTMLを生成して送る
表示速度（初回）	JSの読み込み・実行を待つ	HTMLが届いた時点で見た目は表示される
SEO	検索エンジンが内容を認識しづらい場合がある	最初から完成したHTMLなので認識しやすい

3 SSRで何ができるようになった？



初回表示が速い



検索エンジンに強い



表示後にHydrationでJSを有効化する必要がある

関連キーワード

関連するレンダリング方式



CSR

ブラウザ側でHTMLを組み立てる方式。

P.132



SSG

ビルド時にあらかじめHTMLを作る方式。

P.134



Hydration

SSRで届いたHTMLにあとから動きを与える処理。

P.136

ひとことというと

アクセスのたびに生成する SSR とは違い、ビルドの時点で全ページの HTML を事前に作っておく方式。

1 なぜ生まれた？

SSR はアクセスされるたびにサーバーで HTML を生成するため、内容がほとんど変わらないページでも毎回同じ処理を繰り返し、サーバー負荷や速度の面で無駄が多かった。



2 それまでどうしていた？

それまでは SSR で、リクエストが来るたびにサーバーが HTML を生成していた。

	SSR	SSG
HTML 生成のタイミング	リクエストが来るたびに生成	ビルド時にあらかじめ全部生成
サーバーの負荷	アクセスごとに処理が発生	生成済みファイルを配るだけ
内容の更新	リクエスト時点の最新データを反映できる	再ビルドしないと内容が更新されない

3 SSG で何ができるようになった？



配信がとにかく速い



サーバー負荷がほぼゼロ



更新のたびに再ビルドが必要

関連キーワード

関連するレンダリング方式



SSR

アクセスのたびに HTML を生成する方式。

P.133



ISR

SSG を裏側で少しずつ更新する方式。

P.135

ひとことというと

SSGの速さを保ちながら、一定時間ごとや条件に応じて裏側でページを再生成し、内容の鮮度も両立させる方式。

1 なぜ生まれた？

SSGは速いが、内容を更新するたびにサイト全体を再ビルドする必要があり、更新頻度の高いページには不向きだった。



2 それまでどうしていた？

それまではSSGで、更新するたびにサイト全体を再ビルドしていた。

	SSG	ISR
内容の更新	サイト全体を再ビルドしないと反映されない	該当ページだけ裏側で再生成
表示速度	常に生成済みファイルを返すので速い	同じく生成済みファイルを返すので速い
鮮度	再ビルドするまで古いまま	一定時間経過や条件で自動的に新しくなる

3 ISR で何ができるようになった？



SSG並みの表示速度を維持



全体の再ビルドが不要



裏側で必要なページだけ更新

関連キーワード

関連するレンダリング方式



SSG

ビルド時にHTMLを事前生成する方式。

P.134



SSR

アクセスのたびにHTMLを生成する方式。

P.133

Hydration

ひとことという

SSR/SSG で先に届いたHTMLに対して、ブラウザ側でJavaScriptを結びつけ、クリックなどに反応できる状態にする処理。

1 なぜ生まれた？

SSR/SSG は見た目のHTMLを先に届けてくれるが、そのHTMLはただの文字列であり、まだボタンを押しても何も起きない「見た目だけ」の状態になっている。



2 それまでどうしていた？

それまでは（Hydrationなしで）届いたHTMLがそのまま最終形だった。

	SSR/SSGのみ (Hydrationなし)	SSR/SSG + Hydration
表示されるHTML	完成した見た目のHTMLが届く	同じく完成した見た目のHTMLが届く
クリックなどへの反応	ただのHTML文字列なので何も起きない	JSが結びつき、反応できるようになる
必要な処理	なし	ブラウザ側でJavaScriptを実行し直す

3 Hydration で何ができるようになった？



見た目は先に、反応はあとから



SSRやSSGとセットで使われる



完了までクリックしても反応しないことがある

関連キーワード

関連するレンダリング方式



SSR

Hydrationが必要になる方式のひとつ。

P.133



SSG

Hydrationが必要になる方式のひとつ。

P.134



CSR

HydrationではなくJSが最初から画面を組み立てる方式。

P.132

Virtual DOM

ひとことというと

実際のDOMを直接操作する代わりに、メモリ上に仮のツリーを作り、変化した部分だけを計算してから本物のDOMに反映する仕組み。

1 なぜ生まれた？

本物のDOMを直接何度も書き換えると、ブラウザが都度レイアウトの再計算を行うため、Stateが頻繁に変わる画面では動作が重くなりがちだった。



2 それまでどうしていた？

それまではStateが変わるたびに、本物のDOMを直接書き換えていた。

	DOMを直接操作	Virtual DOM経由
更新の単位	変更のたびに本物のDOMを書き換え	メモリ上の仮想ツリーで差分を計算してからまとめて反映
パフォーマンス	更新が多いと重くなりやすい	差分だけ反映するので効率的
書き方	DOM操作を自分で細かく制御	Stateの変化を宣言するだけで反映は任せられる

3 Virtual DOM で何ができるようになった？



差分だけを計算して反映



本物のDOM操作を減らせる



Stateの変化に強い

関連キーワード

関連する技術



DOM

Virtual DOMが最終的に反映する本物の木構造。

P.127



State

Virtual DOMが差分を計算するきっかけになるデータ。

P.130



Component

Virtual DOMを使って更新される部品の単位。

P.129

非同期 (Asynchronous)

ひとことというと

通信やファイル読み込みなど、時間のかかる処理の完了を待たずに、他の処理を並行して進められるようにする考え方。

1 なぜ生まれた？

JavaScriptは基本的に処理を1つずつ順番にこなすため、通信などの時間がかかる処理をそのまま待つ書き方をすると、その間ブラウザ全体が固まってしまう。



2 それまでどうしていた？

それまでは同期処理として、1つの処理が終わるまで次を待つのが基本だった。

	同期処理	非同期処理
通信中の挙動	結果が返るまで他の処理が止まる	結果を待たずに他の処理が続けられる
画面の反応	通信中は固まったように見えることがある	通信中も操作を受け付けられる
書き方の歴史	コールバック関数を渡す(コールバック地獄)	Promiseやasync・awaitで整理

3 非同期 (Asynchronous) で何ができるようになった？



通信中も画面が固まらない



複数の処理を並行して進められる



書き方はPromiseなどで進化してきた

関連キーワード

関連する技術



Promise

非同期処理の結果をまとめて扱うオブジェクト。

P.139



async・await

Promiseを同期処理のように書ける構文。

P.140



Event

非同期の発生をきっかけにする仕組み。

P.128

Promise

ひとことというと

非同期処理の結果（成功か失敗か）を、あとから受け取れる形でまとめて扱えるようにするオブジェクト。

1 なぜ生まれた？

非同期処理をコールバック関数で書くと、処理が増えるたびにコールバックの中に。



2 それまでどうしていた？

それまではコールバック関数を渡す方式で、非同期処理をつなげていた。

	コールバック関数	Promise
複数の非同期処理をつなげる	コールバックの中にコールバックがネストしていく	.then()で横に並べてつなげられる
エラー処理	各コールバックごとに書く必要がある	.catch()でまとめて扱える
見た目	ネストが深くなり右にどんどん寄っていく	処理の順番が上から下に読める

3 Promise で何ができるようになった？



成功・失敗をまとめて扱える



.then()で処理をつなげられる



async・awaitでさらに読みやすく書ける

関連キーワード

関連する技術



非同期
(Asynchronous)

Promiseが扱う処理そのものの考え方。

P.138



async・await

Promiseを読みやすく書くための構文。

P.140

async・await

ひとことというと

Promiseを使った非同期処理を、あたかも上から順番に処理しているかのように書けるようにする構文。

1 なぜ生まれた？

Promiseの.then()が増えると、処理の流れが追いにくなった。



2 それまでどうしていた？

それまではPromiseの.then()を数珠つなぎにする書き方が中心だった。

	Promiseの.then()	async・await
見た目	.then()を数珠つなぎにする	awaitを付けるだけで上から順に読める
エラー処理	.catch()をチェーンにつなげる	try/catchという馴染みのある書き方が使える
変数の受け渡し	.then()の引数として受け取る	通常の変数のように代入して使える

3 async・await で何ができるようになった？



同期処理のような自然な見た目



try/catchでエラー処理できる



中身の仕組みはPromiseのまま

関連キーワード

関連する技術



Promise

async・awaitの土台になっているオブジェクト。

P.139



非同期 (Asynchronous)

async・awaitが扱う処理の考え方。

P.138

CSS フレームワーク

ひとことという

ボタンやレイアウトなど、よく使うデザインパーツのCSSをあらかじめ用意しておき、クラス名を付けるだけで使えるようにしたもの。

1 なぜ生まれた？

CSSは自由度が高い分、余白や色、レイアウトなどの細部を一から調整するのに時間がかかり、デザイナーがいないチームでは見た目の統一も難しかった。



2 それまでどうしていた？

それまではCSSを一から書いて、見た目を1つずつ作り込んでいた。

	CSSを一から書く	CSSフレームワーク
見た目の作成	余白・色・角丸等を1つずつ指定	用意されたクラスを付けるだけ
デザインの統一感	チームやページごとにばらつきが出やすい	フレームワークのルールで自然と揃う
学習コスト	CSSの知識が幅広く必要	クラス名の一覧を覚えれば書き始められる

3 CSSフレームワーク で何ができるようになった？



クラス名を付けるだけで見た目が整う



チーム内でデザインが揃いやすい



独自色を出すにはカスタマイズが必要

関連キーワード

関連する技術



CSS

CSSフレームワークの土台になる言語。

P.122



UIコンポーネントライブラリ

見た目に加えて動きまで含む部品を提供するもの。

P.142

UIコンポーネントライブラリ

ひとことという

ボタンやモーダルなど、見た目だけでなく開閉などの動作（振る舞い）まで含んだComponentを、そのまま使える形で提供するライブラリ。

1 なぜ生まれた？

ボタンやフォームなど定番UI部品を、毎回個別実装する負担を減らす必要があった。



2 それまでどうしていた？

それまではCSSフレームワークで見た目だけ整え、動きの部分は自分で実装していた。

	CSSフレームワーク	UIコンポーネントライブラリ
提供される範囲	見た目（クラス名）のみ	見た目 + 開閉等の動作込みのComponent
実装コスト	モーダル等の開閉ロジックは自分で書く	importして使うだけで動く
カスタマイズ性	自由度が高い分すべて自作	用意されたPropsの範囲での調整が中心

3 UIコンポーネントライブラリで何ができるようになった？



ボタンやモーダルがそのまま使える



見た目と動きがセットで届く



Props経由で細かく調整できる

関連キーワード

関連する技術



CSSフレームワーク

見た目の材料を提供するもの。

P.141



Component

UIコンポーネントライブラリが提供する部品の単位。

P.129



Props

UIコンポーネントライブラリの部品を調整する手段。

P.131

Node.js

ひとことという

ブラウザの中でしか実行できなかった JavaScript を、サーバーやPC上で動かせるようにするランタイム。

1 なぜ生まれた？

JavaScript はブラウザの中でしか実行できず、サーバーサイドは別の言語で書くしかなかった。



2 それまでどうしていた？

サーバーサイドは Ruby・PHP・Java など、フロントとは別の言語で書いていた。

	サーバー専用言語	Node.js
言語	Ruby / PHP / Java 等	JavaScript (フロントと共通)
実行エンジン	各言語専用のランタイム	Chrome の V8 エンジン
得意分野	汎用処理全般	I/O 待ちが多い並行処理

3 Node.js で何ができるようになった？



サーバーサイドを JS で書ける



フロントとバックの言語を統一



npm エコシステムを利用可能

関連キーワード

仲間のランタイム



Bun

Node.js 互換で起動・実行をまるごと高速化。

P.144



Deno

Node.js 作者が作り直した安全志向のランタイム。

P.145

Bun

ひとことというと

Node.js 互換を保ちながら、実行・パッケージ管理・テストまで1つのバイナリで高速化したランタイム。

1 なぜ生まれた？

Node.js 周辺では、起動速度とツール分散をまとめて改善する需要が高まっていた。



2 それまでどうしていた？

Node.js では、実行は node、パッケージ管理は npm、テストは Jest のように道具を組み合わせていた。

	Node.js (組み合わせ型)	Bun
実行エンジン	V8	JavaScriptCore (Safari系)
パッケージ管理	npm / yarn を別途利用	標準搭載で高速
ツール構成	node + npm + Jest 等別々	1つのバイナリに統合

3 Bun で何ができるようになった？



起動・実行がとにかく速い



パッケージ管理も内蔵



テストランナーも標準装備

関連キーワード

比較する技術



Node.js

JSをブラウザの外で動かす元祖ランタイム。

P.143



Deno

安全性を重視したもう1つの後継ランタイム。

P.145

Deno

ひとことというと

デフォルトでは何もアクセスできず、TypeScriptをそのまま実行できる安全志向のJSランタイム。

1

なぜ生まれた？

Node.js は自由度が高い反面、npm パッケージが暗黙に。



2

それまでどうしていた？

Node.js では、パッケージは特に制限なくファイル・ネットワーク・環境変数へアクセスできていた。

	Node.js	Deno
権限	デフォルトで全許可	デフォルトで全拒否 (明示許可制)
TypeScript	別途トランスパイル設定が必要	標準でそのまま実行可能
パッケージ管理	npm + node_modules	URL import も可能 (npm 互換も追加済)

3

Deno で何ができるようになった？



ファイル/ネット
権限を明示指定



TypeScript をその
まま実行



Deno Deploy で手
軽にデプロイ

関連キーワード

比較する技術



Node.js

JSをブラウザの外で動かす元祖ランタイム。

P.143



Bun

Node.js互換で速さに振り切ったランタイム。

P.144

React

ひとことという

UIを再利用可能なコンポーネント単位で組み立てるためのJavaScriptライブラリ。

1 なぜ生まれた？

素のJS・DOM操作で画面を作ると、状態が変わるたびに。



2 それまでどうしていた？

それまではjQueryなどでDOMを直接操作し、画面の状態を自分で追跡・更新していた。

	素のJS / jQuery	React
更新方法	DOMを直接書き換え	状態 (State) からUIを再計算
再利用	コピペで使い回し	コンポーネントとして部品化
学習コスト	低い	JSX・Hooks等独自の書き方あり

3 React で何ができるようになった？



画面をコンポーネント単位で管理



状態が変わると自動で再描画



Next.js など周辺エコシステムが豊富

関連キーワード

比較する技術



Vue

HTMLに近い書き味で始められるフレームワーク。

P.147

関連する技術



Next.js

Reactにルーティングなどを標準搭載したフレームワーク。

P.148



Redux

アプリ全体の状態を1箇所にまとめる状態管理。

P.163

Vue

ひとことというと

テンプレート構文でHTMLに近い書き方をしながら、コンポーネント単位で画面を作るフレームワーク。

1 なぜ生まれた？

React は強力だったが、JSX という独自記法の学習コストがあり、「もっとHTML/CSSに近い書き方で始めたい」という声があった。



2 それまでどうしていた？

React では、JSX の中に HTML に似せた JS 構文で UI を書いていた。

	React (JSX)	Vue (テンプレート)
記法	JS 内に HTML っぽい構文 (JSX)	HTML に <template> タグで分離
学習の入り口	JS の知識が前提	HTML/CSS からでも始めやすい
状態管理の書き方	useState 等の Hooks	ref / reactive 等専用 API

3 Vue で何ができるようになった？



HTML/CSS/JS を
分けて書ける



双方向バインディングが手軽



公式ドキュメント
が読みやすい

関連キーワード

比較する技術



React

状態からUIを再計算するコンポーネント指向ライブラリ。

P.146

関連する技術



Nuxt.js

Vueにルーティングなどを標準搭載したフレームワーク。

P.149



Pinia

Vue公式が推奨する状態管理ライブラリ。

P.165

Next.js

ひとことという

Reactにルーティングやサーバーサイドレンダリングなどを標準搭載した、本番運用向けフレームワーク。

1 なぜ生まれた？

Reactだけではページ遷移やSSRを毎回自分で組み合わせる必要があった。



2 それまでどうしていた？

Reactだけの構成では、ルーティングライブラリやSSR用サーバーを個別に用意していた。

	Reactのみ（素の構成）	Next.js
ルーティング	別ライブラリを追加・設定	ファイル配置だけで自動生成
描画方式	基本CSRのみ	SSR / SSG / ISRを選択可能
初期設定	自分で構成を決める	フレームワークが規約として提供

3 Next.js で何ができるようになった？



ファイルベースでルーティング自動化



SSR/SSGを標準サポート



Vercelへワンコマンドでデプロイ

関連キーワード

土台の技術



React

コンポーネント単位でUIを組み立てるライブラリ。

P.146

比較する技術



Nuxt.js

VueにとってのNext.js的な存在。

P.149

Nuxt.js

ひとことという

Vueにルーティングやサーバーサイドレンダリングなどを標準搭載した、本番運用向けフレームワーク。

関連キーワード

土台の技術



Vue

HTMLに近い書き味のコンポーネント指向フレームワーク。

P.147

比較する技術



Next.js

ReactにとってのNuxt.js的な存在。

P.148

1 なぜ生まれた？

Vueも同様に、ルーティングやSSRを個別に組み合わせる必要があり、プロジェクトごとに構成がバラつく問題があった。



2 それまでどうしていた？

Vueのみの構成では、vue-routerなどを個別に追加し、SSRも自前でサーバーを用意していた。

	Vueのみ（素の構成）	Nuxt.js
ルーティング	vue-routerを個別設定	ファイル配置だけで自動生成
描画方式	基本CSRのみ	SSR / SSG / ISRを選択可能
設定ファイル	プロジェクトごとにバラバラ	規約で統一される

3 Nuxt.js で何ができるようになった？



ファイルベースでルーティング自動化



SSR/SSGを標準サポート



Vueのモジュールを組み込みやすい

Vite

ひとことというと

ブラウザのESモジュール機能を使って、開発中の画面表示をほぼ一瞬で反映するビルドツール。

1

なぜ生まれた？

従来のビルドツールは、ファイルを1つ書き換えるたびに。



2

それまでどうしていた？

従来はWebpackなどでJS全体を事前にバンドルしてからブラウザに渡していた。

	Webpack等（バンドル型）	Vite
開発時の起動	全体をバンドルしてから起動	ESモジュールでそのまま配信
変更の反映	変更範囲によって再バンドル	変更ファイルだけ即時反映（高速HMR）
本番ビルド	同じ仕組みでビルド	Rollupで最適化ビルド

3

Vite で何ができるようになった？



開発サーバーの起動が高速



保存した瞬間に画面へ反映



本番用ビルドは別途最適化

関連キーワード

関連する技術



React

Viteの定番テンプレートの1つ。

P.146



ESLint

開発時のコードチェックを担う Linter。

P.151

ESLint

ひとことというと

コードのルール違反やバグの芽を、実行する前に静的解析で指摘してくれる Linter。

1 なぜ生まれた？

未使用変数や書き方のばらつき、うっかりバグにレビューまで気づけず、レビューコストや事故が増えていた。



2 それまでどうしていた？

それまでは、コードレビューで人力で指摘するか、規約を文書として共有するだけだった。

	人力レビュー・規約文書	ESLint
チェックのタイミング	レビュー時（後追い）	保存・コミット時（即座）
基準のブレ	レビュアーによって差が出る	ルールとして統一・自動化
対応範囲	見つけたものだけ	プロジェクト全体を機械的に走査

3 ESLint で何ができるようになった？



ルール違反を自動検出



保存時に自動修正も可能



チーム全体でルールを統一

関連キーワード

高速化を競う後継



Biome

Lintと整形をRustで1本にまとめたツール。

P.152



OXC

パーサーからRustで作り直すプロジェクト。

P.153

Biome

ひとことというと

Lint（コードチェック）とFormat（整形）をRust製の1つのツールでまとめて高速に行うツール。

1 なぜ生まれた？

ESLintとPrettierの二重設定と、大規模プロジェクトでの遅さが課題だった。



2 それまでどうしていた？

従来はESLint（Lint用）とPrettier（整形用）をそれぞれJavaScriptで実装し、組み合わせて使っていた。

	ESLint + Prettier（組み合わせ）	Biome
役割	Lintと整形を別ツールで担当	1つのツールでLint+整形
実装言語	JavaScript	Rust（高速）
設定ファイル	2つ（.eslintrc / .prettierrc）	1つに統一

3 Biome で何ができるようになった？



Lint・整形が高速



1ツールで設定が完結



ESLint設定からの移行支援あり

関連キーワード

比較する技術



ESLint

JS実装の老舗 Linter。

P.151



OXC

パース処理から Rustで高速化する 次の一手。

P.153

OXC

ひとことというと

パーサー・Linter・整形・ビルドまで、JS周辺ツールをまとめてRustで高速に作り直すプロジェクト。

1 なぜ生まれた？

Lintや整形は速くなっても、パースやビルドはまだJS実装が多かった。



2 それまでどうしていた？

それまでは、Lintと整形はBiomeやESLintのように個別最適化され、パーサーやバンドラーは各ツールがバラバラに実装していた。

	JS実装のツール群 (個別最適化)	OXC
対象範囲	Lint / 整形等個別ツール	パーサー〜ビルドまで一貫してRust化
速度	ツールごとに差がある	全体を通してネイティブ速度
採用状況	各社が個別に高速化を試す	Vite等他ツールへの組み込みが進行中

3 OXC で何ができるようになった？



パース処理から
ネイティブ高速化



他ツールへの組み
込みが進行中



JSツールチェイ
ン全体を刷新

関連キーワード

高速化競争のはじまり



ESLint

JS実装の老舗
Linter。

P.151



Biome

Rustで1本化した
中継地点。

P.152

Tailwind CSS

ひとことについて

あらかじめ用意された小さなユーティリティクラスをHTMLに直接並べてスタイルを作るCSSフレームワーク。

1 なぜ生まれた？

CSSでは、要素ごとに意味のあるクラス名を考えて別ファイルに。



2 それまでどうしていた？

それまでは、独自クラス名を都度設計するCSS設計（BEMなど）で対応していた。

	独自CSS設計 / BEM など	Tailwind CSS
書く場所	別ファイルにCSSを記述	HTMLのclass属性に直接記述
命名	要素ごとに意味のある名前を考える	用意されたユーティリティ名を組み合わせる
見た目の変更	CSSファイルを探して修正	HTML上でクラスを足し引き

3 Tailwind CSS で何ができるようになった？



クラス名を考えなくてよい



HTML上で見た目が完結



未使用クラスはビルド時に自動除去

関連キーワード

組み合わせる技術



MUI

既製のUI部品ごと使えるコンポーネント集。

P.155



shadcn/ui

コードをコピーして使うUI集。

P.156

ひとことというと

Googleのマテリアルデザインに沿ったボタン・フォームなどのReact用UIコンポーネント集。

1 なぜ生まれた？

Tailwindだけでは、ボタンやモーダルなど定番部品を毎回組み立てる手間があった。



2 それまでどうしていた？

それまでは、Tailwindなどでスタイルだけを当てつつ、UI部品自体は自分でHTML構造から組み立てていた。

	自前でUI部品を実装	MUI
部品の中身	都度HTML構造から実装	ボタン・モーダル等を既製品として提供
見た目の統一感	都度デザイン判断が必要	マテリアルデザインの基準に沿う
カスタマイズ	自由（だが手間）	テーマ機能で配色等を調整

3 MUI で何ができるようになった？



既製のUI部品を組み込むだけ



マテリアルデザインで統一感



テーマ設定で配色を一括変更

関連キーワード

比較する技術



Tailwind CSS

ユーティリティクラスでスタイルを組み立てる。

P.154



shadcn/ui

コードをコピーして使う別解のUI集。

P.156

shadcn/ui

ひとことという

MUIのような既製部品ライブラリではなく、必要な分だけコードをプロジェクトにコピーして使うUIコンポーネント集。

1 なぜ生まれた？

MUIのようなライブラリは便利だが、内部の見た目を細かく。



2 それまでどうしていた？

MUIなどでは、npmでライブラリを追加し、用意されたコンポーネントをそのままimportして使っていた。

	MUI等（ライブラリ追加型）	shadcn/ui
導入方法	npm installでライブラリ追加	CLIでソースコードごとプロジェクトへコピー
カスタマイズ	ライブラリの制約内で調整	コードを直接編集して自由に変更
依存関係	ライブラリ本体に依存し続ける	コピー後は自分のコードとして管理

3 shadcn/ui で何ができるようになった？



コードを直接プロジェクトへコピー



見た目を細部まで自由に編集



Tailwind CSSと組み合わせて使う

関連キーワード

比較する技術



MUI

既製部品をライブラリとして利用する方式。

P.155



Tailwind CSS

見た目を組み立てるユーティリティCSS。

P.154

Electron

ひとことというと

HTML・CSS・JavaScriptで書いた画面を、そのままWindows/Mac用のデスクトップアプリとして動かす仕組み。

関連キーワード

比較する技術



PWA

ストア審査なしでアプリっぽさを出す別解。

P.158

1 なぜ生まれた？

デスクトップアプリを作るには、OSごとに別の言語・UI部品を。



2 それまでどうしていた？

それまでは、C++/C#/SwiftなどOSごとのネイティブ言語・フレームワークで別々に実装していた。

	ネイティブ実装 (OSごと)	Electron
言語	C++ / C# / Swift 等	HTML / CSS / JavaScript
対応OS	OSごとに別実装が必要	1つのコードで複数OSに対応
代表例	—	VS Code / Slack / Discord 等

3 Electron で何ができるようになった？



1つのコードで複数OSに対応



Web技術の知識をそのまま流用



ブラウザ機能ごとアプリに同梱

PWA

ひとことというと

ホーム画面への追加やオフライン動作など、通常のWebサイトにネイティブアプリに近い体験を持たせる技術。

1 なぜ生まれた？

ネイティブアプリはストア審査やインストールの手間があり、「開くだけで使えて、でもアプリっぽい体験も欲しい」という要望があった。



2 それまでどうしていた？

それまでは、ネイティブアプリ（iOS/Android別）かElectronのようなデスクトップ包装で「アプリらしさ」を実現していた。

	ネイティブアプリ / Electron	PWA
配布方法	ストア経由 / インストーラー配布	URLを開くだけ・ホーム画面に追加
オフライン対応	各プラットフォームの機能で実装	Service Workerで実現
審査	ストア審査が必要な場合あり	審査不要（Webの延長）

3 PWA で何ができるようになった？



ホーム画面に追加できる



オフラインでも一部動作



ストア審査なしで配信・更新

関連キーワード

比較する技術



Electron

Web技術でデスクトップアプリを作る仕組み。

P.157

関連する指標



Core Web Vitals

ページの体感品質を数値化する指標。

P.162

WebAssembly

ひとことという

C/C++/Rustなどで書いたコードをブラウザ上でネイティブに近い速度で実行できるバイナリ形式。

関連キーワード

関連する技術



Core Web Vitals

速度も含めたページ品質を測る指標。

P.162

1 なぜ生まれた？

画像・動画処理やゲームなど重い処理をJavaScriptだけで行くと速度が足りず、ブラウザでは実現しづらい用途があった。



2 それまでどうしていた？

それまでは、重い処理はサーバー側で行うか、JavaScriptを愚直にチューニングして何とか動かしていた。

	JS実行のみ / サーバーで処理	WebAssembly
実行速度	JSエンジン依存で限界がある	ネイティブに近い速度
書ける言語	JavaScript（派生含む）のみ	C / C++ / Rust / Go等をコンパイルして実行
用途例	—	画像編集・ゲーム・動画処理等

3 WebAssembly で何ができるようになった？



重い処理をブラウザ内で高速実行



JSと連携して部分的に利用可能



C/Rust資産をWebに持ち込める

a11y（アクセシビリティ）

ひとことというと

視覚・聴覚・身体的な違いなどに関わらず、誰もがWebサイトを使えるようにする設計・実装の総称。

1 なぜ生まれた？

見た目の実装に集中し、スクリーンリーダーやキーボード操作が後回しになりがちだった。



2 それまでどうしていた？

それまでは「動けばOK」「見た目が良ければOK」という基準で実装され、a11yは余力があれば対応する程度だった。

	見た目・動作優先の実装	a11y対応の実装
対象ユーザー	標準的な操作ができる人が前提	視覚/聴覚/身体的な違いがある人も含む
チェック項目	デザイン通りに動くか	altテキスト・コントラスト比・操作性も確認
評価軸	見た目の完成度	WCAG等の基準への準拠度

3 a11y（アクセシビリティ）で何ができるようになった？



誰でも使える設計を評価軸に追加



altテキストやラベルを適切に付与



キーボードのみでも操作可能に

関連キーワード

関連する語



i18n（国際化）

言語や地域による差分を吸収する対応。

P.161



Core Web Vitals

速度など体感品質を測る指標。

P.162

i18n（国際化）

ひとことでいうと

アプリの文言や日付・通貨の表記を、言語や地域ごとに切り替えられるようにする仕組み・対応のこと。

関連キーワード

関連する語



a11y（アクセシビリティ）

誰もが使える設計にするための視点。

P.160

1 なぜ生まれた？

最初は日本語だけを想定して文言をコードに直接書いていたが、海外展開が必要になり、箇所を洗い出す羽目になった。



2 それまでどうしていた？

それまでは、表示文言をコンポーネントの中に直接ハードコーディングしていた。

	文言を直接ハードコーディング	i18n対応
文言の管理	コード内に直接記述	言語ごとの辞書ファイルに分離
言語切り替え	対応不可（作り直しが必要）	辞書を差し替えるだけで対応
日付・通貨表記	決め打ちの書式	ロケールに応じて自動整形

3 i18n（国際化）で何ができるようになった？



言語ごとに辞書ファイルを切り替え



日付・通貨をロケール対応で表示



後から言語追加がしやすい

ひとことという

表示速度・操作反応・レイアウトのガタつきなど、Web ページの体感品質を3つの指標で数値化したもの。

1 なぜ生まれた？

「サイトが遅い気がする」「レイアウトがガクッと動く」といった。



2 それまでどうしていた？

それまでは、ページ速度は主観的な感想や、ツールごとにバラバラな独自指標で語られていた。

	主観的な感想 / 独自指標	Core Web Vitals
評価方法	「なんか遅い」等の感覚	LCP / INP / CLS という共通指標で数値化
比較のしやすさ	ツール・人によってバラバラ	業界標準の指標で横並び比較可能
SEO との関係	明確な関連は薄かった	検索順位の評価要素の1つに

3 Core Web Vitals で何ができるようになった？



表示速度を LCP で数値化



操作の反応速度を INP で測定



レイアウトのガタつきを CLS で測定

関連キーワード

関連する語



PWA

アプリっぽい体験を実現する技術。

P.158



a11y (アクセシビリティ)

誰もが使える設計にするための視点。

P.160

Redux

ひとことという

アプリ全体の状態（State）を1箇所にまとめて管理する、React向けの状態管理ライブラリ。

関連キーワード

土台の技術



React

コンポーネント単位でUIを組み立てるライブラリ。

P.146

軽量な後継



Zustand

手順の少ない軽量な状態管理。

P.164

1

なぜ生まれた？

画面が複雑になるにつれ、Props経由で何段もコンポーネントを。



2

それまでどうしていた？

それまでは、Reactの標準機能（useState/Props）だけで、必要な状態を親から子へ渡していた。

	useState + Propsの パケツリレー	Redux
状態の置き場所	各コンポーネントに分散	ストアという1箇所に集約
更新方法	各コンポーネントで直接更新	Action/Reducerという決まった手順で更新
追跡のしやすさ	どこで変わったか追にくい	変更履歴を1本の流れとして追跡可能

3

Redux で何ができるようになった？



状態を1箇所のストアに集約



離れたコンポーネント間でも状態共有



決まった手順でしか状態を変更できない

Zustand

ひとことというと

Action/Reducerのような決まった手順を省き、少ないコードでReactの状態を共有できる軽量な状態管理ライブラリ。

1 なぜ生まれた？

Reduxは強力だが、小〜中規模の画面には覚えることが多すぎた。



2 それまでどうしていた？

それまでは、状態管理といえばRedux一択で、小さな機能追加にもRedux一式の手順を書いていた。

	Redux（手順が多い）	Zustand
書く量	Action/Reducer/Dispatchを定義	1つの関数でストアを定義するだけ
学習コスト	専用の設計思想を理解する必要	素のJSの感覚に近く読み書きしやすい
向いている規模	大規模・チーム開発向け	小〜中規模でも導入しやすい

3 Zustandで何ができるようになった？



少ないコードで状態を共有



覚えることが少ない



必要な機能だけ後から追加可能

関連キーワード

比較する技術



Redux

手順は多いが予測可能性の高い状態管理。

P.163



Pinia

Vue版の似た立ち位置の状態管理。

P.165

Pinia

ひとことというと

Vue 公式が推奨する、TypeScript との相性やシンプルさを重視した状態管理ライブラリ。

1 なぜ生まれた？

Vue の状態管理は Vuex が定番だったが、TypeScript との相性の。



2 それまでどうしていた？

それまでは、Vuex という公式の状態管理ライブラリで Mutation・Action を分けて状態を更新していた。

	Vuex (Mutation/Action 分離)	Pinia
更新の手順	Mutation と Action を分けて定義	Action だけで状態を直接更新可能
TypeScript 対応	型推論が弱く相性が悪い	型推論が効きやすい設計
公式の立ち位置	長らく公式推奨だった	現在の公式推奨はこちら

3 Pinia で何ができるようになった？



ストアをシンプルな構文で定義



型推論が効いて書きやすい



複数ストアに分割して管理しやすい

関連キーワード

土台の技術



Vue

HTML に近い書き味のコンポーネント指向フレームワーク。

P.147

比較する技術



Zustand

React 版の似た立ち位置の状態管理。

P.164

Ruby

ひとことという

書きやすさ・読みやすさを最優先に設計された、Ruby on Railsの生みの親でもあるプログラミング言語。

1 なぜ生まれた？

Perlなど既存の言語は便利だが、人間の感情に寄り添っていないと感じたまつもとゆきひろ氏が、「プログラマが幸せになる言語」を目指して開発した。



2 それまでどうしていた？

それまではCやPerlなど、書きやすさより実行効率や記号的な簡潔さを優先する言語が主流だった。

	C / Perl など	Ruby
設計の軸	実行速度・記号の簡潔さ	書きやすさ・読みやすさ
文法	記号中心	英語に近い自然な文法
オブジェクト指向	言語による	全てがオブジェクト
向いている場面	システムプログラミング等	Web開発・スクリプト

3 Ruby で何ができるようになった？



書きやすいコード
が書ける



Web アプリを素
早く開発できる



豊富なgemで機能
を拡張できる

関連キーワード

関連する技術



Rails

RubyのWebフレームワーク。設定より規約の思想で作られている。

P.177

比較する言語



Python

同じく書きやすさを重視するスクリプト言語。

P.172

172 Python

ひとことという

インデントで構造を表す簡潔な文法を持ち、Web 開発・データ分析・機械学習まで幅広く使われる言語。

1 なぜ生まれた？

「読みやすいコードこそ書きやすいコード」という思想のもと、Guido van Rossum が誰にでも読める言語を目指して開発した。



2 それまでどうしていた？

それまでは{}やendでブロックを表す言語が主流で、書き手によってコードの見た目がバラバラになりがちだった。

	他の主要言語 (Java/C++など)	Python
ブロックの表現	{ }やend	インデント (強制)
文法量	多い・冗長になりがち	少ない・簡潔
用途の広さ	得意分野が限定的	Web・AI・自動化まで横断
学習コスト	高め	低め (初学者にも人気)

3 Python で何ができるようになった？



シンプルな文法で
書ける



AI・データ分析に
も使える



ライブラリが豊富

関連キーワード

関連する技術



Django

電池付属の思想を持つフルスタックフレームワーク。

P.180



FastAPI

型ヒントを活かした高速なAPIフレームワーク。

P.181



Flask

必要な機能だけを足していく軽量フレームワーク。

P.182

比較する言語



Ruby

同じく書きやすさを重視するスクリプト言語。

P.171

ひとことというと

並行処理を言語レベルでサポートし、シンプルな文法と高速な実行を両立する Google 製の言語。

1 なぜ生まれた？

Google の大規模なコードベースで、C++ の複雑なビルドや Java の冗長さに悩まされていたエンジニアたちが、「シンプルに書いて、速く動く」言語を目指して開発した。



2 それまでどうしていた？

それまでは C++ や Java で並行処理を書く場合、スレッドやロックを自前で細かく管理する必要があった。

	C++ / Java	Go
並行処理の書き方	スレッド・ロックを自前管理	goroutine で軽量に書ける
文法の量	多機能・複雑になりがち	意図的にシンプル
コンパイル	遅い・セットアップが重い場合も	高速コンパイル・単一バイナリ
向いている場面	大規模アプリ全般	インフラ・API・CLI ツール

3 Go で何ができるようになった？



高速に動作する



並行処理を手軽に書ける



単一バイナリで配布できる

関連キーワード

関連する技術



Gin

Go の速さを活かしたシンプルな Web フレームワーク。

P.183

比較する言語



Rust

速度に加えてメモリ安全性をコンパイル時に保証する言語。

P.176

ひとことというと

HTMLに埋め込んで使える手軽さから広まり、大規模なWebサービスの基盤にもなっているサーバーサイド言語。

1 なぜ生まれた？

個人サイトのアクセス数を数える簡単なツールとして誕生し、「HTMLに直接埋め込んでサーバーサイドの処理を書ける」。



2 それまでどうしていた？

それまでのWebページはCGIとしてC言語やPerlスクリプトを別途用意し、HTMLと処理を分けて書く必要があった。

	CGI (C/Perl など)	PHP
HTMLとの関係	別ファイル・別プロセスで連携	HTMLに直接埋め込む
導入の手軽さ	サーバー設定が煩雑	レンタルサーバーでもすぐ動く
学習コスト	高め	低め (Web特化で分かりやすい)
向いている場面	汎用的な処理全般	動的なWebページ生成

3 PHP で何ができるようになった？



HTMLに処理を埋め込める



手軽にWebサーバーを動かせる



CMS・ECの基盤として定番

関連キーワード

関連する技術



Laravel

PHPをモダンにしたフルスタックフレームワーク。

P.184

比較する言語



Ruby

同じく書きやすさを軸にしたスクリプト言語。

P.171

Java

ひとことというと

JVM上で動作し、書いたコードをOSを問わず動かせることを武器に、企業システムの定番であり続ける言語。

1

なぜ生まれた？

家電向けの組み込み言語として開発が始まったが、「一度書けば、どこでも動く（Write Once, Run Anywhere）」。



2

それまでどうしていた？

それまではCやC++でOSごとにコンパイルし直す必要があり、動作環境を変えるたびに移植作業が発生していた。

	C / C++	Java
動作環境	OSごとに再コンパイルが必要	JVM上でOSを問わず動く
メモリ管理	手動（自分で解放）	自動（ガベージコレクション）
型の厳しさ	言語による	静的型付けで厳格
向いている場面	OS・ハードウェア寄りの処理	大規模・長期運用の業務システム

3

Java で何ができるようになった？



OSを問わず動かせる



大規模開発でも破綻しにくい



企業システムの定番として実績豊富

関連キーワード

関連する技術



Spring Boot

Javaの設定を自動化し、すぐに動くアプリを作れるフレームワーク。

P.185

比較する言語



Go

シンプルさと並行処理のしやすさを重視した言語。

P.173

Rust

ひとことという

メモリ管理の安全性をコンパイラがチェックすることで、C/C++並みの速度と高い安全性を両立する言語。

1 なぜ生まれた？

C/C++は高速だが、メモリの解放忘れや二重解放といったバグが実行時にクラッシュや脆弱性を引き起こしやすかった。



2 それまでどうしていた？

それまでのC/C++では、メモリの確保・解放をプログラマが手動で管理し、ミスがあっても実行するまで気づけないことが多かった。

	C / C++	Rust
メモリ管理	手動（解放忘れのリスクあり）	コンパイラが所有権をチェック
バグの発見	実行時にクラッシュして判明	コンパイル時にエラーとして検出
実行速度	速い	同等に速い
向いている場面	OS・組み込み・ゲーム等	同上 + WebAssembly 等

3 Rust で何ができるようになった？



安全性と速度を両立できる



メモリ安全性をコンパイル時に保証



C/C++並みの実行速度

関連キーワード

比較する言語



Go

シンプルさと開発速度を重視した言語。

P.173

Rails

ひとことについて

Ruby製のWebフレームワーク。「設定より規約（CoC）」の思想で、少ないコードでWebアプリを組み立てられる。

1 なぜ生まれた？

RubyでWebアプリを作る際、ルーティングやDB接続を毎回個別に組み立てる負担が大きかった。



2 それまでどうしていた？

それまでのWebアプリ開発では、フレームワークごとに細かい設定ファイルを書き、命名やディレクトリ構成も一から決める必要があった。

	設定ベースのフレームワーク	Rails
開発方針	設定ファイルで細かく指定	規約に従えば設定不要（CoC）
DB操作	SQLを都度記述	ActiveRecordで直感的に操作
初期構築	構成をゼロから設計	rails newで即座に土台が完成
向いている場面	自由な設計が必要な場合	素早くWebサービスを立ち上げたい場合

3 Rails で何ができるようになった？



短期間でWebアプリを構築できる



MVCで役割を整理できる



gemで機能を簡単に追加できる

関連キーワード

関連する技術



Ruby

書きやすさ・読みやすさを重視するプログラミング言語。

P.171

比較するフレームワーク



Laravel

PHP版のフルスタックフレームワーク。

P.184



Django

Python版のフルスタックフレームワーク。

P.180

178

Express

ひとことというと

Node.js 上で動く、必要なものを自分で組み合わせるミニマルな Web フレームワーク。

1 なぜ生まれた？

Node.js だけで Web サーバーを書こうとすると、ルーティングやリクエスト処理を毎回自前で実装する必要があった。



2 それまでどうしていた？

それまでは Node.js の標準モジュール (http) だけを使い、ルーティングなどを自分でゴリゴリ実装する必要があった。

	Node.js 標準 (http モジュール)	Express
ルーティング	自前で分岐処理を実装	app.get() 等で簡潔に記述
機能の追加	全て自作	ミドルウェアを差し込むだけ
構成の自由度	低い (そもそも骨組みがない)	高い (薄いので好きに組める)
向いている場面	極小のスク립ト	API・軽量の Web サーバー

3 Express で何ができるようになった？



最小限の記述で API を作れる



ミドルウェアで機能を拡張できる



構成を自由に設計できる

関連キーワード

比較するフレームワーク



Hono

Edge 環境向けに生まれた、より軽量のフレームワーク。

P.179

Hono

ひとことというと

TypeScript で書かれ、Node.js 以外の Edge ランタイムでも高速に動く軽量 Web フレームワーク。

1 なぜ生まれた？

Edge 環境では Node.js 前提のフレームワークがそのまま動かず、軽量な実装が求められた。



2 それまでどうしていた？

それまで Express のようなフレームワークは Node.js 環境を前提にしており、Edge ランタイムでは動作しないか、動いても重かった。

	Express (Node.js 前提)	Hono
動作環境	Node.js 前提	Cloudflare Workers 等マルチランタイム対応
起動の軽さ	比較的重い	超軽量・高速起動
型の扱い	JavaScript が中心	TypeScript ファースト
向いている場面	通常のサーバー	Edge・サーバーレス環境

3 Hono で何ができるようになった？



Edge 環境でも高速に動く



起動が軽くレスポンスが速い



複数のランタイムで動かせる

関連キーワード

比較するフレームワーク



Express

Node.js 向けの定番の軽量フレームワーク。

P.178

Django

ひとことという

認証・管理画面・ORMなど、Web アプリに必要な機能を標準で備えたPython製のフルスタックフレームワーク。

1 なぜ生まれた？

PythonでWeb アプリを作る際、認証・管理画面・DB接続を毎回個別実装する負担が大きかった。



2 それまでどうしていた？

それまでのPython製Web フレームワークは、ルーティングなど最小限の機能しか持たず、認証や管理画面は別途ライブラリを探して組み合わせる必要があった。

	軽量フレームワーク (Flask など)	Django
付属機能	最小限 (必要なら自分で追加)	認証・管理画面・ORM等標準搭載
管理画面	自作が必要	自動生成される (Django Admin)
構成の自由度	高い	やや決まった型に沿う
向いている場面	小規模・独自構成のAPI	機能が多い業務系Web アプリ

3 Django で何ができるようになった？



必要な機能が最初から揃っている



管理画面が自動で手に入る



大規模なWeb アプリを素早く構築できる

関連キーワード

関連する技術



Python

シンプルな文法を持つプログラミング言語。

P.172

比較するフレームワーク



FastAPI

型ヒントを活かした高速なAPIフレームワーク。

P.181



Flask

必要な機能だけを足していく軽量フレームワーク。

P.182



Rails

Ruby版のフルスタックフレームワーク。

P.177

FastAPI

ひとことという

Pythonの型ヒントを使って、高速なAPIサーバーとドキュメントを同時に生成できるフレームワーク。

1 なぜ生まれた？

既存のPython製フレームワークはAPIドキュメントを手動で書くか、別ツールで管理する必要があり、コードと仕様書がずれがちだった。型ヒントから自動でAPI仕様書（OpenAPI）を生成できるフレームワークとして開発された。



2 それまでどうしていた？

それまでのPython製フレームワークでは、型チェックやAPI仕様書は別のライブラリを組み合わせる必要があった。

	Flaskなど（型・仕様書は別途）	FastAPI
型ヒント	任意（チェックは別ツール）	必須に近く、そのまま検証に使う
APIドキュメント	手動で作成・管理	型ヒントから自動生成
実行速度	標準的	非同期処理で高速
向いている場面	幅広いWebアプリ	API・マイクロサービス

3 FastAPI で何ができるようになった？



非同期処理で高速に動く



API仕様書が自動で手に入る



型ヒントでバグを早期発見できる

関連キーワード

関連する技術



Python

シンプルな文法を持つプログラミング言語。

P.172



Django

電池付属の思想を持つフルスタックフレームワーク。

P.180



Flask

必要な機能だけを足していく軽量フレームワーク。

P.182

Flask

ひとことという

最小限の機能だけを持ち、必要なライブラリを自分で組み合わせて育てていく Python 製の軽量フレームワーク。

1 なぜ生まれた？

Django のようなフルスタックのフレームワークは機能が多く、小さなアプリやプロトタイプには。



2 それまでどうしていた？

それまでのフルスタックフレームワーク（Django など）は、小さなアプリを作る場合でも多機能な構成に付き合う必要があった。

	Django（フルスタック）	Flask
付属機能	認証・ORM・管理画面が標準搭載	最小限（ルーティング程度）
構成の自由度	やや決まった型に沿う	高い（必要な分だけ足す）
学習コスト	やや高い（覚える機能が多い）	低い（小さく始められる）
向いている場面	機能が多い業務系 Web アプリ	小規模 API・プロトタイプ

3 Flask で何ができるようになった？



最小構成からすぐに書き始められる



必要な機能だけ拡張できる



小規模〜中規模に向いている

関連キーワード

関連する技術



Python

シンプルな文法を持つプログラミング言語。

P.172

比較するフレームワーク



Django

電池付属の思想を持つフルスタックフレームワーク。

P.180



FastAPI

型ヒントを活かした高速な API フレームワーク。

P.181

Gin

ひとことについて

Go 言語の標準ライブラリだけでは冗長になりがちなルーティング処理を、シンプルな記法で扱えるようにした Web フレームワーク。

1 なぜ生まれた？

Go 標準の net/http だけでは、ルーティングやミドルウェアの記述が冗長になりがちだった。



2 それまでどうしていた？

それまでは Go の標準ライブラリだけでルーティングを書き、パラメータの取得なども自前で処理する必要があった。

	net/http (標準ライブラリ)	Gin
ルーティング	自前で条件分岐を実装	router.GET()等で簡潔に記述
ミドルウェア	仕組みを自作	標準で組み込み・追加が容易
実行速度	速い	標準ライブラリに迫る速さを維持
向いている場面	極小の API	REST API・マイクロサービス

3 Gin で何ができるようになった？



Go 標準に迫る速度を維持できる



シンプルな記法で API を作れる



ミドルウェアを手軽に追加できる

関連キーワード

関連する技術



Go

並行処理を得意とするシンプルな言語。

P.173

Laravel

ひとことという

認証・ルーティング・ORMなどを整った形で備え、PHPでのWebアプリ開発を一気にモダンにしたフレームワーク。

1 なぜ生まれた？

PHPには多くのフレームワークが乱立していたが、設計が古かったり機能が断片的だったりして。



2 それまでどうしていた？

それまでのPHPフレームワークは機能がバラバラで、認証やルーティングを組み合わせているだけでも設定に手間がかかった。

	従来のPHPフレームワーク	Laravel
機能の揃い方	断片的・組み合わせが必要	認証・ORM・キュー等統合済み
開発体験	設定や記法が古い	モダンな記法・豊富なドキュメント
DB操作	SQLを都度記述	Eloquent ORMで直感的に操作
向いている場面	小規模なWebページ	本格的なWebアプリ・SaaS

3 Laravel で何ができるようになった？



モダンな作法でWebアプリを構築できる



認証・ORMなどが標準で揃っている



豊富なパッケージで拡張できる

関連キーワード

関連する技術



PHP

HTMLに埋め込んで使えるサーバーサイド言語。

P.174

比較するフレームワーク



Rails

Ruby版のフルスタックフレームワーク。

P.177

Spring Boot

ひとことという

煩雑だった Spring フレームワークの設定を自動化し、Java で素早くアプリケーションを起動できるようにしたフレームワーク。

1 なぜ生まれた？

Spring ベースの Java 開発では、初期設定と依存構成の準備負担が大きかった。



2 それまでどうしていた？

それまでの Spring Framework では、DB やサーバーの接続設定を XML など
で細かく記述し、動かすまでに多くの準備が必要だった。

	従来の Spring Framework	Spring Boot
設定方法	XML 等で細かく記述	規約に沿えば自動設定 (auto-configuration)
起動までの準備	多い (サーバー構築も別途)	組み込みサーバーで即座に起動
依存関係管理	個別に指定・バージョン調整	starter でまとめて管理
向いている場面	既存の大規模 Spring アプリ	新規の Java アプリ・マイクロサービス

3 Spring Boot で何ができるようになった？



すぐに動くアプリ
を構築できる



設定の大部分を自
動化できる



エンタープライズ
規模でも安定動作

関連キーワード

関連する技術



Java

JVM 上で動作し、OS を問わず動かせる言語。

P.175

ド ッ カ ー
Docker

ひとことというと

アプリとその実行に必要なものを「箱」に入れて、どこでも同じように動かす仕組み。

1 なぜ生まれた？

環境（OS・ライブラリ・設定など）の違いで、「ローカルでは動くのに、本番では動かない」が頻発していた。



2 それまでどうしていた？

仮想マシン（VM）を使って、OSごと分離していた。

	VM（仮想マシン）	Docker
分離の単位	OSごと	アプリ＋必要なものだけ
重さ・起動	重い・遅い	軽い・速い
再現性・配布	環境の用意が大変	同じ箱を渡せる
向いている場面	OSごと分離したいとき	開発・テスト・配布

3 Docker で何ができるようになった？



すぐに起動できる



同じ環境を共有できる



配布・デプロイが簡単

関連キーワード

関連キーワード



VM（仮想マシン）

OSごと分離する従来型。Dockerと比較の定番。

P.401



Docker Compose

複数コンテナをまとめて定義・実行。

P.321



Kubernetes

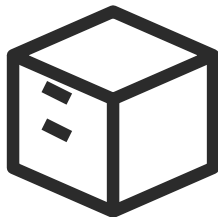
コンテナを大規模に自動運用する仕組み。

P.441

キーワードメモ

箱に入れるものは、アプリや設定、ライブラリなど。中身は自由に作れる。

イ メ ー ジ
Image



インフラ
コンテナ

ひとことというと

アプリと必要なものを1枚に固めた、コンテナの
もとになる読み取り専用のテンプレート。

1

なぜ生まれた？

同じ環境を何度でも・どこでも再現するために、
状態を1つに固めておきたいから。



2

それまでどうしていた？

手順書を見ながら、サーバーへ必要なものを手動でインストールしてい
た。

	手動構築	イメージ
環境の再現	手順書頼み	イメージで同じ
共有方法	手順書を渡す	イメージごと渡す
個人差	人によって違う	必ず同じになる
管理	バラバラ	レジストリで管理

3

イメージで何ができる？



層を重ねて作る



どこでも同じを再
現



レジストリで共有

関連キーワード

一緒に使う技術



Container

イメージから生ま
れた、動くアプリ
の箱。

P.309



Docker

イメージを作り・
動かす代表的なツ
ール。

P.301

次に読むなら



Docker
Compose

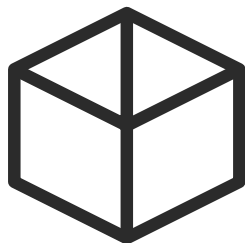
複数のイメージを
まとめて動かす設
定。

P.321

キーワードメモ

イメージは層（レイヤ
ー）の積み重ね。差分
だけ足すので軽い。

Container



インフラ
コンテナ

ひとことでいうと

イメージを実行して動き出した、アプリ 1 つ分の
隔離された実行環境。

1 なぜ生まれた？

アプリを周りから隔離して、
どこでも同じように・軽く動かしたかったから。



2 それまでどうしていた？

VM（仮想マシン）を立てて、OS ごと分離していた。

	VM（仮想マシン）	コンテナ
分離の単位	OS ごと	アプリ + 依存
重さ	重い	軽い
起動	分単位	秒単位
使い捨て	しにくい	しやすい

3 コンテナで何ができる？



すぐ起動・破棄で
きる



同じイメージから
量産



つないでシステム
に

関連キーワード

比較する技術



VM（仮想マ
シン）

OS ごと分離する、
重いが強い仮想
化。

P.401

一緒に使う技術



Image

コンテナのもとに
なる設計図。

P.305



Docker

コンテナを作り・
動かす代表的なツ
ール。

P.301

次に読むなら



Kubernetes

大量のコンテナを
自動で運用する仕
組み。

P.441

キーワードメモ

1 コンテナ = 1 役割が基
本。増やして並べて、
システムを組む。

Volume



インフラ
データ

ひとことでいうと

コンテナが削除されてもデータが消えないよう、
ホスト側の保存場所をコンテナにつなぐ仕組み。

1

なぜ生まれた？

コンテナは「使い捨て」が前提なので、
中にデータを書くとコンテナごと消えてしまう
から。



2

それまでどうしていた？

コンテナ内のファイルシステムに直接データを書き込んでいた。

	コンテナ内保存	Volume
コンテナ削除後	消える	残る
複数コンテナ共有	できない	できる
バックアップ	手動・難しい	ホスト側で管理
DB との相性	悪い	良い

3

Volume で何ができる？



データを永続化



コンテナ間で共有



バックアップが楽

関連キーワード

一緒に使う技術



Container

Volume をマウン
トして使うコンテ
ナ本体。

P.309



Docker Compose

volumes: でファ
イルに宣言でき
る。

P.321

次に読むなら



Docker

コンテナを作り・
動かす土台のツ
ール。

P.301

キーワードメモ

docker volume
create または
compose の volumes:
で作成。名前付きポリ
ュームがおすすめ。

317 Network

ひとことというと

Docker上でコンテナ同士や外の世界をつなぐ、仮想的なネットワークの単位。

1 なぜ生まれた？

コンテナを起動しても、名前解決や通信経路がバラバラだとアプリ同士が繋がらない。



2 それまでどうしていた？

ネットワークを意識する前は、ホストのIPやポート番号を直書きしてコンテナ間をつないでいた。

	ホスト依存のつなぎ方	Docker Network
名前解決	IPやポートを覚える	コンテナ名で呼び合える
分離	全部が同じ平面に見えがち	網ごとに通信範囲分けられる
Composeとの相性	手動で橋渡し	networks: で宣言すればつながる
外への公開	全部ホスト経由になりやすい	必要なポートだけ公開できる

3 Network で何ができるようになった？



コンテナ同士を名前ですつなげる



環境ごとに通信範囲を分離できる



Composeのnetworks: で宣言できる

関連キーワード

関連キーワード



Container

実際に動いているプロセスの箱。

P.309



Docker Compose

複数コンテナとネットワークをまとめて定義。

P.321



Volume

データをコンテナの外に残す仕組み。

P.313

コンポーザー Compose



インフラ
ツール

ひとことでいうと

何個ものコンテナの構成を 1 つのファイルに書いて、まとめて起動・停止できるツール。

1 なぜ生まれた？

Web・DB・キャッシュなど複数のコンテナを、毎回手で起動するのが大変だったから。



2 それまでどうしていた？

docker run でコンテナを 1 個ずつ手動で起動していた。

	docker run	Compose
対象	1 コンテナずつ	まとめて一括
設定	コマンドに毎回	ファイルに固定
再現性	手順に依存	誰でも同じ
起動	何度もコマンド	up 一発

3 Compose で何ができる？



構成をファイル化



up で一括起動



依存関係を自動解決

関連キーワード

一緒に使う技術



Docker

コンテナを作り・動かす土台のツール。

P.301



Container

Compose がまとめて動かす対象。

P.309

次に読むなら



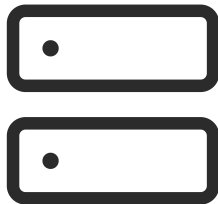
Kubernetes

大規模・本番向けのコンテナ運用基盤。

P.441

キーワードメモ

書くのは
compose.yaml。誰が動かしても同じ環境が立ち上がる。



インフラ
仮想化

ひとことでいうと

物理マシンの上に、OS ごと丸ごとの“もう一台”を立ち上げて動かす仕組み。

1

なぜ生まれた？

高性能なサーバを 1 つの用途で遊ばせず、独立した複数の環境に分けて使いたかったから。



2

それまでどうしていた？

物理サーバーを 1 台に 1 用途で購入・設置していた。

	物理サーバー	VM (仮想マシン)
1 台の使い方	1 用途のみ	複数に分割できる
コスト	用途の数だけ購入	1 台で済む
柔軟性	増やすには購入	すぐ追加できる
分離	物理的に完全に別	OS レベルで分離

3

VM で何ができる？



別 OS をそのまま動かせる



環境を丸ごと保存できる



スナップショットで戻せる

関連キーワード

関連キーワード



Docker

アプリを箱に入れて軽く動かす技術。

P.301



Container

VMより軽い、アプリ単位の実行環境。

P.309



Kubernetes

大量のコンテナを自動運用する仕組み。

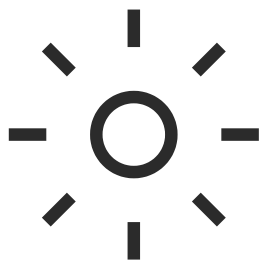
P.441

キーワードメモ

仮想化は「1 台を分ける」発想。分け方の細かさで VM とコンテナに分かれる。

ク バ ネ テ イ ス

Kubernetes



インフラ
運用基盤

ひとことでいうと

たくさんのコンテナを、複数のサーバ上で自動で
配置・維持・回復してくれる仕組み。

1 なぜ生まれた？

コンテナが増えすぎて、
人手で配置・監視・再起動しきれなくなったから。



2 それまでどうしていた？

Compose や手動スクリプトで、1 台ずつ管理・監視していた。

	Docker Compose	Kubernetes
規模	1 台想定	多数のサーバ
障害時	手で対応	自動で復旧
スケール	手動	自動で増減
用途	開発中心	本番運用

3 Kubernetes で何ができる？



複数サーバに自動
配置



落ちても自動で復
旧



負荷に応じて増減

関連キーワード

関連キーワード



Docker Compose

1台で複数コンテナをまとめて動かす。

P.321



Container

Kubernetesが配置・管理する最小単位。

P.309



Docker

コンテナ・イメージを作る土台。

P.301

キーワードメモ

「宣言的」に理想状態を書く。ズレたら自動で直していく。

サーバレス アーキテク チャ

ひとことという

サーバーの用意や運用をクラウド側に任せ、イベントが来たときだけ処理を動かすアーキテクチャの考え方。

1 なぜ生まれた？

常時サーバーを立てておくと、アイドル時間も料金と運用コストがかかる。



2 それまでどうしていた？

EC2やVMのように、OSごとサーバーを借りて常時稼働させる構成が一般的だった。

	常時稼働サーバー	サーバレスアーキテクチャ
サーバー管理	OS・パッチ・容量を自分で面倒見	クラウドが面倒見
課金	起動時間分を払う	実行時間・リクエスト数に応じて
スケール	手動設定が必要	イベント量に応じて自動
向いている用途	常時接続が必要	断続的・突発的な処理

3 サーバレスアーキテクチャで何ができるようになった？



APIやバッチをイベント駆動で動かせる



アイドル時間の運用コストを抑えられる



小さく始めて需要に合わせて広げやすい

関連キーワード

比較する構成



VM

OSごと借りて常時動かす従来型。

P.401



Kubernetes

コンテナを常時管理する別アプローチ。

P.441

一緒に覚えたい言葉



FaaS

Function as a Service。関数単位で動かすサーバレスの形。



Lambda

AWSの代表的なサーバレス実行環境（Ch18）。

キーワードメモ

常時接続（WebSocket等）には向かないことも。サーバレス＝サーバー不要、ではない。

ひとことというと

iPhone や iPad 向けのオペレーティングシステム。
App Store 経由での配布が基本。

1 なぜ生まれた？

スマホが本格的なコンピュータになるにつれ、
タッチ操作向けの専用 OS とアプリ配布基盤が
必要になった。



2 それまでどうしていた？

フィーチャーフォン時代はメーカー独自の環境や、限られた Java アプリで
動いていた。

	ガラケー / 独自環境	iOS
UI の前提	物理キー中心	マルチタッチ前提
アプリ配布	メーカーやキャリア経 由が多い	App Store が中心
開発言語	機種・メーカー依存	Swift / Objective-C が 主
ハードウェア	機種差が大きい	Apple 製デバイスに閉 じる

3 iOS で何ができるようになった？



iPhone 向けアプ
リを配布できる



審査付きストアで
品質下限を揃えや
すい



端末×OS の組み合
わせが少ない

関連キーワード

関連キーワード



Swift

iOS アプリ開発の
第一言語。

P.457



ネイティブア
プリ

OS の正式 API で作
るアプリ。

P.452



Android

対になるもう一方
の巨大プラットフ
ォーム。

P.451

Android

ひとことという

Google が中心となって育てる、オープン寄りのスマートフォン向け OS。多様なメーカーの端末で動く。

1 なぜ生まれた？

スマホ市場が広がる中、特定メーカーに閉じないオープンなプラットフォームと、豊富な端末ラインナップが求められた。



2 それまでどうしていた？

メーカーごとの独自 OS や、PC 向けソフトの縮小版で「なんとなく動く携帯」を作っていた。

	メーカー独自 OS 時代	Android
端末の多様性	メーカーごとに世界が違う	多くのメーカーが同じ OS を採用
アプリストア	ばらばら	Google Play が中心（ほかも可）
カスタマイズ	ユーザーには届きにくい	メーカーやユーザーが手を入れやすい
開発言語	機種依存	Kotlin / Java が主

3 Android で何ができるようになった？



多メーカー端末向けに配布できる



Google Play 中心の配布基盤がある



端末差を前提にテスト設計できる

関連キーワード

関連キーワード



Kotlin

Android公式が推す第一言語。

P.458



iOS

もう一方の巨大プラットフォーム。

P.450



Flutter

Android/iOSをまとめて書く選択肢。

P.455

ネイティブアプリ

ひとことというと

iOSやAndroidが用意した公式の言語・API・UI部品で作る、プラットフォーム直結のアプリ。

1

なぜ生まれた？

スマホのカメラや通知、ジェスチャを最大限使いたかったが、Webページだけでは端末の能力を引き出しきれなかった。



2

それまでどうしていた？

スマホ向けも、まずはブラウザで動くWebサイトや、簡易ランタイム上のアプリで賄おうとしていた。

	Webや簡易ランタイム	ネイティブアプリ
性能	ブラウザ経由で一段遠い	OSに近く高速に動かしやすい
端末機能	制限されがち	公式APIで深く触れる
UIの馴染め	Webっぽさが残ることも	OS標準の見た目・操作感
配布	URLを開いてもらう	ストア経由が基本

3

ネイティブアプリで何ができるようになった？



端末の性能を引き出しやすい



OS標準のUI/UXに寄せられる



ストア配布・更新の流れに乗る

関連キーワード

関連キーワード



ハイブリッドアプリ

Web技術を包んでアプリにする方式。

P.453



React Native

JSでネイティブUIを描く枠組み。

P.454



Swift

iOSネイティブの主力言語。

P.457

ハイブリッドアプリ

ひとことでいうと

HTML/CSS/JSなどのWeb技術で画面を作り、ネイティブの殻（WebViewなど）で包んでストア配布するアプリ。

1 なぜ生まれた？

ネイティブ機能を使いたい一方で、iOSとAndroidを別実装する負担を減らす必要があった。



2 それまでどうしていた？

完全ネイティブで二台分作るか、モバイルWebだけで我慢するかのも二択になりがちだった。

	ネイティブ二刀流 or モバイルWeb	ハイブリッドアプリ
コード共有	プラットフォームごと	Web部分を大きく共有できる
ストア配布	ネイティブなら可 / Webは不可	アプリとして配布できる
性能・一体感	ネイティブが有利	WebView次第で差が出る
更新	ストア審査が基本	中身のWebはサーバー更新しやすいことも

3 ハイブリッドアプリで何ができるようになった？



Web技術で素早く作れる



コンテンツ更新をサーバー側に寄せられる



ネイティブ機能はブリッジで足せる

関連キーワード

関連キーワード



ネイティブアプリ

OS公式APIで作る本流。

P.452



React Native

WebViewではなくネイティブ部品を使う道。

P.454



PWA

インストール感をWebのまま近づける別路線。

P.158

React Native

ひとことでいうと

Reactのコンポーネント発想で画面を書き、実際の描画はiOS/Androidのネイティブ部品に任せるフレームワーク。

1 なぜ生まれた？

Reactの開発体験を活かしつつ、ネイティブUIを共通コードで扱う必要があった。



2 それまでどうしていた？

選択肢は「SwiftとKotlinで二重開発」か「ハイブリッド（WebView）」に偏りがちだった。

	二重ネイティブ/ WebViewハイブリッド	React Native
言語・発想	SwiftとKotlinで別々	JS/TS + Reactで共通化
UIの実体	WebView or 完全別実装	ネイティブコンポーネント
ホットリロード	環境次第	開発中の再読み込みが速い
エコシステム	各OSの流儀	npmとReactの資産を活かせる

3 React Native で何ができるようになった？



Reactの発想をモバイルへ展開できる



JS/TSから両OS向けに寄せられる



足りない部分はネイティブで補える

関連キーワード

関連キーワード



Flutter

Dartで自前描画するライバル枠。

P.455



React

発想の原点であるWebのUIライブラリ。

P.146



ネイティブアプリ

最終的に触っている先の世界。

P.452

455

Flutter

ひとことというと

Dart 言語と独自の描画エンジンで、iOS/Android（ほか）に同じ見た目の UI を描く UI キット。

1 なぜ生まれた？

プラットフォームごとの UI 部品差を吸収しつつ、デザインどおりのピクセルを両 OS で再現したかった。



2 それまでどうしていた？

ネイティブ二刀流か、React Native のように OS の部品に橋を架ける方式が主流だった。

	OS 部品に頼る開発	Flutter
描画	各 OS の UI 部品	自前エンジンでキャンバスに描く
見た目の一致	OS 差が出やすい	指定どおりに揃えやすい
言語	Swift / Kotlin / JS 等	Dart が中心
ホットリロード	ツール次第	ステートを保ったまま高速に反映

3 Flutter で何ができるようになった？



デザインの再現性を両 OS で高められる



滑らかなアニメーションを出しやすい



一つのコードベースでマルチ向けに展開しやすい

関連キーワード

関連キーワード



Dart

Flutter 公式の言語。

P.456



React Native

JS でネイティブ UI を駆動する別路線。

P.454



Android

配布先の一大プラットフォーム。

P.451

Dart

ひとことという

Googleが開発したプログラミング言語。いまはFlutterアプリを書くための主役として知られる。

1 なぜ生まれた？

大規模なクライアントアプリ向けに、習得しやすく、ツールチェーンも一体で使える言語が欲しかった。



2 それまでどうしていた？

モバイルではSwift / Kotlin、クロスではJavaScriptが候補の中心で、Dartは「知る人ぞ知る」側だった。

	Flutter 以前の印象	Flutter と組んだ Dart
主な用途	一部のWeb・実験的用途	Flutter アプリ開発が主流
型	オプション寄りに見えがち	健全な型づけで大規模向き
非同期	Future 等独自の流儀	async/await で読みやすい
学習動機	弱い	Flutter をやるなら必須級

3 Dart で何ができるようになった？



FlutterでUIを書く言語になる



型付きでUIロジックを整理できる



Hot Reloadで試行錯誤が速い

関連キーワード

関連キーワード



Flutter

Dartの最大の活躍の場。

P.455



Swift

iOSネイティブ側の言語。

P.457



Kotlin

Androidネイティブ側の言語。

P.458

Swift

ひとことというと

Appleが開発したプログラミング言語。
iOS/macOSなどのアプリ開発でObjective-Cに代
わって第一言語になった。

1 なぜ生まれた？

Objective-Cは冗長で、null事故も起きやすかつ
た。もっと安全に書きたいニーズが強まった。



2 それまでどうしていた？

Objective-CのみでiOS開発していた。

	Objective-C	Swift
読みやすさ	独特で学習コスト高	モダンで読みやすい
安全性	ポインタやnilに注意 が必要	オプション等で事故 を減らせる
相互運用	—	既存のObjective-C資 産とも共存可
進化	相対的に緩やか	言語もツールも活発に 更新

3 Swift で何ができるようになった？



iOSネイティブア
プリを現代的に書
ける



型とオプション
で実行時事故を減
らせる



SwiftUIなど新し
いUI枠組みに乗
れる

関連キーワード

関連キーワード



iOS

Swiftが最も活躍す
るOS。

P.450



Kotlin

Android側の対に
なる言語。

P.458



ネイティブア プリ

Swiftで作る典型的
な成果物。

P.452

ひとことというと

JetBrains が作った言語で、Android アプリ開発の第一言語として Google も推奨。Java と仲が良い。

1 なぜ生まれた？

Javaは安定しているが冗長で、null 事故も起きやすかった。



2 それまでどうしていた？

Android は長らく Java が標準。ボイラープレートと null に泣きながら書いていた。

	Java 中心の Android	Kotlin
記述量	冗長になりやすい	簡潔に書ける
null 安全	実行時に気づきがち	型でかなり防げる
Java との関係	—	相互運用が得意
公式の立場	伝統的標準	Android の推奨言語

3 Kotlin で何ができるようになった？



Android アプリを短く安全に書ける



既存 Java コードと混在できる



コルーチンで非同期を扱いやすい

関連キーワード

関連キーワード



Android

Kotlin が本領を発揮する OS。

P.451



Swift

iOS 側の対になる言語。

P.457



Java

互換と歴史のパートナー。

P.175

Unit Test

ひとことというと

関数やクラスなど、プログラムの最小単位が期待どおり動くかを検証するテスト。

1 なぜ生まれた？

関数やクラスが大きくなると、変更のたびに手動確認が追いつかなくなった。



2 それまでどうしていた？

動作確認は、アプリを起動して人の手と目で見るのが基本だった。

	手動の動作確認	Unit Test
対象	画面や一連の操作	関数・クラスなど小さい単位
速さ	遅い	秒単位で大量に回せる
原因の切り分け	どこで壊れたか追にくい	失敗した単位がすぐ分かる
外部依存	本物のDBやAPIに寄りがち	モックして切り離しやすい

3 Unit Test で何ができるようになった？



小さな変更をすぐ検証できる



壊れた場所をピンポイントで見つけられる



リグレッションを自動で防ぎやすくなる

関連キーワード

関連キーワード



Integration Test

部品をつないだ状態で確かめる。

P.501



Mock

依存を偽物に差し替える道具。

P.503



Jest

JS界隈でよく使うテストランナー。

P.518

501 Integration Test

ひとことという

複数のモジュールや、DB・APIなどの外部をつないだ状態で、連携が正しいかを確かめるテスト。

1 なぜ生まれた？

単体では正しくても、つなぐと型やトランザクション、設定の食い違いで落ちることがある。



2 それまでどうしていた？

結合確認はステージングで人手、またはユニット + モックだけだった。

	単体だけ / 手動結合	Integration Test
見る範囲	部品の内側	部品の境界と相互作用
依存	モックで消しがち	本物かそれに近いものを使う
速さ	非常に速い	ユニットより遅い
見つかるバグ	ロジック誤り	配線・設定・契約のずれ

3 Integration Test で何ができるようになった？



モジュール間の契約ずれを拾える



DBやAPIとの実連携を検証できる



E2Eより狭く原因を追しやすい

関連キーワード

関連キーワード



Unit Test

もっと小さい単位のテスト。

P.500



E2E Test

ユーザー操作に近い端-to-端のテスト。

P.502



Fixture

結合時に使う固定の試験データ。

P.505

E2E Test

ひとことというと

画面操作やAPI呼び出しなど、システムを端から端まで通して、利用シナリオが成立するかを確かめるテスト。

1 なぜ生まれた？

部品も結合も正しくても、「会員登録してログインして購入する」が通るかは別問題。



2 それまでどうしていた？

リリース前に人がブラウザを手で操作し、チェックリストを消化するのが普通だった。

	手動確認中心	E2E Test
見る範囲	画面の一部	ユーザー操作の一連の流れ
再現性	担当者の記憶に依存	スクリプトで繰り返せる
コスト	人時がかかる	初期構築後は繰り返し安い
弱点	見落とし	UI変更でテストも折れやすい

3 E2E Test で何ができるようになった？



重要シナリオの退行を自動検知できる



夜中のCIでも同じ確認を回せる



リリース判断の材料を機械的に増やせる

関連キーワード

関連キーワード



Playwright

モダンなE2E自動化ツール。

P.520



Puppeteer

Chromeを操作する自動化ライブラリ。

P.521



スモークテスト

「とりあえず起動するか」の薄い確認。

P.512

Mock

ひとことというと

テスト中に本物の依存（APIやDBなど）の代わりに置き、戻り値や「呼ばれたか」を制御・検証する偽物。

1 なぜ生まれた？

外部サービスが落ちていたり、課金APIを本当に叩いたりすると、テストが不安定で高くつく。



2 それまでどうしていた？

テストでも本番と同じDBや外部APIに繋ぎ、環境ごとしんどい思いをしていた。

	本物依存のままテスト	Mockを使う
安定性	ネットや相手次第	ローカルで決定的に動かせる
検証できること	結果中心	「どう呼ばれたか」も見られる
速さ	I/O待ちが発生	メモリ上で一瞬
危険	課金やメール誤送信	副作用を止められる

3 Mock で何ができるようになった？



外部依存を止めて
単体テストできる



呼び出し回数や引
数を検証できる



テストを速く安定
させられる

関連キーワード

関連キーワード



Stub

決まった値を返すことに寄せた代役。

P.504



Unit Test

Mockの出番が多い土俵。

P.500



Fixture

入力データ側の固定化。

P.505

ひとことというと

テストのために、依存先の代わりとして決め打ちの戻り値や状態だけを返す簡易実装。

1 なぜ生まれた？

ロジックのテストに必要なのは「今は成功扱いのユーザーが返る」など結果だけで、本物の複雑さや呼び出し検証までは不要なことが多かった。



2 それまでどうしていた？

毎回フルのモックフレームワークで厳格に振る舞いまで検証するか、本物に繋ぐか、になりがちだった。

	本物 or 厳格Mock	Stub
目的	結合や相互作用の検証	決め打ちの入力・応答を渡す
複雑さ	高くなりやすい	薄く保てる
検証	呼ばれ方まで見ることが多い	戻り値の用意が主
読みやすさ	セットアップが長い	意図が短く書ける

3 Stub で何ができるようになった？



テストに必要な応答だけを用意できる



本番実装の重さをテストから外せる



セットアップを短くできる

関連キーワード

関連キーワード



Mock

呼び出しまで検証する寄り。

P.503



Fixture

データ側の固定セット。

P.505



Unit Test

Stubがよく出る場所。

P.500

Fixture

ひとことというと

テストを走らせるためにあらかじめ用意する、固定のデータや状態・環境のこと。

1 なぜ生まれた？

毎回手でユーザーを作ったりDBを初期化したりしていると、テストの本題より準備が長くなる。



2 それまでどうしていた？

テスト関数の中に、データのINSERTやファイル配置をベタ書きしていた。

	テスト内に準備を直書き	Fixture
再利用	コピペが増える	共通の前提を使い回せる
読みやすさ	本題が埋もれる	「何を試すか」が前面に出る
一貫性	テストごとに微妙に違う	同じ初期状態から始められる
メンテ	変更が散らばる	一箇所に寄せやすい

3 Fixture で何ができるようになった？



試験データをまとめて管理できる



テストの準備コードを短くできる



前提条件のばらつきを減らせる

関連キーワード

関連キーワード



Seeder

アプリ側の初期データ投入に近い概念。

P.93



Integration Test

Fixtureの出番が多い層。

P.501



Factory

動的にテストデータを作る派（一緒に覚えたい）。

ひとことという

Test-Driven Development。失敗するテストを先に書き、通る最小実装→リファクタ、を回す開発手法。

1 なぜ生まれた？

作り終わってからテストを足すと、「今の実装に合わせたテスト」になりやすい。



2 それまでどうしていた？

実装→手動確認→余裕があればテスト、の順番がデフォルトだった。

	実装ファースト	TDD
順番	コード→テスト	テスト→コード→リファクタ
設計への影響	後付けになりがち	使いやすさが先に決まる
フィードバック	動かし始めてから	数分単位の短いループ
過剰実装	つい作り込みがち	赤→緑の範囲に収まりやすい

3 TDD で何ができるようになった？



小さなフィードバックループで進める



過剰な設計を抑えやすい



仕様の意図がテストとして残る

関連キーワード

関連キーワード



BDD

振る舞い・事例の言葉で仕様を書く寄り。

P.507



Unit Test

TDDで最もよく回す粒度。

P.500



リファクタリング

緑のあとに必ず来る工程（設計章）。

BDD

ひとことというと

Behavior-Driven Development。振る舞いを自然言語に近い形で書き、開発・QA・企画の認識を揃えるアプローチ。

1 なぜ生まれた？

テストはエンジニアの内側で閉じてしまい、ビジネス側の「そう動いてほしい」とずれやすかった。



2 それまでどうしていた？

仕様書は別文書、テストはコード、口頭の認識合わせは会議、と情報が分裂していた。

	仕様とテストが別物	BDD
書き方	実装寄りの assert	Given/When/Then 等 振る舞い
読み手	エンジニア中心	非エンジニアも参加し やすい
目的	正しさの検証	認識合わせ + 検証
成果物	テストコード	実行可能な仕様に近づく

3 BDD で何ができるようになった？



仕様をテスト可能な文章に落とせる



職種をまたいで認識を揃えやすい



受け入れ条件を自動化しやすくなる

関連キーワード

関連キーワード



TDD

テスト先行のサイクル。BDDと併記されやすい。

P.506



E2E Test

振る舞いシナリオの実行場所になりやすい。

P.502



テスト設計

何を書くかを決める上流。

P.517

カバレッジ

ひとことというと

テスト実行によって実行されたコードの割合。
行・分岐などの単位で測ることが多い。

1 なぜ生まれた？

「テスト書いた」と言っても、通っていない分岐だらけかもしれない。



2 それまでどうしていた？

テストの十分さは、感覚とコードレビューのコメント頼みだった。

	感覚での十分さ	カバレッジ
見え方	書いた気になる	通った行・分岐が見える
目標管理	曖昧	数値目標を置ける（功罪あり）
弱点発見	属人的	未カバー箇所が地図になる
誤解	—	100% = バグゼロではない

3 カバレッジ で何ができるようになった？



未テスト箇所を可視化できる



PR でカバー率の変化を追える



重要な経路の穴を見つけやすくなる

関連キーワード

関連キーワード



Unit Test

カバレッジ計測の主戦場。

P.500



ホワイトボックステスト

内部構造を見て網羅を考える。

P.515



Jest

カバレッジ計測がしやすいランナー例。

P.518

デバッグ

ひとことというと

プログラムの不具合原因を特定し、修正して確認するまでの一連の活動。

1 なぜ生まれた？

動くはずのプログラムが期待どおり動かないとき、勘で直すと別の場所が壊れる。



2 それまでどうしていた？

printデバッグと気合、そして「一度消して書き直す」が頼りだった。

	勘と再実装	意図的なデバッグ
再現	運任せ	手順を固定して何度も見る
観測	ログを散らかす	ブレークポイントや段階的検証
仮説	思いつき修正	仮説を立ててから変える
回帰防止	その場しのぎ	テストに残せる

3 デバッグで何ができるようになった？



原因を切り分けて
確実に潰せる



デバッガで実行時の
状態を覗ける



再発防止のテスト
やログ改善につな
げられる

関連キーワード

関連キーワード



ログ

本番でデバッグするときの手がかり（観測章へ）。



回帰テスト

直した穴が再び開かないか確認。

P.511



Unit Test

仮説検証を高速にする道具。

P.500

スナップショットテスト

ひとことというと

コンポーネントの出力やUIの結果を保存し、次回以降の実行結果と差分比較するテスト手法。

1 なぜ生まれた？

UIの細かい文言やクラス名の変化を、全部手でassertするのは現実的でない。



2 それまでどうしていた？

見目の確認は目視、または重要な要素だけを個別assertしていた。

	目視 / 個別assert	スナップショット
記述量	多い / 抜けやすい	一発で出力全体を固定
変更検知	見落としがある	差分として必ず出る
意図	何を守るか明示しやすい	更新時にレビューが必要
脆さ	—	リファクタでも折れやすい

3 スナップショットテストで何ができるようになった？



UIの意図しない変化を検知できる



細かいassertを書く時間を節約できる



差分レビューで変更意図を確認できる

関連キーワード

関連キーワード



Jest

toMatchSnapshotが有名。

P.518



Vitest

同様の機能を持つ高速ランナー。

P.519



回帰テスト

変化検知という目的に近い。

P.511

回帰テスト

ひとことというと

変更後に、以前できていたことが壊れていないかを確認するテスト（またはその活動）。

1 なぜ生まれた？

機能追加のたびに、関係ない画面が壊れることがあった。



2 それまでどうしていた？

リリース前に、思い出した範囲を手手でざっと触る——が回帰確認の実体だった。

	思い出し手動確認	回帰テスト
範囲	担当者の記憶頼み	守るシナリオを資産化
タイミング	リリース直前に集中	変更のたびに自動で実行可
抜け	起きやすい	完全ではないが再現性がある
コスト	人の時間が毎回必要	初期投資 + 実行コスト

3 回帰テスト で何ができるようになった？



過去の不具合の再発を防ぎやすい



変更のたびに安全網をかけられる



リファクタの勇気を増やせる

関連キーワード

関連キーワード



スモークテスト

もっと薄い「死んでないか」確認。

P.512



E2E Test

回帰の実行形態のひとつ。

P.502



デバッグ

回帰で落ちたあと必ず来る工程。

P.509

スモークテスト

ひとことというと

ビルドやデブroy直後に、起動・主要導線など最小限が動くかを短時間で確認するテスト。

1 なぜ生まれた？

重いフルテストを回す前に、そもそも起動しない・ログインできない、を早く知りたかった。



2 それまでどうしていた？

デブroyしたら、とりあえずトップページをブラウザで開いてみる——が儀式だった。

	人手の「開けてみた」	スモークテスト
深さ	その場の気分	事前に決めた最小セット
速さ	人待ち	短時間で機械実行
目的	雰囲気確認	致命傷の早期発見
位置づけ	非公式	パイプラインの門番にできる

3 スモークテストで何ができるようになった？



壊れたデブroyをすぐ止められる



重いテストの前に足切りできる



主要導線の生存を確認できる

関連キーワード

関連キーワード



E2E Test

もっと厚いシナリオ確認。

P.502



回帰テスト

広さ・深さのある再確認。

P.511



CI

スモークを自動で回す場所（Ch14）。

境界値分析

ひとことという

仕様の境界（0と1、最大値の前後など）を重点的に試すテスト設計の技法。

1 なぜ生まれた？

条件分岐のバグは、真ん中の普通の値より、境界の前後で顕在化しやすい。



2 それまでどうしていた？

適当な代表値をいくつか入れて「動いた」ことにしていた。

	適当な代表値	境界値分析
選ぶ値	感覚	境界とその前後を意図的に
バグ検出	運が絡む	典型的なオフパイワンを狙い撃ち
ケース数	ばらつく	必要最小に近づけやすい
根拠	説明しにくい	設計として説明できる

3 境界値分析 で何ができるようになった？



オフパイワン系の欠陥を狙いやすい



テストケースに設計根拠を持たせられる



少ないケースで危険地帯をカバーできる

関連キーワード

関連キーワード



同値分割

同じ扱いのグループから代表を取る技法。

P.514



ブラックボックステスト

仕様ベースで境界を見る立場。

P.516



テスト設計

技法を組み合わせる上流工程。

P.517

514 | 同値分割

ひとことという

入力を「同じ結果になるグループ」に分け、各グループから代表値だけを試すテスト設計技法。

1 なぜ生まれた？

取りうる入力を全部試すのは不可能。



2 それまでどうしていた？

思いつく値を並べるか、全数に近い組み合わせを試みて疲弊していた。

	ランダム / 総当たり	同値分割
考え方	とにかく試す	同じ結果の塊に分ける
効率	ケースが膨らむ	代表値で済ませられる
見落とし	運任せ	境界とセットで設計
向いている	小さな入力	入力条件の整理

3 同値分割 で何ができるようになった？



入力空間を整理してテストできる



無駄な重複ケースを減らせる



カバレッジの議論を仕様ベースにできる

関連キーワード

関連キーワード



境界値分析

区画の端を突く姉妹技法。

P.513



テスト設計

これらの技法を使う工程。

P.517



ブラックボックステスト

内部を見ずに仕様から区画する。

P.516

ホワイトボックステスト

ひとことというと

プログラムの内部構造（分岐や経路）を理解したうえで、その経路を通すように設計するテスト。

1 なぜ生まれた？

仕様どおりの入出力だけでは、通っていない分岐やエラーハンドリングが残る。



2 それまでどうしていた？

仕様書の項目を外側から叩くだけで、実装の裏道は見ないことが多かった。

	外側からの確認だけ	ホワイトボックス
視点	仕様・画面	分岐・経路・内部状態
設計者	誰でも可能なことも	コードが読める人向き
強み	—	死にコードや抜け経路を見つけやすい
弱み	—	実装に引きずられ仕様漏れに気づきにくい

3 ホワイトボックステストで何ができるようになった？



分岐網羅など構造ベースの確認ができる



カバレッジと相性が良い



リファクタ前後の経路維持を支えられる

関連キーワード

関連キーワード



ブラックボックステスト

中を見ない対の立場。

P.516



カバレッジ

構造網羅の指標。

P.508



Unit Test

白箱になりやすい粒度。

P.500

ブラックボックステスト

ひとことというと

内部実装を知らなくても（見なくても）仕様上の入力に対する出力や振る舞いを確認するテスト。

1 なぜ生まれた？

実装者以外も品質を確認したかったし、実装に引っ張られたテストでは仕様漏れに気づけない。



2 それまでどうしていた？

テストも開発者の頭の中の実装イメージに合わせて書かれ、仕様書とのズレが見逃されがちだった。

	実装に寄り添う確認	ブラックボックス
必要な知識	コード構造	仕様・要求
見つけやすいもの	実装バグ	仕様不一致・抜け
技法	経路網羅など	同値分割・境界値など
実行者	開発者中心	QAや利用者視点も取りやすい

3 ブラックボックステストで何ができるようになった？



仕様どおりかを外側から検証できる



実装変更に近いテストにしやすい



開発以外の視点を取り込みやすい

関連キーワード

関連キーワード



ホワイトボックステスト

中身を見る対の立場。

P.515



同値分割

黒箱で使う代表技法。

P.514



E2E Test

黒箱で回すことが多い層。

P.502

テスト設計

ひとことというと

観点・条件・期待結果を整理し、どのテストケースをどの粒度で持つかを決める活動。

1 なぜ生まれた？

やみくもにテストコードを増やすと、薄いケースだらけで本番の事故は防げない。



2 それまでどうしていた？

思いついた操作をそのままテストにし、観点の抜けや重複に後から気づいていた。

	実装から逆算したテスト	テスト設計
着手	コードができてから	要件・リスクから逆算
網羅	書いたところだけ	技法で漏れを減らす
保守	変更で大量修正	設計で修正範囲を限定
共有	属人化しやすい	チームで粒度を揃えやすい

3 テスト設計 で何ができるようになった？



何を・どこまで試すか決められる



技法を組み合わせ
て設計できる



テスト工数を見積
もりやすくなる

関連キーワード

関連キーワード



境界値分析

設計で使う代表技法。

P.513



同値分割

同じく代表技法。

P.514



BDD

振る舞いの言葉で設計する流派。

P.507

Jest

ひとことというと

Meta（旧Facebook）発のJavaScriptテストフレームワーク。ランナー・アサーション・モックが一式そろっている。

1 なぜ生まれた？

JSのテストはランナーと断言ライブラリとモックがバラバラで、組み合わせが面倒だった。



2 それまでどうしていた？

Mocha + Chai + Sinonなどを自分で繋いで、設定ファイルと戦っていた。

	寄せ集め構成	Jest
セットアップ	複数ライブラリの配線	一式が最初から入っている
スナップショット	別途用意	標準機能
モック	ライブラリ依存	jest.fn 等が標準
エコシステム	選択が多い	React界限で特に普及

3 Jest で何ができるようになった？



JS/TSのユニットテストをすぐ始められる



スナップショットテストがしやすい



モジュールモックが標準で使える

関連キーワード

関連キーワード



Vitest

Vite親和の高速オルタナティブ。

P.519



Unit Test

Jestの主戦場。

P.500



スナップショットテスト

Jestが広めた手法の一つ。

P.510

Vitest

ひとことというと

Viteと同じパイプラインを使う、Jest互換寄りの高速テストランナー。

1 なぜ生まれた？

Jestは強力だが、現代のViteプロジェクトでは設定の二重管理や起動の重さが気になった。



2 それまでどうしていた？

フロントの開発はViteなのに、テストだけJest + 別変換、というねじれが起きがちだった。

	Jest 中心の構成	Vitest
設定	Jest用に別途	vite.config と共有しやすい
速度	プロジェクト次第で重い	ESM前提で軽快なことが多い
API	デファクトの書き方	Jest互換を意識したAPI
ウォッチ	強い	HMR 感に近い体験を狙う

3 Vitest で何ができるようになった？



Viteプロジェクトでテストを速く回せる



開発とテストの設定を寄せられる



Jest資産からの移行が比較的しやすい

関連キーワード

関連キーワード



Jest

比較対象であり互換の参照点。

P.518



Vite

同じ思想圏のバンドラ / devサーバ。

P.150



Unit Test

主に回すテストの種類。

P.500

Playwright

ひとことという

Microsoft製のブラウザ自動操作ライブラリ。
Chromium / Firefox / WebKit を統一APIで扱える。

1 なぜ生まれた？

E2Eはブラウザ差とフレーク（ちらつき失敗）
がつきもの。



2 それまでどうしていた？

SeleniumやPuppeteerで単一ブラウザを叩き、待機やセレクトに苦労していた。

	従来のブラウザ自動化	Playwright
ブラウザ	ドライバ管理が面倒	主要ブラウザを統一的に
待機	sleepに頼りがち	自動待機が強力
トレース	自前で工夫	トレース・動画等が充実
言語	さまざま	JS/TSほか複数言語公式

3 Playwright で何ができるようになった？



E2Eを安定して書ける



複数ブラウザを同じAPIで試せる



自動待機でフレークを減らせる

関連キーワード

関連キーワード



E2E Test

Playwrightの主用途。

P.502



Puppeteer

Chrome操作の先輩格。

P.521



スモークテスト

薄いE2Eとして回すことも。

P.512

Puppeteer

ひとことという

Google製のライブラリで、DevTools プロトコル 経由に Chrome/Chromium を自動操作する。

1 なぜ生まれた？

Seleniumは重く、Chrome だけ試したい場面では設定が面倒だった。



2 それまでどうしていた？

ブラウザ操作は Selenium が巨艦で、設定やドライバのバージョン地獄がつきものだった。

	Selenium 中心	Puppeteer
対象	多ブラウザ・多言語	Chrome/Chromium に特化
セットアップ	ドライバ管理が必要	ブラウザ同梱で始めやすい
API	WebDriver 抽象	DevTools に近い操作感
得意	広域な互換	Chrome 前提の自動化・計測

3 Puppeteer で何ができるようになった？



Chrome をスクリーンショットで操作できる



スクリーンショットやPDF生成がしやすい



クローラや煙テストの土台にできる

関連キーワード

関連キーワード



Playwright

多ブラウザE2Eの有力株。

P.520



E2E Test

用途のひとつ。

P.502



ヘッドレス

画面なしでブラウザを走らせるモード。

パッケージ

ひとことという

ライブラリやツールを、名前・バージョン・依存関係つきで配布・インストールできる単位にまとめたもの。

1 なぜ生まれた？

便利なコードをコピー＆ペーストで持ち回ると、更新もライセンスも追跡できない。



2 それまでどうしていた？

他プロジェクトのファイルを丸コピーするか、自前で再実装するのが普通だった。

	コピペ / 自前実装	パッケージ
入手	探す・コピーする	レジストリからインストール
更新	手で差し替え	バージョン指定で上げられる
依存	見えにくい	依存関係として明示される
共有	属人的	チームで同じ名前を指せる

3 パッケージで何ができるようになった？



再利用可能な部品
を名前で扱える



公開レジストリ経
由で配布できる



依存関係を明示し
て管理できる

関連キーワード

関連キーワード



依存関係

パッケージが必要とする他パッケージ。

P.551



セマンティックバージョンニング

バージョン番号の約束事。

P.552



npm

JSの代表的なパッケージマネージャ。

P.554

依存関係

ひとことという

あるパッケージやプロジェクトが動作するために必要とする、他のパッケージとの関係。

1 なぜ生まれた？

ライブラリは単独では完結せず、さらに別のライブラリを必要とすることが多い。



2 それまでどうしていた？

「自分のマシンでは動く」が、入れたものが記録されず再現できなかった。

	依存が暗黙	依存関係を宣言
再現性	低い	同じ一覧から再現しやすい
更新影響	見えない	ツリーで影響範囲を追える
衝突	実行時に発覚	解決方針をツールが示す
本番	手作業で揃えがち	ロックファイルで固定できる

3 依存関係 で何ができるようになった？



必要な部品を明示できる



間接依存まで追跡できる



バージョン衝突に向き合える

関連キーワード

関連キーワード



パッケージ

依存の単位。

P.550



ロックファイル

解決結果を固定するファイル。

P.553



セマンティックバージョンニング

互換の期待値を番号で表す。

P.552

セマンティックバージョンニング

ひとことというと

バージョン番号を major.minor.patch の形で付け、互換性の破り方にルールを持たせる約束 (SemVer)。

1 なぜ生まれた？

「1.2」と「1.10」のどちらが新しいか、更新して安全かも番号からは読み取れなかった。



2 それまでどうしていた？

日付や気分でバージョンを上げ、利用者はリリースノートを読取るしかなかった。

	気分バージョン	SemVer
破壊的変更	番号から不明	MAJORを上げる
後方互換の機能追加	不明	MINORを上げる
バグ修正	不明	PATCHを上げる
依存の指定	難しい	^や~で意図を書ける

3 セマンティックバージョンニングで何ができるようになった？



更新の危険度を番号から推測できる



依存指定にルールを持たせられる



ライブラリ作者と利用者の契約になる

関連キーワード

関連キーワード



ロックファイル

SemVer範囲の解決結果を固定。

P.553



依存関係

バージョン範囲が効く場所。

P.551



npm

SemVerが日常になる世界。

P.554

553

ロックファイル

ひとことというと

依存解決の結果（実際に入ったパッケージと版）を記録し、インストールを再現可能にするファイル。

1 なぜ生まれた？

package.json などの範囲指定だけだと、日が変わると別バージョンが入ることがある。



2 それまでどうしていた？

「だいたい同じ依存」で開発し、本番だけ微妙に違うバージョンで落ちる事故が起きていた。

	範囲指定だけ	ロックファイルあり
再現性	インストール時点に依存	同じ結果を再現しやすい
CI	揺れる	固定できる
レビュー	何が入るか見えにくい	差分で更新内容が見える
コミット	し忘れることも	リポジトリに含めるのが基本

3 ロックファイルで何ができるようになった？



依存ツリーを固定できる



環境差による「自分だけ動く」を減らせる



更新内容を差分としてレビューできる

関連キーワード

関連キーワード



依存関係

ロックされる対象。

P.551



npm

package-lock.json の世界。

P.554



pnpm

pnpm-lock.yaml の世界。

P.556

ひとことというと

Node.js 付属のパッケージマネージャ。npm レジストリからパッケージを入れ、scripts も実行する。

1 なぜ生まれた？

JS の再利用部品が増えるにつれ、ダウンロード・依存解決・公開を標準化する道具が必要になった。



2 それまでどうしていた？

スクリプトをファイルで共有するか、Git リポジトリを 직접 参照する運用が多かった。

	手動入手	npm
インストール	ダウンロードして配置	npm install で解決
メタデータ	README 頼み	package.json に集約
実行	パスを覚える	npm scripts で統一
公開	個別配布	レジストリに publish

3 使用例

依存追加と script 実行の基本例。

```
npm install axios
npm run dev
```

関連キーワード

関連キーワード



yarn
代替クライアントの代表格。

P.555



pnpm
ディスク効率に振る代替。

P.556



ロックファイル
package-lock.json。

P.553

555

yarn

ひとことについて

Facebook 発の JavaScript パッケージマネージャ。
高速化やロックファイル体験を武器に普及した。

1 なぜ生まれた？

npm の当時の速さ・決定性・DX に不満があり、
代替クライアントが求められた。



2 それまでどうしていた？

npm install の遅さと、環境差によるずれが日常だった（当時）。

	当時の npm 体験	yarn
ロック	弱かった時期がある	yarn.lock で強く固定
速さ	遅く感じる事が	キャッシュや並列で改善を狙う
UI	素朴	進捗やワークスペース体験を強化
互換	—	npm レジストリを利用可能

3 使用例

依存追加と script 実行の基本例。

```
yarn add axios
yarn dev
```

関連キーワード

関連キーワード



npm

比較対象でありエコシステムの中心。

P.554



pnpm

もう一つの有力代替。

P.556



ロックファイル

yarn.lock。

P.553

pnpm

ひとことというと

コンテンツアドレス可能なストアとシンボリックリンクで、依存を効率的に入れる JavaScript パッケージマネージャ。

1 なぜ生まれた？

プロジェクトごとに `node_modules` が肥大化し、同じ版の複製がディスクを圧迫した。



2 それまでどうしていた？

npm/yarn の flat な `node_modules` は便利だが、幽霊依存（書いてもないのに require できる）も生んだ。

	従来の <code>node_modules</code>	<code>pnpm</code>
ディスク	複製が多い	ストア共有で節約
幽霊依存	起きやすい	厳格で気づきやすい
ロック	各ツール流儀	<code>pnpm-lock.yaml</code>
モノレポ	可能	<code>workspace</code> が強い

3 使用例

依存追加と script 実行の基本例。

```
pnpm add axios
pnpm dev
```

関連キーワード

関連キーワード

- 

npm
エコシステムの基
準点。
P.554
- 

依存関係
厳格さの対象。
P.551
- 

ロックファイル
`pnpm-lock.yaml`。
P.553

pip

ひとことというと

Pythonの標準的なパッケージインストーラ。PyPIからパッケージを取得して環境に入れる。

1 なぜ生まれた？

Pythonの第三者ライブラリを、手作業でパスに置く運用は限界だった。



2 それまでどうしていた？

ソースをダウンロードして setup.py を叩くか、OSのパッケージに頼っていた。

	手動配置	pip
入手	探して展開	pip install で取得
版管理	曖昧	requirements やロックで固定
環境	システム全体を汚しがち	venv と組み合わせるのが定石
公開	個別	PyPI へアップロードする流れ

3 使用例

Python パッケージを入れる基本例。

```
pip install requests
```

関連キーワード

関連キーワード



pyenv

Python本体のバージョン管理。

P.566



パッケージ

pipが扱う単位。

P.550



ロックファイル

詩やuvなど上位ツール側で強化されがち。

P.553

gem

ひとことという

Rubyのパッケージ（gem）を管理する仕組み。
RubyGemsと、Bundlerによるアプリ単位の固定
がセットで語られる。

1 なぜ生まれた？

Rubyのライブラリ流通が増え、インストールと
バージョン衝突を標準化したかった。



2 それまでどうしていた？

ライブラリを手で配置し、requireパスと格闘していた。

	手動ライブラリ管理	gem / Bundler
インストール	配置とパス設定	gem install / bundle install
アプリの固定	難しい	Gemfile.lock で再現
公開	個別配布	RubyGems.org へ
衝突	実行時に発覚	解決をツールが支援

3 使用例

Rubyのgemを入れる基本例。

```
gem install bundler
```

関連キーワード

関連キーワード



rbenv
Ruby本体のバージョン管理。

P.565



ロックファイル

Gemfile.lock。

P.553



Rails
gem生態系の巨大ユーザー。

P.177

559

cargo

ひとことについて

Rustの公式パッケージマネージャ兼ビルドツール。crates.ioからの依存取得とビルドを一体で担う。

1 なぜ生まれた？

言語が新しくても、依存取得・ビルド・テストがバラバラだと学習コストが跳ねる。



2 それまでどうしていた？

他言語のように、ビルドシステムとパッケージマネージャを別々に学ぶ必要があった。

	ツール分裂モデル	cargo
依存	別ツール	Cargo.toml で宣言
ビルド	別システム	cargo build
テスト	別ランナーが多い	cargo test が標準
公開	手順が多様	cargo publish へ一本化

3 使用例

依存追加とビルドの基本例。

```
cargo add serde
cargo build
```

関連キーワード

関連キーワード



パッケージ

crateと呼ばれる単位。

P.550



ロックファイル

Cargo.lock。

P.553



Rust

cargoの宿主言語。

P.176

go mod

ひとことというと

Goのモジュールモードで依存を管理する仕組み。
go.mod / go.sum が中心になる。

1 なぜ生まれた？

GOPATH時代は配置場所とバージョン管理が独特で、再現や複数版共存がなかった。



2 それまでどうしていた？

すべてのGoコードをGOPATH配下に置き、版は「今取れた最新」に近い感覚になりがちだった。

	GOPATH時代	go mod
配置	GOPATH必須感	どこでもモジュールにできる
版	曖昧になりやすい	go.modで明示
改ざん検知	弱い	go.sumで検証
再現	環境依存	モジュール単位で揃えやすい

3 使用例

依存追加と整理解の基本例。

```
go get github.com/gin-gonic/gin
go mod tidy
```

関連キーワード

関連キーワード

- 

依存関係
go.modに書く対象。
P.551
- 

セマンティックバージョンニング
モジュール版の基本感覚。
P.552
- 

Go
宿主言語。
P.173

Homebrew

ひとことというと

macOS（とLinux）で人気のパッケージマネージャ。brew install で開発ツールを入れられる。

1 なぜ生まれた？

MacでUNIX系ツールを入れようとすると、公式以外の手段がばらばらだった。



2 それまでどうしていた？

ソースビルドや、サイトごとのインストーラを集めるのが日常だった。

	個別インストーラ	Homebrew
入れ方	サイトごとに違う	brew install で統一
更新	個別	brew upgrade でまとめて
アンインストール	残骸が残りやすい	手順が比較的明確
開発体験	環境構築が長い	オンボーディングが短い

3 使用例

macOSでツールを入れる基本例。

```
brew install git
```

関連キーワード

関連キーワード



apt

Debian/Ubuntu側の定番。

P.562



mise

言語ランタイム版管理の上位互換寄り。

P.563



パッケージ

brewが配る単位。

P.550

apt

ひとことという

Debian/Ubuntuなどで使うパッケージ管理コマンド。OSのソフトウェアを安全に入れたり更新したりする。

1 なぜ生まれた？

Linux にソフトを入れる方法がソースビルド中心だと、依存地獄と更新追従がづらい。



2 それまでどうしていた？

tarballを展開して ./configure && make が通過儀礼だった。

	ソースからの導入	apt
依存解決	自分で集める	パッケージマネージャが解決
更新	手作業	apt upgrade で追従
削除	残骸管理が難しい	アンインストール手順がある
信頼	ダウンロード元まちまち	ディストリのリポジトリ

3 使用例

Ubuntu系でパッケージ一覧更新と導入を行う例。

```
sudo apt update
sudo apt install git
```

関連キーワード

関連キーワード

 **Homebrew**
Mac側の類似体験。

P.561

 **パッケージ**
debなどの単位。

P.550

 **Shell**
aptを叩く場所 (Ch12)。

P.601

mise

ひとことというと

asdf 互換の開発環境マネージャ。Node や Python など複数言語のバージョンをまとめて切り替える。

1 なぜ生まれた？

nvm、rbenv、pyenv...とツールが増え、シェル設定がパンパンになった。



2 それまでどうしていた？

言語ごとに別のバージョンマネージャを入れ、それぞれの初期化スクリプトを.zshrcに並べていた。

	言語別ツール乱立	mise
設定	ツールごとに初期化	まとめて管理しやすい
プロジェクト	各ツールの記法	.tool-versions 等で共有
速さ	起動が重くなりがち	体感の軽さを売りにする実装
範囲	言語ごと	複数ランタイム + タスクも

3 使用例

Node.js の版を入れて固定する例。

```
mise use -g node@22
mise use node@22
```

関連キーワード

関連キーワード



nvm

Node特化の先輩。

P.564



rbenv

Ruby特化。

P.565



pyenv

Python特化。

P.566

nvm

ひとことという

Node Version Manager。シェル上で複数の Node.js バージョンをインストールし、切り替えて使う。

1 なぜ生まれた？

プロジェクトごとに要求する Node の版が違い、システムの単一 Node では足りなくなった。



2 それまでどうしていた？

公式インストーラや OS パッケージで入れた Node を、全プロジェクトで共有していた。

	システムに一つの Node	nvm
複数版	難しい	并存して切り替え
プロジェクト	手動で合わせる	.nvmrc で自動寄りに
入れ直し	大ごと	版単位で追加・削除
影響範囲	OS 全体	ユーザー環境に閉じやすい

3 使用例

Node.js の版を入れて切り替える例。

```
nvm install 22
nvm use 22
```

関連キーワード

関連キーワード



mise
多言語をまとめる
上位互換寄り。

P.563



npm
切り替えた Node
に付いてくるマネ
ージャ。

P.554



Node.js
対象ランタイム。

P.143

ひとことについて

複数のRubyバージョンをユーザー環境に入れ、ディレクトリごとに切り替えるツール。

1 なぜ生まれた？

Rails アプリごとに必要なRubyが違い、システムRubyを上げ下げすると他が壊れた。



2 それまでどうしていた？

OSのRubyや、RVMの重い魔法に頼るか、コンテナで逃げるかが選択肢だった。

	システムRuby一本	rbenv
複数版	衝突しやすい	shimで切り替え
プロジェクト	手動	.ruby-versionで指定
思想	—	薄いラップを好む
gem	混ざりやすい	版ごとに分離しやすい

3 使用例

Rubyの版を入れて固定する例。

```
rbenv install 3.3.0
rbenv local 3.3.0
```

関連キーワード

関連キーワード



gem

Rubyパッケージ側。

P.558



mise

rbenv代替になりうる統合ツール。

P.563



Rails

版固定が重要な代表アプリ。

P.177

pyenv

ひとことという

複数のPythonバージョンをインストールし、グローバルやプロジェクト単位で切り替えるツール。

1 なぜ生まれた？

システムPythonはOSが使うため、プロジェクト都合で上げ下げしづらい。



2 それまでどうしていた？

aptのpython3や公式インストーラー本でまかない、venvだけでもがいていた。

	システムPython頼み	pyenv
複数版	つらい	并存して切り替え
OSへの影響	怖い	ユーザー環境に分離
指定	手動	.python-version など
venvとの関係	—	本体版 + venvの二段が定石

3 使用例

Pythonの版を入れて固定する例。

```
pyenv install 3.12.4
pyenv local 3.12.4
```

関連キーワード

関連キーワード

- 

pip
入れたPythonの上で動くインストーラ。
P.557
- 

mise
統合版管理。
P.563
- 

Python
対象言語。
P.172

Terminal

ひとことというと

キーボード入力と文字出力でOSやプログラムとやり取りするための端末（エミュレータ含む）のこと。

1 なぜ生まれた？

GUIだけでは自動化やリモート作業、細かい制御がしづらい。



2 それまでどうしていた？

全部をマウスとウィンドウで操作し、繰り返し作業も手でやっていた。

	GUIだけの操作	Terminal
自動化	マクロや手作業	コマンドとスクリプトで再現
リモート	画面転送が重いことも	SSHで文字だけ届く
記録	操作履歴が残らない	履歴やログに残しやすい
学習曲線	低い入口	最初は記号の海

3 Terminal で何ができるようになった？



文字コマンドでOSを操作できる



同じ操作をスクリプトにできる



リモートサーバー作業の入口になる

関連キーワード

関連キーワード



Shell

コマンドを解釈して実行するプログラム。

P.601



Bash

定番のシェル。

P.602



Ghostty

端末エミュレータの一例。

P.617

601 | Shell

ひとことというと

ユーザーが打ったコマンド行を解釈し、プログラム起動やパイプなどの制御を行うインターフェースプログラム。

1 なぜ生まれた？

カーネルに直接話すのはつらい。人間向けの対話層として、コマンドを受け付ける殻が必要だった。



2 それまでどうしていた？

決まったメニュー操作か、個別アプリのGUIだけが入口だった。

	メニュー操作中心	Shell
表現力	用意された操作	コマンドの組み合わせが自由
パイプ	アプリ間が閉じる	標準入出力でつなげる
環境	見えにくい	変数やPATHで制御
種類	—	Bash / Zsh など選択できる

3 Shell で何ができるようになった？



コマンド行を解釈して実行できる



パイプやリダイレクトで処理を組める



シェルスクリプトで手順をファイル化できる

関連キーワード

関連キーワード



Bash

広く使われるシェル。

P.602



Zsh

macOSデフォルトになったシェル。

P.603



PATH

コマンド検索パス。

P.604

Bash

ひとことというと

Bourne Again SHell。Linuxやスクリプト文化で標準に近い位置にいるシェル。

1 なぜ生まれた？

古いBourne shellの互換を保ちつつ、実用的な機能を足した対話・スクリプト用シェルが必要だった。



2 それまでどうしていた？

shの最小主義か、ベンダーごとの独自シェルに分かれていた。

	素のsh / 独自シェル	Bash
普及	環境差が大きい	Linuxで事実上の共通語
スクリプト	機能が限られることも	配列や関数等が実用的
学習資源	分散	事例と記事が圧倒的
対話	素朴	補完や履歴も実用レベル

3 Bash で何ができるようになった？



シェルスクリプト
を書ける



サーバー作業の共
通土台になる



短い自動化をすぐ
回せる

関連キーワード

関連キーワード



Zsh

対話体験で選ばれ
やすい兄弟。

P.603



Shell

上位概念。

P.601



Cron

Bashスクリプト
の実行タイミング
役。

P.609

Zsh

ひとことというと

高機能なUNIXシェル。強力な補完やテーマ文化で、対話利用のデファクト寄りになった。

1 なぜ生まれた？

毎日打つシェルなら、補完・履歴・見た目の快適さが生産性に直結する。



2 それまでどうしていた？

サーバーではBash、手元もBashのまま、補完の弱さを我慢していた。

	Bash 中心の対話	Zsh
補完	十分だが素朴	強力で拡張しやすい
カスタム	可能	フレームワーク文化が厚い
macOS	かつてはBash	近年はZshがデフォルト
スクリプト移植	—	Bashとの差分に注意

3 Zsh で何ができるようになった？



日々のコマンド入力を快適にできる



プラグインで体験を拡張できる



プロンプトやテーマを整えられる

関連キーワード

関連キーワード



Bash

スクリプト共通語。

P.602



Terminal

Zshを表示する窓。

P.600



PATH

設定をいじりがちな場所。

P.604

PATH

ひとことというと

シェルがコマンド名だけで実行ファイルを探すと
き、順番に見て回るディレクトリのリスト（環境
変数）。

1 なぜ生まれた？

毎回フルパスを打つのはつらい。一方で同名コ
マンドが複数あると、どれが起動するかも制御
したい。



2 それまでどうしていた？

実行ファイルの場所を全部覚えて、絶対パスで起動していた（あるいは起
動できずにいた）。

	フルパス必須	PATH
起動	場所を指定	名前だけで探せる
優先順位	—	左（先）に書いた方が 勝つ
トラブル	見つからない	PATH漏れが原因の常 連
版管理	—	shimの仕組みと組み 合わさる

3 PATH で何ができるようになった？



コマンドの探索順
序を制御できる



command not
foundを切り分け
られる



ツールのバージョ
ン切替と相性が良
い

関連キーワード

関連キーワード



Environment Variable

PATHもその一種。

P.606



Shell

PATHを見てコマ
ンドを探す主体。

P.601



mise

shim経由でPATH
体験に介入
（Ch11）。

P.563

SSH

ひとことというと

Secure Shell。ネットワーク経由で遠隔マシンに安全にログインしたり、コマンドを実行したりするプロトコル/道具。

1 なぜ生まれた？

遠隔管理でtelnetのような平文通信は盗聴に弱い。



2 それまでどうしていた？

同じ部屋のコンソールか、平文の遠隔ログイン、あるいはVPN + 別手段に頼っていた。

	平文リモート/現地作業	SSH
通信	覗かれやすい	暗号化される
認証	パスワードのみが多い	鍵認証が定石
用途	ログイン中心	ポート転送やファイル転送も
自動化	難しい	CIやデプロイの土台になる

3 使用例

遠隔サーバーへ接続する基本例。

```
ssh user@example.com
```

関連キーワード

関連キーワード



Terminal

SSHセッションを映す窓。

P.600



VPN

網全体をトンネルする別手段 (Ch2)。

P.40



Daemon

sshd として常駐する側。

P.610

Environment Variable

ひとことというと

OSやシェルがプロセスに渡す、名前=値の設定情報。コードを変えずに振る舞いを切り替えるのに使う。

1 なぜ生まれた？

環境ごとに接続先や秘密情報を切り替えたいが、コードに直書きすると危険で配布できない。



2 それまでどうしていた？

設定をソースコード定数や、マシンごとの手作り設定ファイルに埋め込んでいた。

	コードに直書き	環境変数
環境差	ビルドや分岐が増える	値だけ差し替え
秘密情報	リポジトリに混入しやすい	環境側に置ける
継承	—	子プロセスへ渡せる
可視性	コードを読めば見える	実行環境を見ないと分からない

3 Environment Variable で何ができるようになった？



設定をコードから外に出せる



開発 / 本番で値を切り替えられる



秘密情報の取り回し口ができる

関連キーワード

関連キーワード



.env

環境変数をファイルで扱う慣習。

P.618



PATH

代表的な環境変数。

P.604



Process

変数を受け取る実行主体。

P.607

Process

ひとことというと

実行中のプログラムのインスタンス。メモリ空間やPIDなどの資源を持ち、OSに管理される。

1 なぜ生まれた？

同じプログラムでも、同時に複数動かしたり、個別に止めたりしたい。



2 それまでどうしていた？

コンピュータは一度に一つの作業、あるいはアプリ単位の粗い管理しか意識しなかった。

	アプリ = 動いているもの	Process
識別	ウィンドウの見た目	PIDなどで一意に管理
隔離	弱い理解	メモリ空間が分かれる
終了	閉じる	シグナルで制御できる
親子	見えにくい	起動関係を追える

3 Process で何ができるようになった？



動いているプログラムを個体として扱える



問題のプロセスだけを調査・終了できる



サーバー上の多重実行を理解できる

関連キーワード

関連キーワード



Thread

プロセスの中の実行の流れ。

P.608



Daemon

常駐するプロセスの形態。

P.610



SSH

遠隔でプロセスを扱う入口。

P.605

Thread

ひとことというと

プロセス内で並行に走れる実行の流れ。メモリ空間を共有しつつ、スタックなどを別々に持つ。

1 なぜ生まれた？

待ち時間のあいだも他の作業を進めたいが、プロセスを増やすと重くて共有も面倒。



2 それまでどうしていた？

重い処理はプロセスを増やすか、順番待ちするか、のどちらかになりがちだった。

	プロセス増殖 / 逐次	Thread
生成コスト	高い	相対的に軽い
メモリ	基本分離	共有が前提
同期	IPCが必要	ロックなど社内同期が必要
失敗の影響	プロセス単位	共有破壊が響きやすい

3 Thread で何ができるようになった？



待ちを隠して並行処理できる



マルチコアを活かしやすい



共有データの競合に向き合える

関連キーワード

関連キーワード



Process

スレッドの入れ物。

P.607



Lock

共有資源の交通整理 (Ch6)。

P.95



非同期

並行の別モデル (Ch7)。

P.138

Cron

ひとことというと

指定した時刻・周期でコマンドやスクリプトを自動実行する、UNIX系の定番ジョブスケジューラ。

1 なぜ生まれた？

バックアップや集計を人の手と目覚ましに頼ると忘れる。



2 それまでどうしていた？

担当者が出勤してから手でスクリプトを回すか、常駐プログラムを自作していた。

	人手 / 自作常駐	Cron
起動タイミング	人が覚える	crontabで宣言
記述	口頭・メモ	分時日月曜のフィールド
失敗通知	気づきにくい	メールやログ設計が必要
分散	1台前提になりやすい	どのマシンのcronかを意識

3 Cron で何ができるようになった？



定期バッチを自動実行できる



運用タスクを宣言的に残せる



サーバーメンテの定番手段になる

関連キーワード

関連キーワード



Daemon

cron自体も常駐サービス。

P.610



Bash

よく実行されるスクリプトの中身。

P.602



Process

起動される実行主体。

P.607

Daemon

ひとことというと

ユーザー操作に紐づかず、バックグラウンドで待ち受け・定期作業などを行う常駐プロセス。

1 なぜ生まれた？

WebサーバーやSSHのように、「誰かがログインしている間だけ」では足りないサービスがある。



2 それまでどうしていた？

必要なときだけプログラムを起動し、終わったら終了——が基本モデルだった。

	都度起動のプログラム	Daemon
寿命	作業と一緒に終わる	長く常駐する
対話	ユーザーと向き合う	裏で待ち受ける
管理	手元で起動	サービスマネージャで管理
例	CLIツール	sshd / cron / nginx 等

3 Daemon で何ができるようになった？



サービスを常時待ち受けさせられる



定期・イベント駆動の裏方を置ける



systemdなどで起動管理できる

関連キーワード

関連キーワード



Process

デーモンもプロセスの一種。

P.607



SSH

sshd が代表例。

P.605



Cron

定期実行のデーモン利用。

P.609

正規表現

ひとことというと

文字列のパターンを簡潔に記述し、検索・置換・抽出に使う表記法（と、それを解釈するエンジン）。

1 なぜ生まれた？

ログやテキストから「メールっぽいもの」「数字3桁」を人手で拾うのは限界。



2 それまでどうしていた？

完全一致検索か、プログラミングのループで一文字ずつ判定していた。

	完全一致 / 手書き判定	正規表現
柔軟性	低い	パターンで幅を持たせられる
記述量	コードが長くなりがち	短い式で表現できることも
可読性	—	書きすぎると暗号になる
用途	単純検索	grep / バリデーション / 置換

3 正規表現 で何ができるようになった？



複雑な文字列条件を短く書ける



ログ抽出や一括置換ができる



入力バリデーションの道具になる

関連キーワード

関連キーワード



grep系

正規表現検索の代表コマンド（近い言葉）。



Vim

正規表現置換が強い編集器。

P.615



Shell

パイプと組み合わせる使う舞台。

P.601

パーミッション

ひとことというと

ファイルやディレクトリに対し、ユーザ/グループ/その他が読み書き実行できるかを表す権限設定。

1 なぜ生まれた？

多人で使うUNIXでは、他人の秘密ファイルを読めたり、システムファイルを消せてしまっただけで困る。



2 それまでどうしていた？

単一ユーザー前提か、物理的に触れる人だけが操作できる、という世界だった。

	権限の概念が薄い	パーミッション
制御	ほぼ全部可能	rwXを主体に制限
表記	—	記号（rwX）や8進数（755）
所有	曖昧	user/groupが紐づく
事故	消し放題	権限不足 / 過剰の両方に注意

3 パーミッション で何ができるようになった？



ファイルアクセスを主体ごとに制限できる



chmod/chownで制御できる



サーバー公開時の最小権限を設計できる

関連キーワード

関連キーワード



SSH

秘密鍵ファイルの権限が厳しい。

P.605



.env

中身を守る権限設計が重要。

P.618



Process

どのユーザー権限で走るのが効く。

P.607

613

ping

ひとことというと

ICMPなどで相手ホストに応答を求め、到達性と往復時間を見る基本的なネットワーク診断コマンド。

1 なぜ生まれた？

「繋がらない」の原因が相手ダウンなのか、自分側なのか、経路なのか切り分けたい。



2 それまでどうしていた？

アプリを開いて失敗するか、人に「今どう？」と聞くか、しかなかった。

	アプリ失敗だけ見る	ping
確認粒度	アプリ全体	ホスト到達の最小単位
遅延	体感	RTTが数値で見える
切り分け	曖昧	ネットワーク層の第一手
限界	—	ICMPが塞がれると無力

3 使用例

到達性を確認する基本例。

```
ping google.com
```

関連キーワード

関連キーワード



curl

アプリ層の到達確認。

P.614



IP Address

pingの宛先（Ch2）。

P.29



DNS

名前解決の次に疑うポイント（Ch2）。

P.27

ひとことというと

HTTPなど様々なプロトコルでデータを送受信できるコマンドラインツール。API確認の定番。

1 なぜ生まれた？

ブラウザなしで、HTTPのリクエスト/レスポンスを再現・確認したかった。



2 それまでどうしていた？

ブラウザの開発者ツールか、専用GUIクライアントに頼っていた。

	ブラウザ / GUIクライアント	curl
再現	手順が重い	一行で同じリクエスト
自動化	しづらい	シェルからすぐ呼べる
ヘッダ	GUIで設定	-Hなどで明示
学習	画面操作	HTTPの生に近い感触

3 使用例

HTTP GET と JSON POST の基本例。

```
curl https://example.com
curl -X POST https://example.com/api -H "Content-Type: application/json" -d '{"name":"mido"}"
```

関連キーワード

関連キーワード



HTTP

curlが最もよく話す相手 (Ch2)。

P.25



ping

もっと下の層の生存確認。

P.613



API

curlで試す対象 (Ch4)。

P.61

ひとことというと

モード型の高機能テキストエディタ。サーバー上での編集や、キー操作での高速編集文化の象徴。

1 なぜ生まれた？

遠いサーバーにはGUIエディタが無いことが多い。



2 それまでどうしていた？

nanoのような簡易編集か、ファイルを手元に持ち帰って編集して戻していた。

	簡易編集 / 往復転送	Vim
操作	メニューや矢印	モード + キーバインド
学習	低い	初期コストが高い
速度	普通	慣れると手が速い
存在感	—	サーバーにほぼ必ずある文化

3 使用例

ファイルを開く最小例。終了は :q、保存して終了は :wq。

```
vim README.md
```

関連キーワード

関連キーワード



Terminal

Vimが動く場所。

P.600



SSH

Vimが真価を発揮する遠隔作業。

P.605



正規表現

置換でよく使う道具。

P.611

Markdown

ひとことでいうと

シンプルな記号で見出しやリストを表し、HTML などへ変換できる軽量マークアップ。

1 なぜ生まれた？

文書をリッチにしたいが、Wordや生HTMLは重すぎる／うるさすぎる。



2 それまでどうしていた？

プレーンテキストの無機質さか、WYSIWYGの重いファイルかの二択になりがちだった。

	プレーン / Word / HTML	Markdown
読みやすさ	記号が多いか・バイナリ	テキストのまま構造が見える
差分	つらいことも	Gitと相性が良い
変換	—	HTMLやPDFへ落とせる
方言	—	Flavor 差には注意

3 誰がいつ作った？

作った人 John Gruber

時期 2004 年

文脈 HTMLより軽く書ける文書記法として公開された。

関連キーワード

関連キーワード



HTML

よく変換される先 (Ch7)。

P.121



Vim

Markdownを編集する道具の一例。

P.615



Repository

README.mdの置き場 (Ch1)。

P.1

617 Ghostty

ひとことというと

GPUなどを活用して滑らかさと速さを狙った、比較的新しいクロスプラットフォームな端末エミュレータ。

1 なぜ生まれた？

ターミナルは毎日開く仕事道具なのに、描画の遅れや設定の古臭さがストレスになる。



2 それまでどうしていた？

OS標準端末や、長年使われてきた定番エミュレータで十分、と我慢する選択が多かった。

	標準 / 旧来の端末	Ghostty
描画	十分だが遅延を感じることも	速さ・滑らかさを売りにする
設定	環境ごとに流儀	現代的な設定体験を狙う
互換	—	既存のシェルやTUIとそのまま共存
位置づけ	付属品	選ぶ趣味道具

3 Ghostty で何ができるようになった？



日常のターミナル体験を快適にできる



描画や応答の体感を改善できる



既存のシェル文化のまま乗り換えられる

関連キーワード

関連キーワード



Terminal

Ghosttyが実装する役割。

P.600



Shell

中で動く通訳。

P.601



Zsh

きれいなプロンプトと相性よし。

P.603

.env

ひとことという

環境変数を KEY=VALUE 形式で並べた設定ファイル。アプリ起動時に読み込んで使う慣習的な仕組み。

1 なぜ生まれた？

環境変数を毎回手で export するのはつらいし、チームで共有したい「例」も必要。



2 それまでどうしていた？

シェルのプロファイルに直書きするか、デプロイ先の管理画面にだけ値を置いていた。

	手 export / 画面だけの設定	.env
ローカル開発	手順が属人的	ファイルを置けば揃いやすい
共有	しづらい	.env.example でキーだけ共有
危険	—	本物の .env を Git に上げる事故
本番	ファイル置きがち	シークレットマネージャへ寄せるのが本式

3 .env で何ができるようになった？



環境変数をファイルで扱える



開発環境の立ち上げを揃えやすい



キー一覧を example で共有できる

関連キーワード

関連キーワード



Environment Variable

.env が注入する先。

P.606



パーミッション

ファイルを守る権限。

P.612



Secret

本番での本式管理（セキュリティ章）。

DevOps



開発
運用

ひとことという

開発（Dev）と運用（Ops）を一体で動かし、高速かつ安定したリリースを継続的に実現する文化・考え方。

1

なぜ生まれた？

開発と運用の目標が逆方向になり、リリースのたびに摩擦・遅延・障害が繰り返されていた。



2

それまでどうしていた？

Dev（開発）と Ops（運用）は別チームで、「作る人」と「動かす人」の間に高い壁があった。

	従来の体制	DevOps
チーム構成	Dev と Ops が分断	一体で動く
リリース頻度	数か月に1回・手動	頻繁・自動化
障害対応	責任の押し付け合い	同じゴールで動く
品質管理	後工程でまとめてテスト	常時テスト・監視

3

DevOps で何ができるようになった？



デプロイサイクル
が速くなる



自動化でミスが減
る



開発と運用が同じ
目標で動ける

関連キーワード

関連キーワード



CI/CD

自動ビルド・テスト・デプロイ。

P.705



IaC

インフラをコードで定義・管理する。

P.851



SRE

信頼性に特化した DevOps の実践形。

P.1351

キーワードメモ

「Dev」と「Ops」を合わせた造語。2009年頃から広まった。